

Scalable Accountable Byzantine Agreement and Beyond

Pierre Civit*, Daniel Collins[†], Vincent Gramoli^{‡§}, Rachid Guerraoui*,
Jovan Komatovic*, Manuel Vidigueira*, Pouriya Zarbafian[‡]
*EPFL, [†]Texas A&M University, [‡]University of Sydney, [§]Redbelly Network

Abstract—No t -resilient Byzantine Agreement (or Reliable Broadcast) protocol can guarantee agreement among n correct processes in a non-synchronous network if the actual number of faulty processes f is $\geq n - 2t$. This limitation highlights the need to augment such fragile protocols with mechanisms that detect safety violations, such as forensic support and accountability.

This paper introduces simple and efficient techniques to address this challenge by proposing a new generic transformation, ABC^{++} . The transformation leverages two key primitives: the *ratifier* and the *propagator*. By sequentially composing these primitives with any closed-box Byzantine Agreement (or Reliable Broadcast) protocol, ABC^{++} produces a robust counterpart that provides both (adaptively-secure) forensic support and (1-delayed adaptively-secure) accountability. The transformation incurs a subquadratic additive communication overhead, with only 1 round of overhead for decision and forensic support, and 2 additional rounds for detection in case of a safety violation (or $O(\log n)$ additional rounds with optimized communication).

The generality of ABC^{++} offers a compelling general alternative to the subquadratic forensic support solution by Sheng et al. (FC’23) tailored to HotStuff-like protocols, while being more efficient than the (strongly-adaptively-secure) quadratic ABC accountable transformation (IPDPS’22, JPDC’23). Moreover, it provides the first subquadratic accountable Byzantine Agreement (or Reliable Broadcast) protocols against a (1-delayed) adaptive adversary.

Finally, any subquadratic accountable Reliable Broadcast protocol can be integrated into the τ_{scr} transformation (ICDCS’22) to produce an improved variant, τ_{scr}^{++} . This new version compiles any deterministic (and even beyond) protocol into its accountable counterpart with subquadratic multiplicative communication overhead, significantly improving upon the original quadratic overhead in τ_{scr} .

1. Introduction

Byzantine agreement (BA) protocols enable a group of n processes to reach consensus even when f processes are faulty and can deviate arbitrarily from the protocol. These protocols are crucial for various distributed systems, including state machine replication (SMR), blockchain systems, and secure multi-party computation (MPC). Traditional BA protocols guarantee consensus as long as the

number of faulty processes is below a certain threshold $t < n/3$. However, they fall short in scenarios where this threshold is exceeded. Typically, due to classic partitioning arguments [1], no t -resilient BA protocol can prevent disagreement during a period of asynchrony if $f \geq n - 2t$ [2], [3]. This fundamental limit has motivated the community to introduce mechanisms that, at least, allow detecting faulty processes which are responsible for disagreements.

Forensic support [4], [5], [6], [7] ensures that in case of a disagreement, some correct processes will collectively hold different pieces of information such that when these pieces are combined, they provide irrefutable proof of misbehavior by a significant number of faulty processes. The collective nature of forensic support requires communication overhead to exchange and combine these pieces of evidence simply to realize that safety has been violated, let alone to expose malicious parties. Accountability [8], on the other hand, is a stronger property. It provides irrefutable proof of misbehavior directly to every correct process, without the need to exchange and combine separate pieces of information¹. Recent research has introduced various mechanisms to achieve these properties, but existing solutions come with notable downsides.

The solution of [5] provides forensic support by relying on *transition certificates* to identify nodes that “voted behind their lock” (see Appendix B) in a player-replaceable² version of HotStuff [13], [14], which leads to forensic evidence consisting of full transcripts of received messages, potentially polynomial in κ . Besides not ensuring accountability, this makes dissemination (not considered in [5]) and verification inherently costly, and no prior method extended it to an accountable protocol without the linear overhead of [15], hence superquadratic.

On the other hand, the ABC transformation proposed in [2], [11] offers a more generic approach to ensure accountability, even in the presence of a strongly-adaptive adversary, but suffers from a significant drawback: dissemination is achieved via all-to-all communication, resulting in $\Omega(n^2)$ communication complexity. In this scheme, each process must in particular verify a linear number of signatures, impeding scalability.

1. The original definition of accountability in [8] involves a judge-based model. Following [9], [10], [11], [12], we adopt the convention that *all* correct judges (i.e., processes) must reach a verdict.

2. We elaborate on this notion in the related work section.

In this paper, we decompose the $\mathcal{ABC}$ transformation’s *accountable confirmer* further into two new primitives: the *ratifier* and the *propagator*. This apparently innocuous split yields simpler and stronger observations.

Contributions. Let κ and λ be computational and statistical security parameters, respectively. Our contributions are as follows:

- We define the *ratifier*, a minimal functionality for forensic support. Correct processes can use it to generate forensic evidence. Its simplicity enables an adaptively-secure one-round implementation, incurring $O(\lambda n)$ message complexity (each message of $O(\kappa)$ bits), using standard Algorand-style sortition [16] on top of a VRF setup [17].

Crucially, the ratifier forces the adversary, if it wishes to cause disagreement, to equivocate at a single, well-defined round, independent of the compiled protocol. The resulting evidence is just a compact multi-signature plus a “proof-of-eligibility”, which is lightweight to propagate and check, in contrast to [5].

- We define the *propagator*, a minimal functionality ensuring the dissemination of certificates to all correct nodes. Unlike the all-to-all communication of [11], our propagator achieves subquadratic complexity by revisiting the ERFlood protocol [18] under the 1-delayed-adaptive adversary model, where corruption decisions are postponed just long enough to let in-transit messages be delivered.

This design required and motivated the notion of *propagation-friendliness*, which ensures correctness and scalability by preventing a fake-certificate flooding attack that would arise in more naive designs. It also suggests efficient and compact eligibility proofs via concretely-efficient aggregate lottery functionality, such as Jackpot [19].

We present two alternative subquadratic implementations: (1) $O(\lambda)$ per-process message complexity with $O(\log(n/\lambda))$ expected rounds between a safety violation and accountable detection; or (2) $O(\sqrt{n\lambda})$ per-process message complexity with 2 rounds to detection. Each message has size $O(\lambda \log n + \kappa)$ when using Jackpot proofs.

- We show that sequentially composing the ratifier and the propagator yields an accountable confirmer, as formalized in [11]. This primitive can then be composed with any protocol solving an easily accountable agreement task, such as Byzantine Agreement, Reliable Broadcast, or Consistent Broadcast, using the $\mathcal{ABC}$ transformation. The resulting protocol inherits forensic support, accountability, and the correctness properties of the underlying protocol Π , with complexity and security guarantees derived from the ratifier, the propagator, and Π itself. This yields the first subquadratic accountable protocols for Byzantine Agreement (and Reliable Broadcast, and Consistent Broadcast) as summarized in Table 1.
- We present the $\mathcal{ABC}^{++}$ transformation within the

Accountable Universally Composable (AUC) framework [20], providing strong compositional guarantees for the broad class of easily accountable agreement functionalities, namely the UC abstractions of the easily accountable agreement tasks introduced in [11]. This class not only includes standard primitives such as Byzantine Agreement, Reliable Broadcast, and Consistent Broadcast, but also encompasses other specialized abstractions like one-to-many zero-knowledge proof, which serves as the zero-knowledge counterpart of Reliable Broadcast and is occasionally employed in MPC protocols [21], [22], [23].

- We evaluate the exact constants involved in our complexity analysis to demonstrate the practical feasibility of our solution. Additionally, we implement in Rust the cryptographic certificate logic, showing that aggregation and verification, performed once per decision, can be executed in under 50 ms, with failure probability below 2^{-40} .

TABLE 1: Comparison of Accountability and Forensic Detection Schemes. **Gen./Acc.:** Generality of supported protocols and whether accountability is ensured (Y/N). ‘SMR’ corresponds to a family of HotStuff-like protocols. **Comm.:** Asymptotic communication overhead. **Rnd:** Round complexity overhead. t_{acc} : Maximum number of faults for which detection is guaranteed. **Detect.:** Number of faulty processes that can be held accountable when $t < f \leq t_{acc}$. **Adapt.:** adversary adaptivity — S/W: strong / weak adaptive (with / without after-the-fact removal, i.e., the adversary can remove pending messages from freshly corrupted processes [24]), D: 1-delayed adaptive.

Work	Gen./Acc.	Com.	Rnd	t_{acc}	Detect.	Adapt.
[11]	Any/Y	$+\Omega(n^2)$	+1	n	$n/3$	S
[15] (O)	“SMR”/Y	$+\Omega(n^3)$	+0	n	$n/3$	S
[15] (C)	“SMR”/Y	$\times\Omega(n)$	+0	n	$n/3$	S
[4]	None/N	$\Omega(n^2)$	N/A	$2n/3$	$n/3$	S
[5]	None/N	$o(n^2)$	N/A	$2n/3$	$\lambda/3$	W
$\mathcal{ABC}^{++}$	Any/Y	$+o(n^2)$	+1	$n-\Theta(n)$	$\lambda/3$	D

Applications. $\mathcal{ABC}^{++}$ is directly applicable to protocols whose security reduces to underlying easily accountable agreement tasks, such as those used in scalable and accountable distributed ledgers [25], state machine replication [26], [27], bulletin board services [28], [29], and payment systems [30], [31], [32]. Additionally, Accountable Byzantine Agreement can also be applied to novel problems such as rational agreement [33] and variants of the long-lasting blockchain problem [15], [34], [35], [36].

A complementary line of work focuses on general transformations that bring accountability to a wide class of distributed protocols beyond easily accountable agreement tasks. One of the earliest and most influential contributions in this direction is PeerReview [37], which provides a generic accountability layer inspired by the failure detector paradigm. In PeerReview, accountability is defined as the permanent suspicion of faulty processes, without producing externally-verifiable evidence. Later, the τ_{scr} transformation [12] strengthened this notion by providing full accountability. In this setting, violations of safety yield publicly

verifiable proofs of misbehavior [8], [38], but at the cost of a quadratic multiplicative communication overhead.

We observe that the subquadratic accountable Reliable Broadcast protocols developed in this work can be directly integrated into τ_{scr} to obtain an improved transformation τ_{scr}^{++} . This variant preserves the same accountability guarantees while reducing the communication overhead to sub-quadratic, as detailed in Table 2. These approaches apply to a broad class of protocols that do not naturally fall within the scope of easily accountable agreement tasks. This includes all deterministic protocols that may invoke cryptographic primitives, and even more general settings. Furthermore, previously proposed techniques to preserve hyper-properties (e.g., privacy, fairness, or unpredictability) in transformations à la PeerReview [39], [40], [41] can be naturally adapted to τ_{scr}^{++} , mitigating one of the primary limitations of such generic transformations.

TABLE 2: Comparison of General Accountability Transformations. **Com.** denotes the multiplicative communication overhead; **Ext. Ver.** indicates whether external verifiability is supported; **Adapt.** denotes the corruption model: S = strongly-adaptive, D = 1-delayed, None = static corruption.

Work	Com.	Ext. Ver.	Adapt.
PeerReview [37]	$\times o(n^2)$	N	None
PeerReview' [37]	$\times \Theta(n^2)$	N	S
τ_{scr} [11]	$\times \Theta(n^2)$	Y	S
τ_{scr}^{++} (this)	$\times o(n^2)$	Y	D

Roadmap. In §2, we review the related work. §3 defines the formal system model and the necessary preliminaries. The ratifier and its implementation are introduced in §4, followed by the propagator and its implementation in §5. Both are combined to build the generic ABC^{++} compiler in §6. Generalization to the Universal Composability framework is discussed in §7. §8 conducts an evaluation of our protocol’s cost. Lastly, §9 concludes the work. We discuss τ_{scr}^{++} in Appendix A, while full definitions and proofs are provided in the remaining part of the appendix.

2. Related Work

Verifiable Random Functions (VRFs) and Player-Replaceability. VRFs [17] are cryptographic primitives that generate unpredictable yet verifiable random values, enabling efficient leader election. At each step s , processes use VRFs to determine whether they have been elected as a leader. If elected, the VRF allows the computation of a verifiable proof confirming the legitimacy of the election. In the player-replaceability paradigm [16], [42], [43], a “classic multicast” protocol, where messages are broadcasted publicly via a multicast protocol, is divided into steps that involve either all-to-all communication or leader-to-all communication. For all-to-all steps, a sublinear committee is elected using the VRF primitive to represent the group in communication. Similarly, the leader is elected using the VRF mechanism, which may sometimes elect more than one leader without harming the protocol any more than

having a single malicious leader in the original protocol. To send a message in any round, a player must append a publicly verifiable proof of eligibility, and any message without a valid proof is rejected.

While this solution may seem imbalanced (committee members sending more messages than non-committee members), the multicast primitive can be implemented using a probabilistic flooding protocol [44], which can be made secure against a 1-delayed-adaptive adversary, whose adaptive corruptions are delayed by at least one causal message delivery [18], [45], [46]. However, a broadcast protocol cannot be adaptively-secure with sublinear per-process communication, as the adversary can always immediately corrupt the neighbors of the message’s source.

Subquadratic Byzantine Agreement. Achieving subquadratic BA against an adaptive adversary in a non-synchronous network is highly challenging. Unlike in synchrony [47], subquadratic solutions in non-synchronous networks require a trusted setup [48], which goes beyond an ideal authentication functionality. This remains true even when private channels, Common Random Strings (CRS), and succinct non-interactive arguments of knowledge (SNARKs) are assumed to be freely available [49]. The required trusted setup seems to inherently involve correlated private randomness. Consequently, it is no big surprise that all subquadratic solutions in partial synchrony [5], [14], [16], [24], [49], [50] or full asynchrony [48], [51], [52] rely on the player-replaceability paradigm on top of a VRF-setup. These protocols can also be extended to efficiently handle long input values [53].

Subquadratic Byzantine Reliable Broadcast. As presented in [48], the VRF-based player-replaceability paradigm can be applied to Bracha’s double-echo protocol [54], [55] to achieve subquadratic adaptively-secure performance. Multicast primitives can be implemented using the balanced flooding protocol from [18], but at the price of making the protocol secure against a 1-delayed-adaptive adversary only. Alternatively, at the cost of only providing static security, the Contagion protocol [56] achieves similar guarantees without requiring a VRF setup.

Accountable Byzantine Agreement. It has been observed that any closed-box BA object can be sequentially composed with an *accountable confirmer* to achieve accountable BA [2], [11]. The simplicity of this solution has enabled its formal verification within TLA+ [57]. This generic approach may encourage moving away from specialized solutions designed for specific BA implementations [4], [10], [58], [59], [60], even though some optimizations could avoid the extra round complexity. In this context, [15] proposed a generic transformation applicable to a broad class of HotStuff-like protocols, achieving accountability without incurring any latency overhead. However, this comes at the cost of either a $\Theta(n^3)$ additive communication overhead or a $\Theta(n)$ multiplicative overhead.

The accountable confirmer requires only one round for decision (called post-voting in OFlex [61]) and 1 additional

round for the detection of a potential safety violation, and can be applied to any BA algorithm, regardless of its validity property [62]. For example, it can be used with multi-valued validated BA (MVBA) [63], [64], [65], [66], [67] or agreement on a core set (ACS) [68], [69], [70], [71], also referred to as asynchronous common subset or vector consensus. These (accountable) BA variants can then be employed to implement an (accountable) distributed ledger object (DLO), where processes agree on a (common prefix of a) totally ordered set of transactions [25], [26], [27]. Such transactions can represent messages from an atomic-broadcast primitive or transitions in a state machine, thus solving state-machine replication (SMR). An accountable ledger can be further extended with client logic to implement an accountable *bulletin board* [28], [72], which is commonly used to describe an auditable ledger, e.g. supporting the generation of receipts for successful postings [29].

(Accountable) Finality Gadget. The notion of an (accountable) confirmer [2], [11] (see also post-voting in OFlex [61]) shares similarities with the concept of a finality gadget, a line of work that was initially defined in [73] and originated with Casper [74]. Finality gadgets, also referred to as snap-and-chat protocols [73], provide a flexible BA mechanism by combining two distinct protocols: one optimized for liveness and the other for safety. This results in two confirmation rules: a “liveness-focused” rule and a “safety-focused” rule. Subsequent improvements to the finality gadget presented in [73] have successively enhanced the functionality with predictable validity [75] and accountability [76]. Notably, it was shown in [77] that accountability implies finality.

Synchronous ID-MPC. The classical impossibility of fairness in secure multi-party computation (MPC) when fewer than $n/2$ processes are correct [78] has motivated the study of synchronous MPC with identifiable abort (ID-MPC) [79], [80], [81]. This line of work, originally explored under the notion of covert security [82], [83], culminated in the publicly accountable MPC protocol of [84]. While our setting does not aim to preserve input or output privacy—a central concern in ID-MPC—we focus on asynchronous systems, where omission-sending faults are indistinguishable from network delays, making reliable detection fundamentally more difficult. As such, the two lines of research pursue different goals under different assumptions, and are thus largely incomparable and orthogonal.

3. Preliminaries

Notations. Notations are summarized in Table 3.

Processes and Network. We consider a static set of n processes $\Psi = \{p_1, \dots, p_n\}$. A distributed protocol is defined as the tuple $\Pi = (\Pi_1, \dots, \Pi_n)$, where protocol Π_i is prescribed to process p_i . An adversary (elaborated upon below) can adaptively corrupt processes. Processes that remain uncorrupted are referred to as *correct*, and they

TABLE 3: Summary of Notations.

Notation	Description
n	Number of (#) processes.
f	Actual number of failures.
$\epsilon, \delta, \hat{\delta} > 0$	Small constants for Chernoff bounds
$t = \lceil n(\frac{1}{3} - \epsilon) \rceil - 1$	# tolerated failures (nominal mode)
$t_{acc} = n - \Theta(n)$	# tolerated failures (degraded mode)
$\gamma = \frac{n - t_{acc}}{n}$	Fraction of ensured correct processes.
λ	Expected size of a committee.
κ	The computational security parameter.
$W_{\delta}^{\epsilon, \lambda} = (\frac{2}{3} + \epsilon)(1 - \delta)\lambda$	Size of a VRF-quorum.
$B_{\delta, \hat{\delta}}^{\epsilon, \lambda} = 2W_{\delta}^{\epsilon, \lambda} - (1 + \hat{\delta})\lambda$	Size of VRF-quorums’ intersection.

execute their respective state machines as prescribed by the protocol. Communication occurs over a reliable, authenticated, point-to-point network, meaning that every pair of correct processes can communicate, and a message sent by a correct process to another correct process is eventually delivered. Additionally, the receiver can always identify the sender. We consider a fully-asynchronous network, without upper bound on message delays, but our transformation can apply to (closed-box) protocols that assume partial synchrony [85], where after some unknown Global Stabilization Time (GST), message delays are bounded by a known constant.

Adversary. The adversary is modeled as a state machine that interacts with both the network and the processes. The adversary is assumed to be computationally bounded and unable to break the cryptographic primitives introduced later in the protocol. The adversary controls message scheduling, deciding when messages in transit are delivered. The adversary can corrupt processes dynamically, with f denoting the total number of corruptions. If no restriction is specified, the adversary is adaptive. We also consider 1-delayed-adaptive adversaries [18] (delayed-adaptive for short), where a corruption decision is delayed by the (unknown) maximum time required to deliver a message. Specifically, when a so-far correct process p_i sends a message m to a so-far correct process p_j , the adversary cannot corrupt p_j before it receives and potentially forwards m . Corrupted processes, called Byzantine, are coordinated arbitrarily by the adversary. An adversary defines a probabilistic space over the executions of the global system. The probability is taken over the random coins of the parties, the random coins of the adversary, and the sampling coins from the setup distribution.

We introduce two thresholds: an *optimistic* threshold $t \in [0, n(\frac{1}{3} - \epsilon))$ for some arbitrarily small constant $\epsilon \in \Omega(1)$, and a *pessimistic* threshold $t_{acc} \in [0, n - \Theta(n)]$. We note $\gamma = (n - t_{acc})/n$, the ratio of processes that is assumed to be correct even in bad scenarios. The protocol will be required to solve BA when $f \leq t$, provide forensic support for any f , and ensure accountability when $f \leq t_{acc}$.

Byzantine Agreement. BA protocols allow n processes to agree on a common value even in the presence of faulty processes that may behave arbitrarily. The BA’s specification is given in the Module 1 presented below.

Module 1 Byzantine Agreement $\langle val, \rho \rangle$

Parameters:

- A validity property val . ▷ Details can be found in [62]
- $\rho : f \in [1 : n] \mapsto [0, 1]$, the minimum probability of success

Events:

- *request* $\text{propose}(v_i \in \text{Value})$: a process inputs a value v_i
- *notification* $\text{decide}(v_o \in \text{Value})$: a process outputs a value v_o .

Properties:

Let f be the actual number of failures. With probability at least $\rho(f)$, the following is ensured:

- *Termination*: All honest processes eventually decide on a value.
 - *Agreement*: No two honest processes decide on different values.
 - *Validity*: Any decided value is valid according to val .
-

The module is parametrized with a validity property val and function ρ . A validity property, formalized in [62], maps any input configuration, representing the proposals of the correct processes, with a set of admissible values. For example, the strong unanimity validity property states that if all correct processes unanimously propose a common value, then only that value can be decided. The function ρ maps the actual number of failures f with the corresponding guaranteed probability of success. Let us note that ρ is non-increasing and $\rho(\lceil n/3 \rceil) = 0$. Classically, ρ is associated with a threshold $t' < n/3$, such that $\rho(t') = 1 - o(1)$, while nothing is guaranteed for $\rho(t' + 1)$.

Complexity. We focus on the communication and round complexities of BA under asynchronous and partially synchronous conditions. For each execution, the bit complexity refers to the number of bits sent by correct processes, while the round complexity measures how long the decision process takes based on message delays. In the partially synchronous model, the time and bits exchanged before Global Stabilization Time (GST) are not counted. The expected bit and round complexities of any BA protocol are defined as the worst-case expectations over all adversarial strategies.

Forensic support and accountability. We adopt the nomenclature of accountability outlined in [8], [20]. In this framework, the breach of a specific targeted safety property should result in a *verdict*, identifying the parties responsible for the security violation. Verdicts are computed by *judges* based on *pieces of evidence*. We focus exclusively on *judges* that are *fair* and *public*. A public judge relies solely on publicly verifiable evidence, that can be published by individual parties, allowing external observers to access and validate them, thereby enabling *external verifiability* [8], [20]. A fair judge produces, with all but negligible probability, *fair* verdicts, i.e., verdicts that do not blame honest parties adhering to the protocol. Furthermore, we consider *individual accountability*, where verdicts take the form $\text{dis}(D)$, meaning that all processes in the set D are deemed dishonest.

Jumping slightly ahead, the judge we consider operates on two triplets, π_1 and π_2 , where each $\pi_i = (m_i, Q_i, \sigma_i)$ for $i \in \{1, 2\}$. Here, $Q_i \subseteq \Psi$ represents a subset of processes, m_i is a message, and σ_i is a valid multi-signature certifying that *all* members of Q_i have signed m_i (see next section).

If these signatures are valid and m_1 and m_2 are *conflicting*, meaning they have the same header but different payloads (referred to as *mutant messages* in [86]), then the judge outputs the verdict $\text{dis}(Q_1 \cap Q_2)$. This verdict is fair, since a correct process will never sign conflicting messages.

Forensic support ensures that if a specific safety property is violated, correct processes possess evidence that, when presented to a fair judge, leads to the identification of malicious parties. Accountability strengthens this by eliminating the need for centralized evidence collection. Each correct process independently holds self-verifiable proof, allowing it to act as the judge or directly forward the proof to any external party (client), who can also serve as the judge.

Concretely, let Π be a distributed protocol equipped with some output $\text{yield_certificate}(s \in \text{String})$ or $\text{generate_proof}(s \in \text{String})$, and J a fair judge (for Π). Adapting the presentation of [5], we say that Π provides $(m, k, d, J, \text{valid}(\cdot))$ -*forensic support for safety property* P , if the violation of P with $f \leq m$ implies, except with negligible probability, the existence of a set of k honest processes $(p_{j_1}, \dots, p_{j_k})$ that have triggered $\text{yield_certificate}(s_1), \dots, \text{yield_certificate}(s_k)$ respectively, such that $J(s_1 :: \dots :: s_k)$ returns a set of (undeniably malicious) d distinct processes, while $\text{valid}(s_j) = \text{true}$ for each $j \in [1..k]$. The role of the valid predicate can be ignored for now, but will take on its full meaning when we look at the composition of the ratifier and the propagator. Let us note that this definition does not say how the pieces of evidence $s_1, \dots, s_k$ are collected by the judge. We say that Π provides (m, d, J) -*accountability for safety property* P , if the violation of P with $f \leq m$ implies that, except with negligible probability, every correct process $p_i \in \Psi$ eventually triggers $\text{generate_proof}(s_i)$ such that $J(s_i)$ returns a set of (undeniably malicious) d distinct processes. We call *detection-round complexity*, the expected number of rounds that separate a safety violation from its detection.

3.1. Cryptographic Primitives

We now present the cryptographic primitives assumed by our protocol, including hash functions, multi-signature schemes, verifiable committee sampling, and public key infrastructure. Let κ be the security parameter.

Hash. We assume a collision-resistant hash function hash that maps any value into a sequence of $O(\kappa)$ bits.

Multi-signature. We assume the existence of a dynamic accountable non-interactive multi-signature scheme [87], [88], [89], [90], [91] (DANMSS) defined as a tuple of polynomial-time algorithms

$$\text{DANMSS} = (\text{setup}, \text{kg}, \text{sign}, \text{cb}, \text{ver})$$

with the following functionalities:

- $\text{setup}(1^\kappa) \rightarrow \text{pp}$: On input a security parameter κ , outputs public parameters pp . These parameters are implicitly provided to all other algorithms.

- $\text{kg}() \rightarrow (\text{pk}_i, \text{sk}_i)$: Key generation outputs a fresh public/secret key pair $(\text{pk}_i, \text{sk}_i)$ for signer p_i .
- $\text{sign}(\text{sk}_i, m) \rightarrow \sigma_i$: Signs a message $m \in \{0, 1\}^{\text{poly}(\kappa)}$ using secret key sk_i , yielding a signature σ_i .
- $\text{cb}(m, \{(\text{pk}_i, \sigma_i)\}_{p_i \in G}) \rightarrow \sigma/\perp$: Combines a set of individual signatures from a group of signers G on the message m into a single signature σ , or returns $\perp$ if verification fails.
- $\text{ver}(m, \sigma, \{\text{pk}_i\}_{i \in G}) \rightarrow \{0, 1\}$: Verifies the (potentially combined) signature σ on message m with respect to the group G of public keys.

The scheme satisfies the following security guarantees, except with negligible probability:

- **Unforgeability** No (PPT) adversary can forge a valid aggregate signature on a message m under a set of public keys, unless it queried the signing oracle for each key.
- **Robustness** The scheme guarantees correct behavior in the presence of potentially adversarial signers, as follows:
 - 1) The verification algorithm $\text{ver}(m, \sigma_i, \text{pk}_i)$ accepts any signature σ_i honestly produced via $\text{sign}(\text{sk}_i, m)$ for any $i \in [n]$.
 - 2) For any message m and set G of public keys, if each (pk_i, σ_i) passes individual verification, then the combination $\text{cb}(m, \{(\text{pk}_i, \sigma_i)\}_{i \in G})$ outputs a multi-signature σ that is accepted by $\text{ver}(m, \sigma, \{\text{pk}_i\}_{i \in G})$.

This property ensures that even in the presence of malicious signers, an honest aggregator can compute a valid aggregate signature as long as the individual signatures verify correctly.

Public Key Infrastructure (PKI). We assume a (synchronous) bulletin-board PKI model, where each party independently generates its private signing key(s) and posts the corresponding public verification key(s) to a public bulletin board. The adversary may adaptively corrupt parties and, upon corruption, may modify their keys based on the complete public setup, including the verification keys of honest parties and any available common reference string (CRS), if present. The synchronous aspect of the bulletin-board PKI lies in the existence of a fixed time τ_0 , after which any missing publication is definitively interpreted as the absence of a key (i.e., mapped to $\perp$). Hence, any correct process must publish its public key(s) before τ_0 to avoid being excluded. The (synchronous) bulletin-board PKI model is equivalent to allowing parties to register arbitrary strings as public keys with the ideal certification authority $\mathcal{F}_{\text{CA}}$ [92] before the starting time τ_0 of the online protocol.

Verifiable Committee Sampling. Using VRFs and a (synchronous) bulletin-board-PKI, it is possible [49] to implement (with a confined bias) validated committee sampling (VCS), which is a primitive that allows processes to elect committees without communication and later prove their election. We follow the presentation of Cohen et al. [93]. VCS provides every process p_i with a private function $\text{sample}_i(s, \lambda)$, which gets a string s (typically the label

of some specific step) and a threshold $\lambda \in [1 : n]$ and returns a tuple $\langle v_i, \sigma_i \rangle$, where $v_i \in \{\text{true}, \text{false}\}$ and σ_i is a proof (of size $O(\kappa)$) that $v_i = \text{sample}_i(s, \lambda)$. If $v_i = \text{true}$ we say that p_i is sampled to the committee for s and λ . The primitive ensures that p_i is sampled with probability λ/n . In addition, there is a public (known to all) function, $\text{committee_val}(s, \lambda, i, \sigma_i)$, which gets a string s , a threshold λ , a process identification i and a proof σ_i , and returns true or false . Finally, we note $C(s, \lambda) = \{p_i \in \Psi \mid \text{sample}_i(s, \lambda) \in \{\langle \text{true}, \sigma \rangle \mid \sigma \in \{0, 1\}^*\}\}$. Intuitively, $C(s, \lambda)$ corresponds to the committee (secretly) elected for step s . Consider a string s . For every $p_i \in \Psi$, let $\langle v_i, \sigma_i \rangle$ be the return value of $\text{sample}_i(s, \lambda)$. The following is satisfied:

- (Correctness) $\text{committee_val}(s, \lambda, i, \sigma_i) = v_i$.
- (Unpredictability) If p_i is correct, then it is infeasible for the adversary to compute $\text{sample}_i(s, \lambda)$.
- (Unforgeability) It is infeasible to find $\langle v, \sigma \rangle$ s.t. $v \neq v_i$ and $\text{committee_val}(s, \lambda, i, \sigma) = \text{true}$.

We fix $\epsilon \in (0, \frac{1}{3})$ and $t < (\frac{1}{3} - \epsilon)n$. We fix $\delta, \hat{\delta} \in \Omega(1)$ as arbitrarily small positive constants, which capture deviation parameters in the relevant Chernoff bounds, and define:

- $W_{\delta}^{\epsilon, \lambda} = (\frac{2}{3} + \epsilon)(1 - \delta)\lambda$: conservative size of a “VRF-quorum”.
- $B_{\delta, \hat{\delta}}^{\epsilon, \lambda} = 2W_{\delta}^{\epsilon, \lambda} - (1 + \hat{\delta})\lambda$: conservative size of the intersection of two VRF-quorums.

For readability, we may write W (resp., B) as shorthand for $W_{\delta}^{\epsilon, \lambda}$ (resp., $B_{\delta, \hat{\delta}}^{\epsilon, \lambda}$), when the parameters $\delta, \hat{\delta}, \epsilon$, and λ are clear from context.

The following results, based on standard Chernoff bounds, and proven in Appendix C.1.2, represent VRF-based analogs of quorum system properties:

Theorem 1. Let $H(s, \lambda)$ denote the random variable that returns the set of (so-far honest) processes p_i such that the sampling event $\text{sample}_i(s, \lambda)$ is not preceded by the corruption of p_i in the execution, and let $C(s, \lambda) = \{p_i \in \Psi \mid \text{sample}_i(s, \lambda) = \langle \text{true}, * \rangle\}$, sampled uniformly at random with expected size λ .³

- **Liveness.** With $W = (1 - \delta)(\frac{2}{3} + \epsilon)\lambda$, we have:

$$\Pr[|C(s, \lambda) \cap H(s, \lambda)| \leq W] \leq \exp\left(-\frac{\delta^2}{6}(2 + 3\epsilon)\lambda\right)$$

- **Forensics.** Let $Q, Q' \subseteq C(s, \lambda)$ with $|Q| = |Q'| = W_{\delta}^{\epsilon, \lambda}$. Let $B = 2W_{\delta}^{\epsilon, \lambda} - (1 + \hat{\delta})\lambda$. Then:

$$\Pr[|Q \cap Q'| \leq B] \leq \exp\left(-\frac{\hat{\delta}^2}{2 + \delta}\lambda\right)$$

The Liveness result shows that if a set $C(s, \lambda)$ is assigned a task, with overwhelming probability, a “VRF-quorum” of size $W \approx 2\lambda/3$ will perform it. This mirrors the quorum availability property in classic quorum systems. The Forensic result ensures that if two “VRF-quorums” perform some task, their intersection will contain $\approx \lambda/3$ (accountable) processes, providing the analog of quorum intersection in classic quorum systems.

3. Here, $\Pr$ represents the worst-case probability over adversarial strategies with up to t adaptive corruptions (no liveness guarantees when $f > t$).

4. Ratifier

After some (unreliable) BA instance yielding the (pre)decision v , processes want to answer the question: “Do we agree that we have the same pre-decided value v ?”. Moreover, we want to ensure forensic support in the event of disagreement. We capture these requirements through the *ratifier* primitive, which is specified in Module 2. Naively, this can be implemented by a single round of all-to-all communication where parties sign and broadcast their value v . However, this incurs quadratic communication overhead, which is undesirable in large-scale deployments.

Instead, in our protocol, a VRF-sampled committee of expected size λ broadcasts a SUBMIT message containing (a hash of) their signed pre-decided value to all other processes. Upon receiving sufficiently many ($W \simeq 2\lambda/3$) valid SUBMIT messages supporting the (hash of) pre-decision v , a process can confirm its decision on v and store the signed messages from a “VRF-quorum” of size W for future auditing. When a DANMSS scheme [90] is used, those signatures can be combined into a multi-signature σ , to produce a “VRF-quorum certificate” ($\text{hash}(v), Q, \sigma$). This certificate proves that all the members of Q signed (the hash of) v . When this is sequentially composed with a (closed-box) t' -resilient BA protocol, all properties hold as long as $f \leq \min(t, t')$, as guaranteed by the first Chernoff bound in Theorem 1.

In the event of a disagreement, two processes will store conflicting certificates (h, Q, σ) and (h', Q', σ') , for conflicting (hashes of) values. These two certificates form undeniable proof of misbehavior against the processes in $Q \cap Q'$ that signed conflicting SUBMIT messages. With overwhelming probability, the number of such provably guilty processes is approximately $\lambda/3$, as supported by the second Chernoff bound in Theorem 1. This yields Theorem 2.

Module 2 Ratifier

Parameters:

- Integer $t = \lceil (n(\frac{1}{3} - \epsilon)) - 1 \rceil$
- Predicate($\text{String} \rightarrow \{true, false\}$) $\text{valid}(\cdot)$

Events:

- *request* $\text{submit}(v \in \text{Value})$: a process submits a value v_i
- *notification* $\text{confirm}(v_o \in \text{Value})$: a process confirms a value v_o
- *notification* $\text{yield_certificate}(\text{cert} \in \text{String})$: a process outputs a certificate.
 cert will be interpreted as an element of *Certificate* by the judge J_{rat} for the forensic support in case of disagreement

Properties:

- *Integrity*: For any $f \in [0 : n]$, a process cannot confirm a value that it did not submit.
- *Optimistic Convergence*: If $f \leq t$ and all correct process submit the same value v , every correct process eventually confirms v .
- *Validity*: A process only yields valid certificates w.r.t. $\text{valid}(\cdot)$

Forensic-Properties:

- *Agreement*: No two correct processes output different values.
 This property does not have to be ensured (if preconditions of optimistic convergence are not met), but forensic support (defined in §3) will be then required in case of disagreement.

Algorithm 2 Π_{rat} - Pseudocode (for process p_i)

```

1: Parameters:
2:   String  $step$ 
3:   Real  $\lambda, \epsilon, \delta$  ▷ See parameters in Table 3.

4: Local variables:
5:   Dictionary(Processes, Signatures)  $signatures_i \leftarrow \perp$ 
6:   Dictionary(Processes, VRFproofs)  $\pi_i^{poe} \leftarrow \perp$ 
7:   Value  $predecided_i \leftarrow \perp$ 
8:   Hash_Value  $h_i \leftarrow \perp$ 

9: upon  $\text{submit}(v \in \text{Value})$ :
10:  ( $predecided_i, h_i$ )  $\leftarrow (v, \text{hash}(v))$ 
11:  ( $\langle elected_i, \sigma_i^e \rangle$ )  $\leftarrow \text{sample}(step, \lambda)$ 
12:  if  $\langle elected_i \rangle$ :
13:     $\sigma_i^s \leftarrow \text{sign}_i(\langle step, \text{SUBMIT}, h_i \rangle)$ 
14:    broadcast  $\langle step, \text{SUBMIT}, h_i, \sigma_i^e, \sigma_i^s \rangle$ 

15: upon  $m_j = \langle step, \text{SUBMIT}, h_j, \sigma_j^e, \sigma_j^s \rangle$  is received from process
     $p_j$  and  $h_j = h_i \neq \perp$ :
16:  if  $\text{committee\_val}(step, \lambda, j, \sigma_j^e) \wedge$ 
     $\text{ver}(\langle step, \text{SUBMIT}, h_j \rangle, \sigma_j^s, pk_j)$ :
17:     $signatures_i[p_j] \leftarrow \sigma_j^s$ 
18:     $\pi_i^{poe}[p_j] \leftarrow \sigma_j^e$ 

19: upon  $|Q| \geq W_{\delta}^{\epsilon, \lambda}$  with  $Q = \{p_j \in \Pi, signatures_i[p_j] \neq \perp\}$ :
20:   $\sigma^Q \leftarrow \text{cb}(\langle step, \text{SUBMIT}, h_i \rangle, \{pk_j, signatures_i[p_j]\}_{j \in Q})$ 
21:  trigger  $\text{yield\_certificate}(\langle step, h_i, Q, \sigma^Q \rangle, \pi_i^{poe})$ 
    Although the proof of eligibility  $\pi_i^{poe}$  for VRF-quorum  $Q$  is
    disregarded by the judge  $J_{\text{rat}}$ , it will be utilized by the
    propagator subprotocol, as detailed in the next section.
22:  trigger  $\text{confirm}(predecided_i)$ 

```

Theorem 2. Let $(J_{\text{rat}}, \text{valid_full})$ be the pair judge-predicate presented in Algorithm 3. The judge J_{rat} is fair for Π_{rat} (see Algorithm 2), which implements the Ratifier module (see Module 2) with t -resiliency, an expected communication complexity of $O(\lambda n \kappa)$, and 1 round, and provides $(n, 2, B_{\delta, \delta}^{\epsilon, \lambda}, J_{\text{rat}}, \text{valid_full})$ -forensic support for the agreement property, stating that no two correct processes confirm two different values.

PROOF SKETCH. First, we prove that the protocol Π_{rat} implements the Ratifier module with t -resiliency. The integrity property is trivial. Assume $f \leq t$ and $\exists v \in \text{Value}$, such that all correct processes trigger $\text{submit}(v)$. By Theorem 1, the probability that fewer than $W_{\delta}^{\epsilon, \lambda}$ so-far correct processes set their variable $\langle elected_i \rangle$ to $true$ line 11 is bounded by: $\Pr[|C(s, \lambda) \cap H(s, \lambda)| \leq W_{\delta}^{\epsilon, \lambda}] \leq \text{neg}(\lambda)$. Thus, with overwhelming probability, at least $W_{\delta}^{\epsilon, \lambda}$ correct processes send a (justified) SUBMIT message for the (hash of) value v , implying that line 14 is reached by at least $W_{\delta}^{\epsilon, \lambda}$ correct processes. Consequently, with overwhelming probability, line 17 is reached by every correct process p_i at least $W_{\delta}^{\epsilon, \lambda}$ times, leading to the confirmation of v at line 22.

The expected communication complexity is derived as follows. Let X_j be a Bernoulli random variable indicating whether $p_j \in C(s, \lambda)$, where $\Pr[X_j = 1] = \lambda/n$. Define $Z_L = \sum_{p_j \in H} X_j$. By linearity of expectation, $\mathbb{E}(Z_L) \leq \lambda$. Therefore, the expected number of correct processes broadcasting a SUBMIT message at line 14 is $\leq \lambda$. Since each correct VRF-committee’s member broadcasts its SUBMIT

Algorithm 3 Fair judge J_{rat} for Π_{rat} (see Algorithm 2)

Helper Definition:

We define `valid_light` the predicate that parses a string as a tuple $(step, h, Q, \sigma) \in \text{String} \times \text{Hash_Value} \times \text{Set}(\text{Process}) \times \text{MultiSignature}$, and returns `true` if and only if (a) $|Q| \geq W$ and (b) σ is a valid multi-signature of the message $m_h = \langle step, \text{SUBMIT}, h \rangle$ from the subset Q of processes, s.t. $\text{ver}(m_h, \sigma, \{pk_j\}_{p_j \in Q})$ returns `true`. `LCertificate` denotes the corresponding type of strings c such that `valid_light(c) = true`, which we call *lightweight certificates*. If $c = (step, h, Q, \sigma) \in \text{LCertificate}$, we let $c.step = step$, $c.hash = h$, and $c.quorum = Q$.

We denote by `conflictrat` the predicate over pairs of certificates that returns `true` if $c_1.hash \neq c_2.hash$ and $c_1.step = c_2.step$, and `false` otherwise. A tuple $(c_1, c_2) \in \text{LCertificate}^2$ is a *proof of misbehavior* if `conflictrat(c1, c2)`. We denote by `Proof` the corresponding type.

A *full certificate* is a pair (c, π^{poe}) , such that (1) $c = (step, h, Q, \sigma) \in \text{LCertificate}$, and (2) (c, π^{poe}) proves the eligibility of processes in Q for step $step$, i.e. we can define the predicate `valid_full` that maps pairs of the form (c, π^{poe}) to `true` if (a) `valid_light(c) = true`, and (b) $\forall p_j \in c.quorum, \text{committee_val}(step, \lambda, j, \pi^{poe}[p_j]) = \text{true}$. `FCertificate` denotes the corresponding type of strings c such that `valid_full(c) = true`. If $\tilde{c} = (c, \pi^{poe}) \in \text{FCertificate}$, we note $\tilde{c}.step = c.step$, $\tilde{c}.hash = c.hash$, $\tilde{c}.quorum = c.quorum$, $\tilde{c}.poe = \pi^{poe}$, and $\tilde{c}.light = c$.

upon input(*string* $\in \text{String}$):

```

parse string as a proof of misbehavior  $(c_1, c_2)$ 
if the parsing has succeeded, i.e. if  $(c_1, c_2) \in \text{Proof}$ :
    return  $\text{dis}(c_1.quorum \cap c_2.quorum)$ 
else:
    return  $\text{dis}(\emptyset)$ 

```

message of size $O(\kappa)$ to all processes, the overall communication complexity is $O(\lambda \kappa n)$.

Second, for any $f \leq n$, the fairness of J_{rat} follows directly from the fact that no correct process signs conflicting SUBMIT messages.

Third, we prove the forensic support property, for any $f \leq n$. Assume agreement is violated, i.e., there exist two correct processes p_i and p_j that have confirmed different values v_i and v_j , with $v_i \neq v_j$. The condition in line 19 implies that line 17 was reached by process p_i (respectively, p_j) at least $W_{\delta}^{\epsilon, \lambda}$ times due to the reception of SUBMIT messages signed by distinct sets of processes Q_i (respectively, Q_j). Therefore, processes p_i and p_j have triggered `yield_certificate((ci, *))` and `yield_certificate((cj, *))`, respectively, with $c_i = (step, \text{hash}(v_i), Q_i, \sigma^{Q_i})$ and $c_j = (step, \text{hash}(v_j), Q_j, \sigma^{Q_j})$. Hence, $J_{rat}(c_i, c_j)$ returns $Q_i \cap Q_j$, while, by Theorem 1 (Forensic), $\Pr[|Q_i \cap Q_j| \leq B_{\delta, \delta}^{\epsilon, \lambda}] \leq \text{neg}(\lambda)$.

Finally, for any $f \leq n$, if a so-far correct process p_i triggers `yield_certificate(ci)`, the checks at line 16, and 19 ensure that `valid_full(ci)`, which concludes the proof. $\square$

5. Propagator

As discussed in §1 and §3, forensic support does not specify how the judge acquires the necessary pieces of

evidence to detect misbehavior. This task is handled by the *propagator*, defined in Module 3. A straightforward implementation would be for every correct process to broadcast its certificate, but this leads to quadratic communication complexity due to the all-to-all broadcast pattern.

Module 3 Propagator

Parameters:

Predicate($\text{String} \rightarrow \{true, false\}$) `valid(·)`
 Predicate($\text{String}^2 \rightarrow \{true, false\}$) `conflict(·, ·)`

Events:

- *request propagate(c)* with `valid(c) = true`: a process propagates a valid certificate c
- *request generate_proof($\pi \in \{0, 1\}^*$)*: a process outputs a proof of misbehavior.

Accountability-Properties:

- *Conflict-freeness*: No two correct process processes p_i, p_j propagate c_1 and c_2 respectively, such that `conflict(c1, c2)`.
Accountability (defined in §3) will be required in case of violation of this property.
-

An alternative might involve leveraging the (Byzantine) Probabilistic Quorum System property [94, Lemma 4.5]. This lemma states that if two correct processes p_i and p_j independently sample two probabilistic quorums Q_i and Q_j , where $|Q_i| = |Q_j| = \lambda\sqrt{n}$, and the adversary can only independently corrupt $n - \Theta(n)$ processes, then with probability $1 - \text{neg}(\lambda)$, $Q_i \cap Q_j$ contains at least one correct process p_k . This process could receive both potentially conflicting certificates and raise an alarm by broadcasting the corresponding proof to all parties. While this solution is simple and appealing, it has a critical drawback: once p_i broadcasts its certificate $cert_i$ to Q_i , the adversary could (adaptively) corrupt Q_i before p_j receives its own certificate. Therefore, this propagation method only provides static security.

Similarly, one might consider electing a VRF-based committee C responsible for monitoring and disseminating potentially conflicting certificates. Each process would forward its confirmed certificate to C , which would then raise an alert upon detecting a conflict. However, this approach suffers from the same core limitation. Once a correct process p_i sends its certificate $cert_i$ to C , the adversary could adaptively corrupt all members of C before another correct process p_j obtains and forwards a conflicting certificate $cert_j$. In such a case, no honest party ever learns both certificates, and the inconsistency goes undetected. Importantly, this vulnerability persists even under a 1-delay-adaptive adversary. The issue is not merely the speed of adaptive corruptions, but the lack of a mechanism to ensure that conflicting certificates are delivered to the monitoring committee with a fixed temporal gap. Without such synchronization between p_i and p_j , the adversary can always exploit timing to maintain plausible deniability.

That said, one might suspect that the problem inherently admits a quadratic lower bound. Indeed, in the pessimistic case, where a network partition or adversarial scheduling causes disagreement, every process must prepare for the possibility of conflicting certificates. Two correct processes

p_i and p_j , unaware of each other's identities, must somehow reach a third process p_k (possibly even p_i or p_j themselves) to serve as a relay: a party that receives both certificates and disseminates proof of disagreement. The challenge is that such a relay can be corrupted almost immediately (after just one causal hop) before it sees both certificates. This suggests that the only robust solution is for every process to broadcast its certificate to all others, ensuring that there are sufficiently many (more than t_{acc}) honest candidates available to act as relays, even under aggressive adaptive corruptions.

The key observation is that, in the nominal case, all correct processes are expected to confirm the same value. Thus, their collective flooding of certificates can be highly optimized: there is no need to forward redundant copies of the same message. While this optimization is not apparent in a classical broadcast pattern, where the sender immediately transmits to all peers, it becomes obvious in a flooding pattern, where each process forwards messages only to locally sampled neighbors. If a process has already forwarded a message for (the hash of) a value v , originating from some source p_i , it need not forward the same message again just because it later sees it endorsed by a different source p_j .

This leads us to employ $\Pi_{\text{ERFlood}}^{\text{prop}}$ (Algorithm 4), a minor modification of the flooding protocol Π_{ERFlood} described in [18]. The protocol's pseudocode is simple (though its analysis is far from trivial): upon receiving a verifiably valid message m , each party flips a coin for each other process that decides, with some probability ρ^* , if m should be relayed to this process. Specifically, the protocol Π_{ERFlood} ensures that every process relays messages to a different random subset of processes, whose size follows a binomial law $\text{Bin}(n-1, \rho^*)$. By making this subset large enough, Π_{ERFlood} guarantees the formation of a connected Erdős-Rényi graph with a low diameter.

In $\Pi_{\text{ERFlood}}^{\text{prop}}$, which is based on Π_{ERFlood} , a message m containing a valid certificate c is relayed by process p_i only if it has not already relayed m or another message m' with a certificate c' where $c'.\text{hash} = c.\text{hash}$.

This approach, however, remains vulnerable to a subtle attack. In the absence of committee eligibility proofs, the adversary can fabricate $\Theta(n/W)$ conflicting *lightweight certificates* (defined in Algorithm 3), each signed by disjoint sets of non-elected signers of size W . These fake certificates, lacking any overlap, cannot be linked to any Byzantine process. By flooding the network with such certificates, the adversary induces a $\Theta(n/W)$ multiplicative communication overhead, effectively reverting the protocol to quadratic complexity. Conversely, if correct processes refrain from relaying all certificates to reduce load, they risk omitting a genuinely conflicting pair of honestly generated certificates, thereby undermining accountability.

Hence, $\Pi_{\text{ERFlood}}^{\text{prop}}$ propagates full certificates, where the corresponding proof of eligibility avoids the aforementioned attack. More formally:

Definition 1. Let *valid* be a predicate over strings, and *conflict* a predicate over pairs of strings, such that not being conflicting ($\text{conflict}(c, c') = \text{false}$) is an equivalence relation. We say that the pair (*valid*, *conflict*) is *propagation-friendly with respect to* (J, d) if $\text{valid}(c_1) = \text{valid}(c_2) = \text{true} \wedge \text{conflict}(c_1, c_2) = \text{true}$ implies that the judge $J(c_1, c_2)$ returns verdict $\text{dis}(G)$ with $|G| \geq d$, except with negligible probability.

The propagation-friendliness condition guarantees that if two apparently valid and conflicting certificates exist, then they necessarily implicate at least d common signers. While this property would not hold for lightweight certificates, it is recovered when full certificates include proofs of eligibility.

Observe that (*valid*_{full}, *conflict*_{rat}), defined in Algorithm 3, is propagation-friendly with respect to $(J_{\text{rat}}, B_{\delta, \delta}^{\epsilon, \lambda})$. Then, it is easy to see that the analysis of $\Pi_{\text{ERFlood}}^{\text{prop}}$ mirrors the (non-trivial) analysis of Π_{ERFlood} from [18].

Algorithm 4 $\Pi_{\text{ERFlood}}^{\text{prop}}(\rho^*)$ - Pseudocode (for process p_i)

```

1: Parameters
2:   Real  $\rho^*$                                 ▷ probability of being chosen as a neighbor
3:   Judge  $J$ 
4:   Integer  $d$ 
5:   Predicate valid                        ▷ e.g., validfull (defined in Algorithm 3)
6:   Predicate conflict                    ▷ e.g., conflictrat (Algorithm 3)
7:   Assert( $(\text{valid}, \text{conflict})$  is propagation-friendly w.r.t.  $(J, d)$ )

8: Uses
9:    $\mathcal{N}(\rho^*)$ , the random variable that returns a subset of processes
   in  $\Psi$ , where the inclusion of a process follows the Bernoulli
   distribution with expected value  $\rho^*$ .

10: Local variables:
11:   Set(Messages) relayed  $\leftarrow \emptyset$ 

12: upon propagate( $c_i \in \text{FCertificate}$ ):
13:   send  $\langle \text{CERT}, c_i \rangle$  to processes in  $\mathcal{N}_{\text{new}}$ ,
   with  $\mathcal{N}_{\text{new}} \xleftarrow{\$} \mathcal{N}(\rho^*)$ 

14: upon  $m_j = \langle \text{CERT}, c_j \rangle$  is received from process  $p_j$ :
15:   if  $\text{valid}(c_j) = \text{true}$ :
16:     if  $\text{certificates}_i = \emptyset$ :
17:        $\text{certificates}_i \leftarrow \text{certificates}_i \cup \{c_j\}$ 
18:       send  $\langle \text{CERT}, c_j \rangle$  to processes in  $\mathcal{N}_{\text{new}}$ ,
       with  $\mathcal{N}_{\text{new}} \xleftarrow{\$} \mathcal{N}(\rho^*)$ 
19:     if  $\exists c_k \in \text{certificates}_i$  s.t.  $\text{conflict}(c_k, c_j)$ :
20:       broadcast  $\langle \text{CONFLICT}, (c_k, c_j) \rangle$ 

21: upon the reception of  $\langle \text{CONFLICT}, \pi \rangle$  s.t.  $\pi \in \text{Proof}$  ( $\pi$  is a valid
   proof of misbehavior)
22:   trigger generate_proof( $\pi$ )
```

Definition 2. Let $x \in \{1, 2\}$. A *x-scalable parametrization* is a tuple $(n, \lambda, t, t_{acc}, \gamma, \epsilon, \delta, \hat{\delta}, \rho_x) \in \mathbb{N}^4 \times \mathbb{R}^5$, s.t. $t = \lceil n(\frac{1}{3} + \epsilon) \rceil - 1$, $t_{acc} = n - \Theta(n)$, $\gamma = (n - t_{acc})/n$, $\epsilon, \delta, \hat{\delta} \in \Omega(1)$, $\rho_1 = \lambda/(\gamma n)$, $\rho_2 = \sqrt{\rho_1}$.

Let $x \in \{1, 2\}$. Let *para* = $(n, \lambda, t, t_{acc}, \gamma, \epsilon, \delta, \hat{\delta}, \rho_x)$ be a *x-scalable parametrization*. We say that the tuple (mc_x, rc_x, d) , representing message complexity, round detection complexity, and number of dishonest processes exposed in case of disagreement, is a *x-scalable*

tuple w.r.t. para if $d = B_{\delta, \hat{\delta}}^{\epsilon, \lambda} = \frac{\lambda}{3}(1 - 3\hat{\delta} - 2(\delta(2 + \epsilon) - \epsilon))$, and:

- if $x = 1$, $mc_1 = O(\lambda n / \gamma)$, and $rc_1 = O(\log(\gamma n / \lambda))$,
- if $x = 2$, $mc_2 = O(n^{3/2} \cdot \sqrt{\lambda / \gamma})$, and $rc_2 = 2$.

Theorem 3. Let $x \in \{1, 2\}$. Let para = $(n, \lambda, t, t_{acc}, \gamma, \epsilon, \delta, \hat{\delta}, \rho_x)$ be a x -scalable parametrization. Let both the Propagator (Module 3) and $\Pi_{\text{ERFlood}}^{\text{prop}}(\rho^*)$ (Algorithm 4) be parameterized by a pair of predicates (valid, conflict) that is propagation-friendly with respect to some judge-integer pair (J, d) . Then, $\Pi_{\text{ERFlood}}^{\text{prop}}(\rho^*)$ provides (t_{acc}, d, J) -accountability for the propagator's conflict-freeness property, against a 1-delayed-adaptive adversary. This guarantee holds with expected round complexity rc_x , and expected communication complexity mc_x , such that (mc_x, rc_x, d) is a x -scalable tuple w.r.t. para, where each message contains a certificate c verifying $\text{valid}(c)$.

PROOF SKETCH. Define the predicate $\text{conflict\&valid}(c_1, c_2)$ to be *true* if and only if $\text{conflict}(c_1, c_2) = \text{true}$ and $\text{valid}(c_1) = \text{valid}(c_2) = \text{true}$.

Assume two distinct correct processes p_i and p_j propagate certificates c_1 and c_2 respectively, such that $\text{conflict\&valid}(c_1, c_2) = \text{true}$. Let C denote the condition that *no* so-far correct process ever passes the check at line 19. Suppose C does not hold for some so-far correct process p . Then p will broadcast a message of the form $\langle \text{CONFLICT}, \tilde{c}, \tilde{c}' \rangle$ with $\text{conflict\&valid}(\tilde{c}, \tilde{c}') = \text{true}$. Since (valid, conflict) is propagation-friendly with respect to (J, d) , each correct process will invoke $\text{generate_proof}(\tilde{c}, \tilde{c}')$ such that $J(\tilde{c}, \tilde{c}')$ returns a set G of size at least d , except with negligible probability.

Now assume, for contradiction, that C holds forever. Since (valid, conflict) is propagation-friendly, the predicate $\text{conflict}(\cdot, \cdot)$ induces an equivalence relation $\sim$ over certificates: being non-conflicting is symmetric, reflexive, and transitive. As a result, at any given time, the set of correct processes can be partitioned into four disjoint groups:

- G^0 : processes whose *certificates* variable is empty;
- G^1 : processes storing a certificate equivalent (i.e., non-conflicting) to c_1 ;
- G^2 : processes storing a certificate equivalent to c_2 ;
- G^3 : processes storing a certificate conflicting with both c_1 and c_2 .

Certificates equivalent to c_1 and c_2 then propagate independently, as if under two separate instances of Π_{ERFlood} . When a process in G^0 receives c_1 (resp., c_2), it joins G^1 (resp., G^2). If a process in $G^2 \cup G^3$ (resp., $G^1 \cup G^3$) receives c_1 (resp., c_2), then the conflict predicate conflict triggers at line 19, and the condition C no longer holds. Moreover, if a process in G^1 (resp., G^2) receives a certificate equivalent to c_1 (resp., c_2), it has already relayed an equivalent certificate, faithfully mirroring the propagation behavior of Π_{ERFlood} .

By [18, Theorem 3], a certificate $c'_1 \sim c_1$ (resp., $c'_2 \sim c_2$) will eventually reach a so-far correct process $p \in G^2 \cup G^3$ (resp., $G^1 \cup G^3$). When this occurs, p will

pass the check at line 19, contradicting the assumption that C holds forever. Hence, C does not hold indefinitely, and accountability is ensured. To avoid quadratic communication even when exposing malicious processes, the same principle applies: a proof of culpability is never relayed more than once. We omit further details for conciseness.

Now consider the case where no detection occurs (precondition of accountability property is not met). Then no correct process has received conflicting certificates. In that case, due to the check at line 17, each process forwards any given certificate at most once. Thus, the execution mirrors a run of Π_{ERFlood} for a single message, even though different (but non-conflicting) messages may be propagated and the causal structure may differ. Therefore, both the message and communication complexities match those of Π_{ERFlood} , and the latency complexity is no worse. The complexity bounds follow directly from [18, Theorem 3]. $\square$

6. Putting everything together with $\mathcal{ABC}^{++}$

Now, we can sequentially compose the ratifier and the propagator with any (subquadratic) BA protocol, to obtain a (subquadratic) BA protocol with additional forensic support and accountability for agreement. This is specified in Algorithm 5.

Algorithm 5 $\mathcal{ABC}^{++}(\rho^*)$ - Pseudocode (for process p_i)

- 1: **Uses:**
 - 2: Byzantine Agreement, **instance** Π_{BA}
 - 3: Ratifier with $(n, 2, B_{\delta, \hat{\delta}}^{\epsilon, \lambda}, J_{rat}, \text{valid_full})$ forensic support for agreement, **instance** Π_{rat}
 - 4: Propagator with $(t_{acc}, B_{\delta, \hat{\delta}}^{\epsilon, \lambda}, J_{rat})$ accountability for conflict-freeness, **instance** $(\Pi_{\text{ERFlood}}^{\text{prop}}(\rho^*, \text{valid_full}, \text{conflict}_{rat}))$
 - 5: **upon** $\text{propose}(v_{in} \in \text{Value})$:
 - 6: $\Pi_{BA}.\text{propose}(v_{in})$
 - 7: **upon** $\Pi_{BA}.\text{decide}(v_{predecision} \in \text{Value})$:
 - 8: $\Pi_{rat}.\text{submit}(v_{predecision})$
 - 9: **upon** $\Pi_{rat}.\text{confirm}(v_o \in \text{Value})$:
 - 10: **trigger** $\text{decide}(v_o)$ ▷ achieves BA
 - 11: **upon** $\Pi_{rat}.\text{yield_certificate}(cert \in \text{FCertificate})$:
 - 12: **trigger** $\text{yield_certificate}(cert)$ ▷ achieves forensic support
 - 13: $\Pi_{\text{ERFlood}}^{\text{prop}}.\text{input}(cert)$ ▷ can be skipped when requiring only forensic support [5]
 - 14: **upon** $\Pi_{\text{ERFlood}}^{\text{prop}}.\text{generate_proof}(\pi \in \text{Proof})$: ▷ achieves accountability
 - 15: **trigger** $\text{generate_proof}(\pi \in \text{Proof})$ ▷ $J_{rat}(\pi)$ returns a verdict against $B_{\delta, \hat{\delta}}^{\epsilon, \lambda}$ malicious processes
-

Theorem 4. Let $x \in \{1, 2\}$. Let para = $(n, \kappa, t, t_{acc}, \gamma, \epsilon, \delta, \hat{\delta}, \rho_x)$ be a x -scalable parametrization, and (mc'_x, rc'_x, d) be a x -scalable tuple w.r.t. para. Let Π_{BA} be a protocol that solves Byzantine agreement with validity property *val*, probability $\rho(f)$ of success under f adaptive corruptions, expected round complexity rc , and expected communication complexity cc . Let $\bar{\Pi}_{BA}$ be a protocol obtained by applying $\mathcal{ABC}^{++}(\rho_x)$

(Algorithm 5) to Π_{BA} . The judge J_{rat} (Algorithm 3) is a fair judge for $\bar{\Pi}_{BA}$, which:

- provides $(n, 2, B_{\delta, \delta}^{\epsilon, \lambda}, J_{rat}, \text{valid_full})$ -forensic support for the agreement property;
- provides $(t_{acc}, B_{\delta, \delta}^{\epsilon, \lambda}, J_{rat})$ -accountability against a 1-delayed adaptive adversary, with detection round complexity drc_x ;
- solves BA with validity property val , $\bar{rc} = rc + 1$ round complexity (for decision), $\bar{cc} = cc + mc'_x \cdot ms'$ and probability $\bar{\rho}(f)$ of success under f adaptive corruptions, where $\bar{\rho}(f) = \rho(f)(1 - \text{neg}(\lambda))$ if $f \leq t$ (and 0 otherwise). Here ms' is the size of full certificates.

PROOF. Let f be the actual number of corruptions. First, let us prove the properties of $\bar{\Pi}_{BA}$. Protocol Π_{BA} guarantees that all correct processes submit the same valid value v to Π_{rat} with probability $\rho(f)$ at communication cost cc and round complexity rc . By Theorem 2, all correct processes confirm v from Π_{rat} with probability $\bar{\rho}(f) = (1 - \text{neg}(\lambda))\rho(f)$. This means that all correct processes decide the valid value v from $\bar{\Pi}_{BA}$ with probability $\bar{\rho}$, with an additional round and $O(\lambda\kappa n)$ expected communication. The complexity is obtained from Theorem 2 and Theorem 3. Second, fairness of J_{rat} follows directly from the fact that no correct process signs conflicting submit messages. Third, let us prove the accountability and the forensic support property. Assume agreement is violated, i.e., there exist two correct processes p_i and p_j that have decided values v_i and v_j respectively, such that $v_i \neq v_j$. This implies that p_i and p_j have confirmed values v_i and v_j respectively from Π_{rat} . Thus, we can apply Theorem 2, which implies (i) the forensic property and (ii) that p_i and p_j have triggered propagate to $\Pi_{\text{ERFlood}}^{\text{prop}}$ with conflicting certificates. Thus, we can apply Theorem 3, which completes the proof. $\square$

7. Generalization in the AUC framework

The $\mathcal{ABC}^{++}$ transformation actually extends to any (ideally subquadratic) protocol that implements what [11] refers to as an *easily-accountable agreement task*. This class includes, for example, Byzantine Reliable Broadcast, and Consistent Broadcast (Reliable Broadcast without the totality property) as used in recent BFT protocols that decouple block transmission from block ordering [14].

We formalize this generalization in the Accountable Universally Composable (AUC) framework [20]. Specifically, we show that applying the $\mathcal{ABC}^{++}$ compiler to any easily-accountable agreement functionality $\mathcal{F}$ yields a protocol that AUC-realizes its accountable counterpart $\mathcal{F}^{\text{acc}}$. While the result is stated here, the full proof is deferred to Appendix C.

Definition 3. Let $\mathcal{F}$ be an ideal functionality providing the following interface, where each request or notification may be triggered at most once:

- *Request:* (propose, sid, ssid, v)
- *Notification:* (decide, sid, ssid, w)

Here, sid, ssid denote the typical (sub-)session IDs used in the UC framework. We say that $\mathcal{F}$ is an *easily accountable agreement* functionality if it satisfies the following properties:

- **Leakage:** All proposals (excluding possibly the payload) are leaked to the adversary. Moreover, any value w decided by a correct process must first be exposed on the adversarial tape and scheduled by the adversary.
- **Agreement:** No two correct processes decide on different values.
- **Termination:** Either of the following holds:
 - **Totality:** If a correct process decides, then all correct processes eventually decide.
 - **Partial-Decidability:** If it is permissible for a correct process p not to decide under some environment behavior β , then it must also be permissible, under the same behavior β , for any subset of correct processes not to decide.

We define $\mathcal{F}^{\text{acc}}$ to be the (m, d, J) -accountable counterpart of $\mathcal{F}$ if, in addition to the above interface and guarantees, $\mathcal{F}^{\text{acc}}$ exposes a detection interface of the form:

$$(\text{detection}, \text{sid}, \text{ssid}, D_i, \pi_i),$$

where D_i is a set of process identifiers and π_i is a proof. The functionality guarantees that if agreement is violated under fewer than m corruptions, then every correct process p_i eventually triggers such a detection event, and the (fair and public) [20] judge $J(\pi_i)$ returns the verdict $\text{dis}(D_i)$ with $|D_i| \geq d$.

Theorem 5. Let J_{rat} be the judge defined in Algorithm 3, and $\mathcal{F}_{\text{ea}}^{\text{acc}}$ be the $(t_{acc}, B_{\delta, \delta}^{\epsilon, \lambda}, J_{rat})$ -accountable counterpart of an easily accountable agreement functionality $\mathcal{F}_{\text{ea}}$ (as defined in Definition 3). If $\mathcal{P}$ UC-realizes $\mathcal{F}_{\text{ea}}$, then $\mathcal{ABC}^{++}(\mathcal{P})$ UC-realizes $\mathcal{F}_{\text{ea}}^{\text{acc}}$ with complexity overhead described in Theorem 4.

8. Evaluation

This section evaluates the practical feasibility of our protocol by examining the constants and cryptographic costs that influence performance. Our focus is on the ratifier's liveness and forensic guarantees, as well as the efficiency of quorum certificate aggregation and verification.

Parameters and their role. We denote by λ the expected committee size, and by $\epsilon \in [0, 1/3]$ the “resilience slack”, meaning that the tolerated fault ratio satisfies $t < (\frac{1}{3} - \epsilon)n$.

For a specific instance of the ratifier, let Z_F and Z_L denote the number of elected processes and the number of elected correct processes, respectively. By construction, $\mathbb{E}(Z_F) = \lambda$ and $\mathbb{E}(Z_L) \geq (\frac{2}{3} + \epsilon)\lambda$.

We recall the size of a VRF quorum:

$$W_{\delta}^{\epsilon, \lambda} = (1 - \delta) \left(\frac{2}{3} + \epsilon \right) \lambda,$$

where δ captures the allowed deviation from the expected value $\mathbb{E}(Z_L)$ under the corresponding Chernoff bound. If

ρ_L denotes the target upper bound on the probability of liveness violation, we require:

$$\rho_L \leq \Pr[Z_L < W_{\delta}^{\epsilon, \lambda}] = \Pr[Z_L < (1 - \delta)\mathbb{E}(Z_L)], \quad (1)$$

which implies:

$$W_{\delta}^{\epsilon, \lambda} \geq \frac{1 - \delta}{\delta^2} \cdot 2 \ln \left(\frac{1}{\rho_L} \right). \quad (2)$$

Similarly, we recall the conservative lower bound on the intersection size of two VRF quorums Q and Q' :

$$B_{\delta, \hat{\delta}}^{\epsilon, \lambda} = 2W_{\delta}^{\epsilon, \lambda} - (1 + \hat{\delta})\lambda,$$

where $\hat{\delta}$ quantifies the deviation in a Chernoff bound for Z_F . Since

$$|Q \cap Q'| \geq |Q| + |Q'| - |Q \cup Q'| \geq 2W_{\delta}^{\epsilon, \lambda} - Z_F,$$

we get:

$$\Pr[|Q \cap Q'| \leq B_{\delta, \hat{\delta}}] = \Pr[Z_F \geq (1 + \hat{\delta})\mathbb{E}(Z_F)], \quad (3)$$

which implies:

$$\lambda \geq \frac{2 + \hat{\delta}}{\hat{\delta}^2} \cdot \ln \left(\frac{1}{\rho_F} \right), \quad (4)$$

where ρ_F denotes the upper bound on the probability that the intersection is too small.

Larger values of δ and $\hat{\delta}$ tighten the Chernoff bounds but reduce the guaranteed overlap size $B_{\delta, \hat{\delta}}^{\epsilon, \lambda}$ between quorums. Therefore, we must ensure that

$$B_{\delta, \hat{\delta}}^{\epsilon, \lambda} = \left(2(1 - \delta) \left(\frac{2}{3} + \epsilon \right) - (1 + \hat{\delta}) \right) \lambda \geq d \quad (5)$$

for $d \geq 1$ (or even 100).

and ideally, we want this overlap to be substantially large, e.g., $B_{\delta, \hat{\delta}}^{\epsilon, \lambda} \geq 100$, which imposes constraints on the tuple $(\epsilon, \delta, \hat{\delta}, \lambda)$.

Assume we want to bound the probability of a security violation by $\rho_F = \rho_L = 10^{-12} \simeq 2^{-40}$, under the assumption that 80% of the processes are correct in the optimistic case (i.e., $\frac{2}{3} + \epsilon = 0.8$), and that at most $W_{\delta, \hat{\delta}}^{\epsilon, \lambda} = 1000$ signatures are aggregated during ratification.

A suitable choice of parameters is:

$$\left\{ \begin{array}{l} \epsilon = 2/15 = 0.8 - 2/3 \\ \delta = 0.21 \text{ (to satisfy (2))} \\ \lambda = 1582 \text{ by the definition of } W_{\delta}^{\epsilon, \lambda} \\ \hat{\delta} = 0.2 \text{ (to satisfy (4))} \\ \lfloor B_{\delta, \hat{\delta}}^{\epsilon, \lambda} \rfloor = 101 \text{ by the definition of } B_{\delta, \hat{\delta}}^{\epsilon, \lambda} \end{array} \right.$$

We do not elaborate on the propagation guarantees, as they directly follow from Theorem 3 of [18]. For instance, setting $\lambda \geq 1500$ and assuming a degraded mode with $\gamma = 0.01$ (i.e., 1% of processes remain correct) yields a probability p_{bad} of security violation several orders of magnitude below $2^{-40} \approx 10^{-12}$, even for a system size as

large as $n = 10^6$ or $n = 10^9$. For comparison, the current number of Ethereum validators is around $n = 10^6$. In fact, the probability p_{bad} does not need to be extremely low to act as a deterrent: to break accountability, the adversary must commit to delivering two conflicting certificates to two different processes, without knowing whether the second certificate will avoid reaching the process that already received the first, thus risking detection. This could allow the use of a smaller statistical security parameter $\lambda' < \lambda$ for propagation, further reducing communication overhead.

VRF Quorum Certificates. The exact complexity of the accountable confirmer essentially reduces to the computation, verification, and propagation of VRF-quorum certificates built from W committee members.

Each SUBMIT message includes a hash value, a VRF proof, and a multi-signature. Both the VRF scheme and the multi-signature scheme can be instantiated using the BLS signature scheme [95]. As a result, computing a SUBMIT message is fast: it requires only two BLS signatures and two hash evaluations (one for the message value and one for the eligibility check). The size of the message is also compact: it consists of two BLS signatures and one hash.

During aggregation, for each SUBMIT message of the form

$$\langle \text{SUBMIT}, h, \sigma_i^{msig}, \sigma_i^{vrf} \rangle,$$

a process must:

- 1) Hash the BLS signature σ_i^{vrf} to check the eligibility of p_i ,
- 2) Verify the VRF signature σ_i^{vrf} ,
- 3) Verify the multi-signature σ_i^{msig} ,
- 4) Aggregate:
 - (a) the BLS signature σ_i^{vrf} ,
 - (b) the BLS signature σ_i^{msig} ,
with the corresponding partial aggregations under construction.

However, verifying BLS signatures is computationally expensive due to the required pairing operations. This overhead becomes significant in what we refer to as the *pes-simistic aggregation mode*.

Optimizations. To mitigate this cost, we propose an optimization. Instead of immediately verifying each signature, the aggregator can:

- (4') Perform step (4) as above, while also aggregating:
 - (c) the public keys of the signers.

Then, instead of performing $2W$ verifications, the aggregator performs only:

- (2') A single verification of the final aggregated VRF signature,
- (3') A single verification of the final aggregated multi-signature.

This approach is what we call the *super-optimistic aggregation mode*. However, a single malformed signature can invalidate the final result, rendering the optimization ineffective.

To address this, we introduce the *optimistic aggregation mode*. In this setting, each signer additionally signs its SUBMIT message (which includes the two BLS signatures) using a lightweight, non-aggregated signature scheme such as EdDSA. The aggregator:

- Aggregates as in the super-optimistic mode,
- Verifies only the efficient signature before accepting the message.

If f' faulty processes attempt to corrupt the aggregation with malformed messages, they will be caught via their efficient signatures. In blockchain-based systems (e.g., based on Proof-of-Stake), this can be coupled with deposit slashing via proof-of-misbehavior, creating a strong economic disincentive against such attacks. Furthermore, by storing aggregated signatures and public keys in a tree-like structure, the aggregator can identify the faulty signers in $O(f' \log W)$ BLS verifications.

Finally, a verifier in the propagator component can apply the same verification technique as the aggregator, reusing the aggregation structure to validate the quorum certificate efficiently.

Evaluation. We evaluate the cost of handling VRF-based quorum certificates, including aggregation, verification, and communication, through a Rust implementation, using the Criterion benchmark crate [96], and executed on a MacBook Pro (2021) with an Apple M1 processor, 16 GB of RAM, and MacOS Sequoia 15.0.1. Our implementation uses the `blstrs` library [97] with the BLS12-381 curve [98], which provides efficient finite field and elliptic curve operations for BLS signatures. For non-aggregatable signatures used in the optimistic aggregation mode, we rely on the `ed25519_dalek` library [99].⁴

When instantiated with a quorum size of $W = 1000$ signers:

- The computation of a SUBMIT message takes less than **0.5ms**,
- The verification of a full quorum certificate takes approximately **49ms**. Importantly, this verification occurs only once per decision in the nominal mode, and twice in case of (detected) safety violation.
- The aggregation time varies depending on the mode:
 - **Pessimistic mode:** 647ms,
 - **Optimistic mode:** 45ms,
 - **Super-Optimistic mode:** 11ms.

A quorum certificate must include a proof of eligibility. Without it, in the event of a disagreement, the adversary could fabricate conflicting certificates using non-elected signers whose intersection is empty, while withholding honest certificates to save bandwidth.

Including individual eligibility proofs prevents this issue but increases the certificate size by a factor of W . Importantly, this does not impact verification time, as explained earlier,

but it does raise bandwidth concerns. For instance, with $W = 1000$, the W BLS signatures for eligibility account for roughly **52KB**, which is approximately **10%** of a typical Bitcoin block size, i.e. still reasonable in many deployment contexts. By slightly relaxing the adversarial model to tolerate a 3-delayed adaptive adversary, certificate dissemination can be optimized into a three-step process: (1) first, the sender transmits only the hash of the certificate; (2) upon receiving an explicit request, (3) the sender delivers the full certificate. This approach ensures that in the nominal case, each correct process receives the full certificate for a given hash value at most once.

Alternatively, eligibility proofs can be combined into a single succinct non-interactive argument, which would be significantly smaller. In particular, given public knowledge of a light certificate $c = (s, h, G, \sigma)$ and the public keys $\{pk_i\}_{p_i \in G}$, the prover wants to prove, in addition to $\text{light_valid}(c)$, that $G \subseteq C(s, \lambda)$, i.e., that for all $p_i \in G$, there exists σ_i such that $\text{committee_val}(s, \lambda, i, \sigma_i) = \text{true}$. Such a proof could be of size $O(\kappa + W \log n)$ by encoding the W public keys as indices (since processes can query these from the PKI). Instead of using a generic STARK solution, one can use Jackpot [19], a recent and concretely efficient construction for non-interactive aggregatable lotteries. Jackpot offers several advantages in our setting: it is proven secure in the UC framework; the aggregate proof consists of only two group elements (approximately 80 bytes when instantiated over BLS12-381); and it supports fast aggregation and verification, with measured runtimes below 7ms and 9ms respectively for 1000 signers on very similar hardware, which aligns with our own benchmarks in the super-optimistic mode. On the other hand, the construction relies on the algebraic group model, assumes a q -SDH setup, and requires participants to publish fresh public keys via the bulletin-board PKI every q lotteries.

9. Conclusion

In this work, we introduced $\mathcal{ABC}^{++}$, a generic transformation that enhances BA protocols with accountability. By leveraging two key primitives, the ratifier and the propagator, we achieve delayed-adaptively-secure accountability with subquadratic communication complexity. This transformation is applicable to a wide range of (subquadratic) BA and Byzantine Reliable Broadcast protocols, offering the first subquadratic accountable counterparts of these primitives. Such a subquadratic accountable Byzantine Reliable broadcast can be then plugged into the transformation τ_{scr} of [12], to obtain τ_{scr}^{++} , which transforms any deterministic (and even beyond) distributed protocol into its accountable counterpart, with a subquadratic multiplicative communication overhead only.

We conjecture that the aforementioned transformations can be easily adapted to Proof-of-Stake blockchains with weighted validators, using weighted VRFs [19], [100] and weighted flooding [45], [46].

4. our code is available at <https://github.com/pcivit/abcpp>

Acknowledgement

This work was supported in part by the Swiss National Foundation (#40B20_218648 and #200021_215383) and the Australian Research Council Discovery funding scheme (project number 250101739).

References

- [1] G. Bracha and S. Toueg, “Resilient consensus protocols,” in *Proceedings of the Second Annual ACM Symposium on Principles of Distributed Computing, Montreal, Quebec, Canada, August 17-19, 1983*, R. L. Probert, N. A. Lynch, and N. Santoro, Eds. ACM, 1983, pp. 12–26. [Online]. Available: <https://doi.org/10.1145/800221.806706>
- [2] P. Civit, S. Gilbert, V. Gramoli, R. Guerraoui, and J. Komatovic, “As easy as ABC: optimal (a)ccountable (b)yzantine (c)onsensus is easy!” *J. Parallel Distributed Comput.*, vol. 181, p. 104743, 2023. [Online]. Available: <https://doi.org/10.1016/j.jpdc.2023.104743>
- [3] A. Momose and L. Ren, “Multi-threshold byzantine fault tolerance,” in *CCS ’21: 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, Republic of Korea, November 15 - 19, 2021*, Y. Kim, J. Kim, G. Vigna, and E. Shi, Eds. ACM, 2021, pp. 1686–1699. [Online]. Available: <https://doi.org/10.1145/3460120.3484554>
- [4] P. Sheng, G. Wang, K. Nayak, S. Kannan, and P. Viswanath, “BFT protocol forensics,” in *CCS ’21: 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, Republic of Korea, November 15 - 19, 2021*, Y. Kim, J. Kim, G. Vigna, and E. Shi, Eds. ACM, 2021, pp. 1722–1743. [Online]. Available: <https://doi.org/10.1145/3460120.3484566>
- [5] —, “Player-replaceability and forensic support are two sides of the same (crypto) coin,” in *Financial Cryptography and Data Security - 27th International Conference, FC 2023, Bol, Brač, Croatia, May 1-5, 2023, Revised Selected Papers, Part I*, ser. Lecture Notes in Computer Science, F. Baldimtsi and C. Cachin, Eds., vol. 13950. Springer, 2023, pp. 56–74. [Online]. Available: https://doi.org/10.1007/978-3-031-47754-6_4
- [6] W. Tang, P. Sheng, R. Ni, P. Roy, X. Wang, G. Fanti, and P. Viswanath, “Cft-forensics: High-performance byzantine accountability for crash fault tolerant protocols,” *arXiv preprint arXiv:2305.09123*, 2023.
- [7] Q. You, H. Yang, X. Zhang, X. Jiang, K. Guo, and K. Hu, “Forensic support for abraham et al.’s bb protocol,” *Entropy*, vol. 27, no. 5, p. 504, 2025.
- [8] R. Küsters, T. Truderung, and A. Vogt, “Accountability: definition and relationship to verifiability,” in *Proceedings of the 17th ACM Conference on Computer and Communications Security, CCS 2010, Chicago, Illinois, USA, October 4-8, 2010*, E. Al-Shaer, A. D. Keromytis, and V. Shmatikov, Eds. ACM, 2010, pp. 526–535. [Online]. Available: <https://doi.org/10.1145/1866307.1866366>
- [9] L. F. de Souza, P. Kuznetsov, T. Rieutord, and S. Tucci Piergiovanni, “Accountability and reconfiguration: Self-healing lattice agreement,” in *25th International Conference on Principles of Distributed Systems, OPODIS 2021, December 13-15, 2021, Strasbourg, France*, ser. LIPIcs, Q. Bramas, V. Gramoli, and A. Milani, Eds., vol. 217. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021, pp. 25:1–25:23. [Online]. Available: <https://doi.org/10.4230/LIPIcs.OPODIS.2021.25>
- [10] P. Civit, S. Gilbert, and V. Gramoli, “Polygraph: Accountable byzantine agreement,” in *41st IEEE International Conference on Distributed Computing Systems, ICDCS 2021, Washington DC, USA, July 7-10, 2021*. IEEE, 2021, pp. 403–413. [Online]. Available: <https://doi.org/10.1109/ICDCS51616.2021.00046>
- [11] P. Civit, S. Gilbert, V. Gramoli, R. Guerraoui, and J. Komatovic, “As easy as ABC: optimal (a)ccountable (b)yzantine (c)onsensus is easy!” in *2022 IEEE International Parallel and Distributed Processing Symposium, IPDPS 2022, Lyon, France, May 30 - June 3, 2022*. IEEE, 2022, pp. 560–570. [Online]. Available: <https://doi.org/10.1109/IPDPS53621.2022.00061>
- [12] P. Civit, S. Gilbert, V. Gramoli, R. Guerraoui, J. Komatovic, Z. Milosevic, and A. Seredinschi, “Crime and Punishment in Distributed Byzantine Decision Tasks,” in *42nd IEEE International Conference on Distributed Computing Systems, ICDCS 2022, Bologna, Italy, July 10-13, 2022*. IEEE, 2022, pp. 34–44. [Online]. Available: <https://doi.org/10.1109/ICDCS54860.2022.00013>
- [13] M. Yin, D. Malkhi, M. K. Reiter, G. Golan-Gueta, and I. Abraham, “Hotstuff: BFT consensus with linearity and responsiveness,” in *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing, PODC 2019, Toronto, ON, Canada, July 29 - August 2, 2019*, P. Robinson and F. Ellen, Eds. ACM, 2019, pp. 347–356. [Online]. Available: <https://doi.org/10.1145/3293611.3331591>
- [14] X. Wang, H. Wang, H. Zhang, and S. Duan, “Pando: Extremely scalable BFT based on committee sampling,” *IACR Cryptol. ePrint Arch.*, p. 664, 2024. [Online]. Available: <https://eprint.iacr.org/2024/664>
- [15] T. Gong, G. F. Camilo, K. Nayak, A. Lewis-Pye, and A. Kate, “Recover from excessive faults in partially-synchronous BFT SMR,” *IACR Cryptol. ePrint Arch.*, p. 83, 2025. [Online]. Available: <https://eprint.iacr.org/2025/083>
- [16] J. Chen and S. Micali, “Algorand: A secure and efficient distributed ledger,” *Theor. Comput. Sci.*, vol. 777, pp. 155–183, 2019. [Online]. Available: <https://doi.org/10.1016/j.tcs.2019.02.001>
- [17] S. Micali, M. O. Rabin, and S. P. Vadhan, “Verifiable random functions,” in *40th Annual Symposium on Foundations of Computer Science, FOCS ’99, 17-18 October, 1999, New York, NY, USA*. IEEE Computer Society, 1999, pp. 120–130. [Online]. Available: <https://doi.org/10.1109/SFCS.1999.814584>
- [18] C. Matt, J. B. Nielsen, and S. E. Thomsen, “Formalizing delayed adaptive corruptions and the security of flooding networks,” in *Advances in Cryptology - CRYPTO 2022 - 42nd Annual International Cryptology Conference, CRYPTO 2022, Santa Barbara, CA, USA, August 15-18, 2022, Proceedings, Part II*, ser. Lecture Notes in Computer Science, Y. Dodis and T. Shrimpton, Eds., vol. 13508. Springer, 2022, pp. 400–430. [Online]. Available: https://doi.org/10.1007/978-3-031-15979-4_14
- [19] N. Fleischhacker, M. Hall-Andersen, M. Simkin, and B. Wagner, “Jackpot: Non-interactive aggregatable lotteries,” in *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part VI*, ser. Lecture Notes in Computer Science, K. Chung and Y. Sasaki, Eds., vol. 15489. Springer, 2024, pp. 365–397. [Online]. Available: https://doi.org/10.1007/978-981-96-0938-3_12
- [20] M. Graf, R. Küsters, and D. Rausch, “AUC: accountable universal composability,” in *44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023*. IEEE, 2023, pp. 1148–1167. [Online]. Available: <https://doi.org/10.1109/SP46215.2023.10179384>
- [21] R. Canetti, Y. Lindell, R. Ostrovsky, and A. Sahai, “Universally composable two-party and multi-party secure computation,” in *Proceedings on 34th Annual ACM Symposium on Theory of Computing, May 19-21, 2002, Montréal, Québec, Canada*, J. H. Reif, Ed. ACM, 2002, pp. 494–503. [Online]. Available: <https://doi.org/10.1145/509907.509980>
- [22] R. Cohen, “Asynchronous secure multiparty computation in constant time,” in *Public-Key Cryptography - PKC 2016 - 19th IACR International Conference on Practice and Theory in Public-Key Cryptography, Taipei, Taiwan, March 6-9, 2016*,

- Proceedings, Part II*, ser. Lecture Notes in Computer Science, C. Cheng, K. Chung, G. Persiano, and B. Yang, Eds., vol. 9615. Springer, 2016, pp. 183–207. [Online]. Available: https://doi.org/10.1007/978-3-662-49387-8_8
- [23] M. Rambaud and A. Urban, “Almost-asynchronous MPC under honest majority, revisited,” *IACR Cryptol. ePrint Arch.*, p. 503, 2021. [Online]. Available: <https://eprint.iacr.org/2021/503>
- [24] I. Abraham, T. H. Chan, D. Dolev, K. Nayak, R. Pass, L. Ren, and E. Shi, “Communication Complexity of Byzantine Agreement, Revisited,” in *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing, PODC 2019, Toronto, ON, Canada, July 29 - August 2, 2019*, P. Robinson and F. Ellen, Eds. ACM, 2019, pp. 317–326. [Online]. Available: <https://doi.org/10.1145/3293611.3331629>
- [25] A. F. Anta, K. M. Konwar, C. Georgiou, and N. C. Nicolaou, “Formalizing and implementing distributed ledger objects,” *SIGACT News*, vol. 49, no. 2, pp. 58–76, 2018. [Online]. Available: <https://doi.org/10.1145/3232679.3232691>
- [26] J. A. Garay, A. Kiayias, and N. Leonardos, “The bitcoin backbone protocol: Analysis and applications,” in *Advances in Cryptology - EUROCRYPT 2015 - 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26-30, 2015, Proceedings, Part II*, ser. Lecture Notes in Computer Science, E. Oswald and M. Fischlin, Eds., vol. 9057. Springer, 2015, pp. 281–310. [Online]. Available: https://doi.org/10.1007/978-3-662-46803-6_10
- [27] —, “The bitcoin backbone protocol: Analysis and applications,” *J. ACM*, vol. 71, no. 4, pp. 25:1–25:49, 2024. [Online]. Available: <https://doi.org/10.1145/3653445>
- [28] M. Graf, R. Küsters, D. Rausch, S. Egger, M. Bechtold, and M. Flinspach, “Accountable bulletin boards: Definition and provably secure implementation,” *IACR Cryptol. ePrint Arch.*, p. 1869, 2023. [Online]. Available: <https://eprint.iacr.org/2023/1869>
- [29] A. Kiayias, A. Kulmaa, H. Lipmaa, J. Siim, and T. Zacharias, “On the security properties of e-voting bulletin boards,” in *Security and Cryptography for Networks - 11th International Conference, SCN 2018, Amalfi, Italy, September 5-7, 2018, Proceedings*, ser. Lecture Notes in Computer Science, D. Catalano and R. D. Prisco, Eds., vol. 11035. Springer, 2018, pp. 505–523. [Online]. Available: https://doi.org/10.1007/978-3-319-98113-0_27
- [30] M. Camaioni, R. Guerraoui, J. Komatovic, M. Monti, P. Roman, M. Vidigueira, and G. Voron, “Carbon: Scaling trusted payments with untrusted machines,” *IEEE Trans. Dependable Secur. Comput.*, vol. 22, no. 2, pp. 1168–1180, 2025. [Online]. Available: <https://doi.org/10.1109/TDSC.2024.3428617>
- [31] D. Collins, R. Guerraoui, J. Komatovic, P. Kuznetsov, M. Monti, M. Pavlovic, Y. Pignolet, D. Seredinschi, A. Tonkikh, and A. Xygis, “Online payments by merely broadcasting messages,” in *50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN 2020, Valencia, Spain, June 29 - July 2, 2020*. IEEE, 2020, pp. 26–38. [Online]. Available: <https://doi.org/10.1109/DSN48063.2020.00023>
- [32] R. Guerraoui, P. Kuznetsov, M. Monti, M. Pavlovic, and D. Seredinschi, “The consensus number of a cryptocurrency,” in *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing, PODC 2019, Toronto, ON, Canada, July 29 - August 2, 2019*, P. Robinson and F. Ellen, Eds. ACM, 2019, pp. 307–316. [Online]. Available: <https://doi.org/10.1145/3293611.3331589>
- [33] A. Ranchal-Pedrosa and V. Gramoli, “TRAP: the bait of rational players to solve byzantine consensus,” in *ASIA CCS '22: ACM Asia Conference on Computer and Communications Security, Nagasaki, Japan, 30 May 2022 - 3 June 2022*, Y. Suga, K. Sakurai, X. Ding, and K. Sako, Eds. ACM, 2022, pp. 168–181. [Online]. Available: <https://doi.org/10.1145/3488932.3517386>
- [34] A. Lewis-Pye and T. Roughgarden, “Beyond optimal fault tolerance,” *CoRR*, vol. abs/2501.06044, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2501.06044>
- [35] A. Ranchal-Pedrosa and V. Gramoli, “ZLB: A blockchain to tolerate colluding majorities,” in *54th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN 2024, Brisbane, Australia, June 24-27, 2024*. IEEE, 2024, pp. 209–222. [Online]. Available: <https://doi.org/10.1109/DSN58291.2024.00032>
- [36] S. Sridhar, D. Zindros, and D. Tse, “Better safe than sorry: Recovering after adversarial majority,” *CoRR*, vol. abs/2310.06338, 2023. [Online]. Available: <https://doi.org/10.48550/arXiv.2310.06338>
- [37] A. Haeberlen, P. Kouznetsov, and P. Druschel, “Peerreview: practical accountability for distributed systems,” in *Proceedings of the 21st ACM Symposium on Operating Systems Principles 2007, SOSP 2007, Stevenson, Washington, USA, October 14-17, 2007*, T. C. Bressoud and M. F. Kaashoek, Eds. ACM, 2007, pp. 175–188. [Online]. Available: <https://doi.org/10.1145/1294261.1294279>
- [38] R. Künnemann, I. Esiyok, and M. Backes, “Automated verification of accountability in security protocols,” in *32nd IEEE Computer Security Foundations Symposium, CSF 2019, Hoboken, NJ, USA, June 25-28, 2019*. IEEE, 2019, pp. 397–413. [Online]. Available: <https://doi.org/10.1109/CSF.2019.00034>
- [39] M. Backes, P. Druschel, A. Haeberlen, and D. Unruh, “CSAR: A practical and provable technique to make randomized systems accountable,” in *Proceedings of the Network and Distributed System Security Symposium, NDSS 2009, San Diego, California, USA, 8th February - 11th February 2009*. The Internet Society, 2009.
- [40] M. Backes, D. Fiore, and E. Mohammadi, “Privacy-preserving accountable computation,” in *Computer Security - ESORICS 2013 - 18th European Symposium on Research in Computer Security, Egham, UK, September 9-13, 2013. Proceedings*, ser. Lecture Notes in Computer Science, J. Crampton, S. Jajodia, and K. Mayes, Eds., vol. 8134. Springer, 2013, pp. 38–56. [Online]. Available: https://doi.org/10.1007/978-3-642-40203-6_3
- [41] A. Papadimitriou, M. Zhao, and A. Haeberlen, “Towards privacy-preserving fault detection,” in *Proceedings of the 9th Workshop on Hot Topics in Dependable Systems, HotDep 2013, Farmington, Pennsylvania, USA, November 3, 2013*, C. Cachin and R. van Renesse, Eds. ACM, 2013, pp. 6:1–6:5. [Online]. Available: <https://doi.org/10.1145/2524224.2524233>
- [42] K. Nayak, “Player Replaceability - Towards Adaptive Security and Sub-quadratic Communication Simultaneously (Part I),” 2023, <https://decentralizedthoughts.github.io/2023-01-05-player-replaceability-I/>.
- [43] —, “Player Replaceability - Towards Adaptive Security and Sub-quadratic Communication Simultaneously (Part II),” 2023, <https://decentralizedthoughts.github.io/2023-01-05-player-replaceability-II/>.
- [44] A. Kermarrec, L. Massoulié, and A. J. Ganesh, “Probabilistic reliable dissemination in large-scale systems,” *IEEE Trans. Parallel Distributed Syst.*, vol. 14, no. 3, pp. 248–258, 2003. [Online]. Available: <https://doi.org/10.1109/TPDS.2003.1189583>
- [45] C. Liu-Zhang, C. Matt, U. Maurer, G. Rito, and S. E. Thomsen, “Practical provably secure flooding for blockchains,” in *Advances in Cryptology - ASIACRYPT 2022 - 28th International Conference on the Theory and Application of Cryptology and Information Security, Taipei, Taiwan, December 5-9, 2022, Proceedings, Part I*, ser. Lecture Notes in Computer Science, S. Agrawal and D. Lin, Eds., vol. 13791. Springer, 2022, pp. 774–805. [Online]. Available: https://doi.org/10.1007/978-3-031-22963-3_26
- [46] C. Liu-Zhang, C. Matt, and S. E. Thomsen, “Asymptotically optimal message dissemination with applications to blockchains,” in *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30,*

- 2024, *Proceedings, Part III*, ser. Lecture Notes in Computer Science, M. Joye and G. Leander, Eds., vol. 14653. Springer, 2024, pp. 64–95. [Online]. Available: https://doi.org/10.1007/978-3-031-58734-4_3
- [47] V. King and J. Saia, “Breaking the $o(n^2)$ bit barrier: Scalable byzantine agreement with an adaptive adversary,” *Journal of the ACM (JACM)*, vol. 58, no. 4, pp. 1–24, 2011.
- [48] E. Blum, J. Katz, C. Liu-Zhang, and J. Loss, “Asynchronous byzantine agreement with subquadratic communication,” in *Theory of Cryptography - 18th International Conference, TCC 2020, Durham, NC, USA, November 16-19, 2020, Proceedings, Part I*, ser. Lecture Notes in Computer Science, R. Pass and K. Pietrzak, Eds., vol. 12550. Springer, 2020, pp. 353–380. [Online]. Available: https://doi.org/10.1007/978-3-030-64375-1_13
- [49] M. Rambaud, “Adaptively secure consensus with linear complexity and constant round under honest majority in the bare PKI model, and separation bounds from the idealized message-authentication model,” *IACR Cryptol. ePrint Arch.*, p. 1757, 2023. [Online]. Available: <https://eprint.iacr.org/2023/1757>
- [50] I. Abraham, K. Nayak, and N. Shrestha, “Communication and Round Efficient Parallel Broadcast Protocols,” *Cryptology ePrint Archive*, 2023.
- [51] I. Abraham, E. Chouatt, I. Damgård, Y. Gilad, G. Stern, and S. Yakubov, “Asynchronous algorand: Reaching agreement with near linear communication and constant expected time,” in *Proceedings of the 2025 ACM Symposium on Principles of Distributed Computing, PODC 2025, Huatulco, Mexico, June 16-20, 2025*. ACM, 2025. [Online]. Available: <https://eprint.iacr.org/2025/303>
- [52] S. Cohen, I. Keidar, and A. Spiegelman, “Not a COINcidence: Sub-Quadratic Asynchronous Byzantine Agreement WHP,” in *34th International Symposium on Distributed Computing (DISC 2020)*. Schloss Dagstuhl-Leibniz-Zentrum für Informatik, 2020.
- [53] A. Bhangale, C. Liu-Zhang, J. Loss, and K. Nayak, “Efficient adaptively-secure byzantine agreement for long messages,” in *Advances in Cryptology - ASIACRYPT 2022 - 28th International Conference on the Theory and Application of Cryptology and Information Security, Taipei, Taiwan, December 5-9, 2022, Proceedings, Part I*, ser. Lecture Notes in Computer Science, S. Agrawal and D. Lin, Eds., vol. 13791. Springer, 2022, pp. 504–525. [Online]. Available: https://doi.org/10.1007/978-3-031-22963-3_17
- [54] G. Bracha, “An Asynchronous $[(n-1)/3]$ -Resilient Consensus Protocol,” in *Proceedings of the Third Annual ACM Symposium on Principles of Distributed Computing, Vancouver, B. C., Canada, August 27-29, 1984*, T. Kameda, J. Misra, J. G. Peters, and N. Santoro, Eds. ACM, 1984, pp. 154–162. [Online]. Available: <https://doi.org/10.1145/800222.806743>
- [55] —, “Asynchronous byzantine agreement protocols,” *Inf. Comput.*, vol. 75, no. 2, pp. 130–143, 1987. [Online]. Available: [https://doi.org/10.1016/0890-5401\(87\)90054-X](https://doi.org/10.1016/0890-5401(87)90054-X)
- [56] R. Guerraoui, P. Kuznetsov, M. Monti, M. Pavlovic, and D. Seredinschi, “Scalable byzantine reliable broadcast,” in *33rd International Symposium on Distributed Computing, DISC 2019, October 14-18, 2019, Budapest, Hungary*, ser. LIPIcs, J. Suomela, Ed., vol. 146. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2019, pp. 22:1–22:16.
- [57] Q. Zhao, G. Pîrlea, K. Grzeszkiewicz, S. Gilbert, and I. Sergey, “Compositional verification of composite byzantine protocols,” in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14–18, 2024*. ACM, 2024. [Online]. Available: <https://doi.org/10.1145/3658644.3690355>
- [58] W. Tang, P. Sheng, R. Ni, P. Roy, X. Wang, G. Fanti, and P. Viswanath, “Cft-forensics: High-performance byzantine accountability for crash fault tolerant protocols,” in *6th Conference on Advances in Financial Technologies, AFT 2024, September 23-25, 2024, Vienna, Austria*, ser. LIPIcs, R. Böhme and L. Kiffer, Eds., vol. 316. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024, pp. 3:1–3:25. [Online]. Available: <https://doi.org/10.4230/LIPIcs.AFT.2024.3>
- [59] E. Buchman, R. Guerraoui, J. Komatovic, Z. Milosevic, D. Seredinschi, and J. Widder, “Revisiting tendermint: Design tradeoffs, accountability, and practical use,” in *52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN 2022, Supplemental Volume, Baltimore, MD, USA, June 27-30, 2022*. IEEE, 2022, pp. 11–14. [Online]. Available: <https://doi.org/10.1109/DSN-S54099.2022.00014>
- [60] A. D. Pozzo and T. Rieutord, “Fork accountability in tenderbake,” in *5th International Symposium on Foundations and Applications of Blockchain 2022, FAB 2022, June 3, 2022, Berkeley, CA, USA*, ser. OASICS, S. T. Piergiovanni and N. Crooks, Eds., vol. 101. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022, pp. 5:1–5:22. [Online]. Available: <https://doi.org/10.4230/OASICS.FAB.2022.5>
- [61] J. Neu, S. Sridhar, L. Yang, and D. Tse, “Optimal flexible consensus and its application to ethereum,” in *IEEE Symposium on Security and Privacy, SP 2024, San Francisco, CA, USA, May 19-23, 2024*. IEEE, 2024, pp. 3885–3903. [Online]. Available: <https://doi.org/10.1109/SP54263.2024.00135>
- [62] P. Civit, S. Gilbert, R. Guerraoui, J. Komatovic, and M. Vidigueira, “On the Validity of Consensus,” in *Proceedings of the 2023 ACM Symposium on Principles of Distributed Computing, PODC 2023, Orlando, FL, USA, June 19-23, 2023*, R. Oshman, A. Nolin, M. M. Halldórsson, and A. Balliu, Eds. ACM, 2023, pp. 332–343. [Online]. Available: <https://doi.org/10.1145/3583668.3594567>
- [63] H. Cheng, Y. Lu, Z. Lu, Q. Tang, Y. Zhang, and Z. Zhang, “Jumbo: Fully asynchronous bft consensus made truly scalable,” *arXiv preprint arXiv:2403.11238*, 2024.
- [64] H. Feng, Z. Lu, T. Mai, and Q. Tang, “Making hash-based mvba great again,” *Cryptology ePrint Archive*, 2024.
- [65] Y. Lu, Z. Lu, and Q. Tang, “Bolt-dumbo transformer: Asynchronous consensus as fast as the pipelined BFT,” in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022*, H. Yin, A. Stavrou, C. Cremers, and E. Shi, Eds. ACM, 2022, pp. 2159–2173. [Online]. Available: <https://doi.org/10.1145/3548606.3559346>
- [66] Y. Lu, Z. Lu, Q. Tang, and G. Wang, “Dumbo-MVBA: Optimal Multi-Valued Validated Asynchronous Byzantine Agreement, Revisited,” *Proceedings of the Annual ACM Symposium on Principles of Distributed Computing*, pp. 129–138, 2020.
- [67] Y. Zhou, Z. Zhang, H. Zhang, S. Duan, B. Hu, L. Wang, and J. Liu, “Dory: Asynchronous BFT with reduced communication and improved efficiency,” *IACR Cryptol. ePrint Arch.*, p. 1709, 2022. [Online]. Available: <https://eprint.iacr.org/2022/1709>
- [68] I. Abraham, G. Asharov, A. Patra, and G. Stern, “Perfectly secure asynchronous agreement on a core set in constant expected time,” *IACR Cryptol. ePrint Arch.*, p. 1130, 2023. [Online]. Available: <https://eprint.iacr.org/2023/1130>
- [69] R. Cohen, P. Forghani, J. Garay, R. Patel, and V. Zikas, “Concurrent asynchronous byzantine agreement in expected-constant rounds, revisited,” in *Theory of Cryptography Conference*. Springer, 2023, pp. 422–451.
- [70] S. Das, S. Duan, S. Liu, A. Momose, L. Ren, and V. Shoup, “Asynchronous consensus without trusted setup or public-key cryptography,” in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*, B. Luo, X. Liao, J. Xu, E. Kirda, and D. Lie, Eds. ACM, 2024, pp. 3242–3256. [Online]. Available: <https://doi.org/10.1145/3658644.3670327>

- [71] V. Shoup, "A theoretical take on a practical consensus protocol," *IACR Cryptol. ePrint Arch.*, p. 696, 2024. [Online]. Available: <https://eprint.iacr.org/2024/696>
- [72] M. Graf, R. Küsters, and D. Rausch, "Accountability in a permissioned blockchain: Formal analysis of hyperledger fabric," in *IEEE European Symposium on Security and Privacy, EuroS&P 2020, Genoa, Italy, September 7-11, 2020*. IEEE, 2020, pp. 236–255. [Online]. Available: <https://doi.org/10.1109/EuroSP48549.2020.00023>
- [73] J. Neu, E. N. Tas, and D. Tse, "Ebb-and-flow protocols: A resolution of the availability-finality dilemma," in *42nd IEEE Symposium on Security and Privacy, SP 2021, San Francisco, CA, USA, 24-27 May 2021*. IEEE, 2021, pp. 446–465. [Online]. Available: <https://doi.org/10.1109/SP40001.2021.00045>
- [74] V. Buterin and V. Griffith, "Casper the friendly finality gadget," *CoRR*, vol. abs/1710.09437, 2017. [Online]. Available: <http://arxiv.org/abs/1710.09437>
- [75] S. Sankagiri, X. Wang, S. Kannan, and P. Viswanath, "Blockchain CAP theorem allows user-dependent adaptivity and finality," in *Financial Cryptography and Data Security - 25th International Conference, FC 2021, Virtual Event, March 1-5, 2021, Revised Selected Papers, Part II*, ser. Lecture Notes in Computer Science, N. Borisov and C. Díaz, Eds., vol. 12675. Springer, 2021, pp. 84–103. [Online]. Available: https://doi.org/10.1007/978-3-662-64331-0_5
- [76] J. Neu, E. N. Tas, and D. Tse, "The availability-accountability dilemma and its resolution via accountability gadgets," in *Financial Cryptography and Data Security - 26th International Conference, FC 2022, Grenada, May 2-6, 2022, Revised Selected Papers*, ser. Lecture Notes in Computer Science, I. Eyal and J. A. Garay, Eds., vol. 13411. Springer, 2022, pp. 541–559. [Online]. Available: https://doi.org/10.1007/978-3-031-18283-9_27
- [77] —, "Short paper: Accountable safety implies finality," in *Financial Cryptography and Data Security - 28th International Conference, FC 2024, Curacao Marriott Beach Resort Willemstad, Curaçao, Mars 4-8, 2024, Revised Selected Papers, Part I*, ser. Lecture Notes in Computer Science. Springer, 2024.
- [78] R. Cleve, "Limits on the security of coin flips when half the processors are faulty (extended abstract)," in *Proceedings of the 18th Annual ACM Symposium on Theory of Computing, May 28-30, 1986, Berkeley, California, USA*, J. Hartmanis, Ed. ACM, 1986, pp. 364–369. [Online]. Available: <https://doi.org/10.1145/12130.12168>
- [79] Y. Ishai, R. Ostrovsky, and V. Zikas, "Secure multi-party computation with identifiable abort," in *Advances in Cryptology - CRYPTO 2014 - 34th Annual Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2014, Proceedings, Part II*, ser. Lecture Notes in Computer Science, J. A. Garay and R. Gennaro, Eds., vol. 8617. Springer, 2014, pp. 369–386. [Online]. Available: https://doi.org/10.1007/978-3-662-44381-1_21
- [80] R. Cohen and Y. Lindell, "Fairness versus guaranteed output delivery in secure multiparty computation," in *Advances in Cryptology - ASIACRYPT 2014 - 20th International Conference on the Theory and Application of Cryptology and Information Security, Kaoshiung, Taiwan, R.O.C., December 7-11, 2014, Proceedings, Part II*, ser. Lecture Notes in Computer Science, P. Sarkar and T. Iwata, Eds., vol. 8874. Springer, 2014, pp. 466–485. [Online]. Available: https://doi.org/10.1007/978-3-662-45608-8_25
- [81] —, "Fairness versus guaranteed output delivery in secure multiparty computation," *J. Cryptol.*, vol. 30, no. 4, pp. 1157–1186, 2017. [Online]. Available: <https://doi.org/10.1007/s00145-016-9245-5>
- [82] Y. Aumann and Y. Lindell, "Security against covert adversaries: Efficient protocols for realistic adversaries," in *Theory of Cryptography, 4th Theory of Cryptography Conference, TCC 2007, Amsterdam, The Netherlands, February 21-24, 2007, Proceedings*, ser. Lecture Notes in Computer Science, S. P. Vadhan, Ed., vol. 4392. Springer, 2007, pp. 137–156. [Online]. Available: https://doi.org/10.1007/978-3-540-70936-7_8
- [83] —, "Security against covert adversaries: Efficient protocols for realistic adversaries," *J. Cryptol.*, vol. 23, no. 2, pp. 281–343, 2010. [Online]. Available: <https://doi.org/10.1007/s00145-009-9040-7>
- [84] M. Rivinius, P. Reiser, D. Rausch, and R. Küsters, "Publicly accountable robust multi-party computation," in *43rd IEEE Symposium on Security and Privacy, SP 2022, San Francisco, CA, USA, May 22-26, 2022*. IEEE, 2022, pp. 2430–2449. [Online]. Available: <https://doi.org/10.1109/SP46214.2022.9833608>
- [85] C. Dwork, N. Lynch, and L. Stockmeyer, "Consensus in the Presence of Partial Synchrony," *Journal of the Association for Computing Machinery*, Vol. 35, No. 2, pp.288-323, 1988.
- [86] K. P. Kihlstrom, L. E. Moser, and P. M. Melliar-Smith, "Byzantine fault detectors for solving consensus," *Comput. J.*, vol. 46, no. 1, pp. 16–35, 2003. [Online]. Available: <https://doi.org/10.1093/comjnl/46.1.16>
- [87] A. Boldyreva, "Threshold signatures, multisignatures and blind signatures based on the gap-diffie-hellman-group signature scheme," in *Public Key Cryptography - PKC 2003, 6th International Workshop on Theory and Practice in Public Key Cryptography, Miami, FL, USA, January 6-8, 2003, Proceedings*, ser. Lecture Notes in Computer Science, Y. Desmedt, Ed., vol. 2567. Springer, 2003, pp. 31–46. [Online]. Available: https://doi.org/10.1007/3-540-36288-6_3
- [88] D. Boneh, M. Drijvers, and G. Neven, "Compact multi-signatures for smaller blockchains," in *Advances in Cryptology - ASIACRYPT 2018 - 24th International Conference on the Theory and Application of Cryptology and Information Security, Brisbane, QLD, Australia, December 2-6, 2018, Proceedings, Part II*, ser. Lecture Notes in Computer Science, T. Peyrin and S. D. Galbraith, Eds., vol. 11273. Springer, 2018, pp. 435–464. [Online]. Available: https://doi.org/10.1007/978-3-030-03329-3_15
- [89] D. Galindo and J. Liu, "Robust subgroup multi-signatures for consensus," in *Topics in Cryptology - CT-RSA 2022 - Cryptographers' Track at the RSA Conference 2022, Virtual Event, March 1-2, 2022, Proceedings*, ser. Lecture Notes in Computer Science, S. D. Galbraith, Ed., vol. 13161. Springer, 2022, pp. 537–561. [Online]. Available: https://doi.org/10.1007/978-3-030-95312-6_22
- [90] M. Rambaud and C. Levrat, "Practical non-interactive multi-signatures, and a multi-to-aggregate signatures compiler," *IACR Cryptol. ePrint Arch.*, p. 1081, 2024. [Online]. Available: <https://eprint.iacr.org/2024/1081>
- [91] T. Ristenpart and S. Yilek, "The power of proofs-of-possession: Securing multiparty signatures against rogue-key attacks," in *Advances in Cryptology - EUROCRYPT 2007, 26th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Barcelona, Spain, May 20-24, 2007, Proceedings*, ser. Lecture Notes in Computer Science, M. Naor, Ed., vol. 4515. Springer, 2007, pp. 228–245. [Online]. Available: https://doi.org/10.1007/978-3-540-72540-4_13
- [92] R. Canetti, "Universally composable signature, certification, and authentication," in *17th IEEE Computer Security Foundations Workshop, (CSFW-17 2004), 28-30 June 2004, Pacific Grove, CA, USA*. IEEE Computer Society, 2004, p. 219. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/CSFW.2004.24>
- [93] S. Cohen, I. Keidar, and A. Spiegelman, "Not a coincidence: Sub-quadratic asynchronous byzantine agreement WHP," in *34th International Symposium on Distributed Computing, DISC 2020, October 12-16, 2020, Virtual Conference*, ser. LIPIcs, H. Attiya, Ed., vol. 179. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020, pp. 25:1–25:17. [Online]. Available: <https://doi.org/10.4230/LIPIcs.DISC.2020.25>
- [94] D. Malkhi, M. K. Reiter, A. Wool, and R. N. Wright, "Probabilistic quorum systems," *Inf. Comput.*, vol. 170, no. 2, pp. 184–206, 2001. [Online]. Available: <https://doi.org/10.1006/inco.2001.3054>
- [95] D. Boneh, B. Lynn, and H. Shacham, "Short signatures from the weil pairing," *Journal of cryptology*, vol. 17, pp. 297–319, 2004.

- [96] <https://docs.rs/criterion/latest/criterion/>.
- [97] <https://docs.rs/blst/latest/blst/>.
- [98] P. S. Barreto, B. Lynn, and M. Scott, "Constructing elliptic curves with prescribed embedding degrees," in *International conference on security in communication networks*. Springer, 2002, pp. 257–267.
- [99] <https://docs.rs/ed25519-dalek/latest/ed25519-dalek/>.
- [100] S. Das, B. Pinkas, A. Tescanu, and Z. Xiang, "Distributed randomness using weighted vufs," in *Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4-8, 2025, Proceedings, Part VII*, ser. Lecture Notes in Computer Science, S. Fehr and P. Fouque, Eds., vol. 15607. Springer, 2025, pp. 314–344. [Online]. Available: https://doi.org/10.1007/978-3-031-91098-2_12
- [101] A. Haeberlen and P. Kuznetsov, "The fault detection problem," in *Principles of Distributed Systems, 13th International Conference, OPODIS 2009, Nîmes, France, December 15-18, 2009. Proceedings*, ser. Lecture Notes in Computer Science, T. F. Abdelzaher, M. Raynal, and N. Santoro, Eds., vol. 5923. Springer, 2009, pp. 99–114. [Online]. Available: https://doi.org/10.1007/978-3-642-10877-8_10
- [102] H. Attiya and J. L. Welch, *Distributed computing - fundamentals, simulations, and advanced topics (2. ed.)*, ser. Wiley series on parallel and distributed computing. Wiley, 2004.
- [103] B. A. Coan, "A compiler that increases the fault tolerance of asynchronous protocols," *IEEE Trans. Computers*, vol. 37, no. 12, pp. 1541–1553, 1988. [Online]. Available: <https://doi.org/10.1109/12.9732>
- [104] A. Clement, F. Junqueira, A. Kate, and R. Rodrigues, "On the (limited) power of non-equivocation," in *Proceedings of the 2012 ACM Symposium on Principles of Distributed Computing*, ser. PODC '12. New York, NY, USA: Association for Computing Machinery, 2012, p. 301–308. [Online]. Available: <https://doi.org/10.1145/2332432.2332490>
- [105] C. Ho, D. Dolev, and R. van Renesse, "Making distributed applications robust," in *Principles of Distributed Systems, 11th International Conference, OPODIS 2007, Guadeloupe, French West Indies, December 17-20, 2007. Proceedings*, ser. Lecture Notes in Computer Science, E. Tovar, P. Tsigas, and H. Fouchal, Eds., vol. 4878. Springer, 2007, pp. 232–246. [Online]. Available: https://doi.org/10.1007/978-3-540-77096-1_17
- [106] R. Canetti, "Universally composable security: A new paradigm for cryptographic protocols," in *42nd Annual Symposium on Foundations of Computer Science, FOCS 2001, 14-17 October 2001, Las Vegas, Nevada, USA*. IEEE Computer Society, 2001, pp. 136–145. [Online]. Available: <https://doi.org/10.1109/SFCS.2001.959888>
- [107] N. A. Lynch, *Distributed Algorithms*. Elsevier, 1996.
- [108] S. Coretti, J. A. Garay, M. Hirt, and V. Zikas, "Constant-round asynchronous multi-party computation based on one-way functions," in *Advances in Cryptology - ASIACRYPT 2016 - 22nd International Conference on the Theory and Application of Cryptology and Information Security, Hanoi, Vietnam, December 4-8, 2016, Proceedings, Part II*, ser. Lecture Notes in Computer Science, J. H. Cheon and T. Takagi, Eds., vol. 10032, 2016, pp. 998–1021. [Online]. Available: https://doi.org/10.1007/978-3-662-53890-6_33
- [109] J. Katz, U. Maurer, B. Tackmann, and V. Zikas, "Universally composable synchronous computation," in *Theory of Cryptography - 10th Theory of Cryptography Conference, TCC 2013, Tokyo, Japan, March 3-6, 2013. Proceedings*, ser. Lecture Notes in Computer Science, A. Sahai, Ed., vol. 7785. Springer, 2013, pp. 477–498. [Online]. Available: https://doi.org/10.1007/978-3-642-36594-2_27
- [110] V. Shoup and N. P. Smart, "Lightweight asynchronous verifiable secret sharing with optimal resilience," *J. Cryptol.*, vol. 37, no. 3, p. 27, 2024. [Online]. Available: <https://doi.org/10.1007/s00145-024-09505-6>
- [111] C. Cachin, K. Kursawe, F. Petzold, and V. Shoup, "Secure and efficient asynchronous broadcast protocols," in *Advances in Cryptology - CRYPTO 2001, 21st Annual International Cryptology Conference, Santa Barbara, California, USA, August 19-23, 2001, Proceedings*, ser. Lecture Notes in Computer Science, J. Kilian, Ed., vol. 2139. Springer, 2001, pp. 524–541. [Online]. Available: https://doi.org/10.1007/3-540-44647-8_31
- [112] C. Cachin, K. Kursawe, and V. Shoup, "Random oracles in constantipole: practical asynchronous byzantine agreement using cryptography (extended abstract)," in *Proceedings of the Nineteenth Annual ACM Symposium on Principles of Distributed Computing, July 16-19, 2000, Portland, Oregon, USA*, G. Neiger, Ed. ACM, 2000, pp. 123–132. [Online]. Available: <https://doi.org/10.1145/343477.343531>
- [113] —, "Random Oracles in Constantinople: Practical Asynchronous Byzantine Agreement Using Cryptography," *J. Cryptol.*, vol. 18, no. 3, pp. 219–246, 2005. [Online]. Available: <https://doi.org/10.1007/s00145-005-0318-0>
- [114] A. Bandrupalli, X. Ji, A. Kate, C. Liu-Zhang, and Y. Song, "Concretely efficient asynchronous MPC from lightweight cryptography," *IACR Cryptol. ePrint Arch.*, p. 1666, 2024. [Online]. Available: <https://eprint.iacr.org/2024/1666>
- [115] J. Camenisch, S. Krenn, R. Küsters, and D. Rausch, "iuc: Flexible universal composability made simple," in *Advances in Cryptology - ASIACRYPT 2019 - 25th International Conference on the Theory and Application of Cryptology and Information Security, Kobe, Japan, December 8-12, 2019, Proceedings, Part III*, ser. Lecture Notes in Computer Science, S. D. Galbraith and S. Moriai, Eds., vol. 11923. Springer, 2019, pp. 191–221. [Online]. Available: https://doi.org/10.1007/978-3-030-34618-8_7
- [116] R. Küsters, M. Tuengerthal, and D. Rausch, "The IITM model: A simple and expressive model for universal composability," *J. Cryptol.*, vol. 33, no. 4, pp. 1461–1584, 2020. [Online]. Available: <https://doi.org/10.1007/s00145-020-09352-1>
- [117] D. Rausch, R. Küsters, and C. Chevalier, "Embedding the UC model into the IITM model," in *Advances in Cryptology - EUROCRYPT 2022 - 41st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Trondheim, Norway, May 30 - June 3, 2022, Proceedings, Part II*, ser. Lecture Notes in Computer Science, O. Dunkelman and S. Dziembowski, Eds., vol. 13276. Springer, 2022, pp. 242–272. [Online]. Available: https://doi.org/10.1007/978-3-031-07085-3_9
- [118] R. Canetti, "Universally composable security," *J. ACM*, vol. 67, no. 5, pp. 28:1–28:94, 2020. [Online]. Available: <https://doi.org/10.1145/3402457>
- [119] M. Rambaud, "Bootstrapping Message-Linear-Constant-Round Consensus from a Bare PKI Setup, and Separation Bounds from the Idealized Message-Authentication Model," 2023. [Online]. Available: <https://perso.telecom-paristech.fr/rambaud/articles/lower.pdf>
- [120] S. Das, Z. Xiang, and L. Ren, "Powers of tau in asynchrony," *IACR Cryptol. ePrint Arch.*, p. 1683, 2022. [Online]. Available: <https://eprint.iacr.org/2022/1683>
- [121] M. Andrychowicz and S. Dziembowski, "Pow-based distributed cryptography with no trusted setup," in *Advances in Cryptology - CRYPTO 2015 - 35th Annual Cryptology Conference, Santa Barbara, CA, USA, August 16-20, 2015, Proceedings, Part II*, ser. Lecture Notes in Computer Science, R. Gennaro and M. Robshaw, Eds., vol. 9216. Springer, 2015, pp. 379–399. [Online]. Available: https://doi.org/10.1007/978-3-662-48000-7_19
- [122] J. Katz, A. Miller, and E. Shi, "Pseudonymous secure computation from time-lock puzzles," *IACR Cryptol. ePrint Arch.*, p. 857, 2014. [Online]. Available: <http://eprint.iacr.org/2014/857>

Appendix A.

τ_{scr}^{++} : Scalable General Accountability

When determining how to address arbitrary Byzantine behaviors (whether to mask, detect, or punish them) it is natural to classify these behaviors [12], [101]. Essentially, faults can be categorized into two types: *omission faults*, where a party fails to send a message that should be sent, and *commission faults*, where a party sends something it should not. Commission faults can further be divided into two subcategories: *equivocation*, where a party makes contradictory statements that a correct process could not simultaneously make, and *evasion*, where a party sends messages without having received the corresponding input messages. Importantly, it has been proven that if a protocol is t -resilient, then violating safety requires at least $t + 1$ commission failures from distinct Byzantine parties [12]. In other words, an attack cannot significantly exploit omission failures to evade detection after the fact.

There exists a well-studied simulation [55], [102], [103], [104], [105], traditionally attributed to Bracha [55], [102], of crash failures on top of Byzantine failures, based on reliable-broadcast [55]. In a nutshell, such a simulation can be viewed as a module θ that (1) connects the networking layer to a crash-resilient algorithm Π , and (2) only allows "benign" executions to reach Π by not forwarding any message from the networking layer to Π unless the sender's valid behavior has already been established. As a result, all Byzantine processes appear to Π as if they have crashed. More concretely, messages are sent via a secure-broadcast primitive, which guarantees that for each correct process p_i , all correct processes agree on a total order of the messages broadcast by p_i . Throughout the protocol, correct processes 'simulate' the behavior of other correct processes: they possess the necessary knowledge of the initial state, update the simulated state at each step, and to be accepted, each new message must be a legitimate output of p_i 's prescribed protocol, applied to the simulated behavior up to that point.

It has been observed that Bracha's simulation, as referred to here, can be adjusted to ensure that evasion faults are *masked* (i.e., their effects are neutralized) regardless of the number of Byzantine failures, leaving *only* equivocation faults as a potential threat to safety [12]. Consequently, since reliable broadcast can tolerate up to $\lceil n/3 \rceil - 1$ Byzantine failures, it follows that any t_c -crash-resilient protocol can be automatically compiled into a protocol that tolerates up to $\min(t_c, \lceil n/3 \rceil - 1)$ Byzantine failures [102, Chapter 12], [55]. Moreover, by using an accountable version of the reliable broadcast primitive, a t_c -crash-resilient protocol can also be automatically compiled into an accountable protocol tolerating up to $\min(t_c, \lceil n/3 \rceil - 1)$ Byzantine failures [12]. The transformation described above, while general, incurs a multiplicative communication overhead, that is proportional to the communication complexity of the underlying Byzantine Reliable Broadcast primitive.

Any subquadratic Byzantine Reliable Broadcast protocol [48], [56], can be transformed using the ABC^{++} compiler into a subquadratic accountable version. This accountable version can then be integrated into the τ_{scr} transformation to produce τ_{scr}^{++} , an enhanced version that compiles any deterministic protocol into its accountable counterpart with subquadratic communication overhead, instead of the quadratic overhead in τ_{scr} .

Additional techniques [39], [40], [41] can further extend the applicability of such transformations when preserving hyperproperties (e.g., privacy, fairness) is required.

Appendix B.

Technical comparison with prior works

This section provides the missing technique-level comparison. We first discuss the player-replaceable forensic support of Sheng *et al.* [5], before contrasting it with the ABC compiler [2], [11].

B.1. Player-Replaceable Forensic Support

We begin by recalling the *deterministic* HotStuff-style forensic argument of Sheng *et al.* [4], then explain why this property collapses under *player replaceability*, and finally describe how the *transition certificates* (TCs) introduced by the same authors are intended to repair it [5].

Deterministic BFT Forensics. HotStuff-like protocols progress through a sequence of *views*, each led by a designated process driving a (pipelined) *certified gradecast*. In each view v , the leader promotes a proposal block b . When allowed by the local locking mechanism, replicas emit partially signed votes supporting the proposal. Aggregating at least $n - t$ such votes yields a *quorum certificate* (QC), a multi-signature certifying (v, b) (along with additional metadata). Votes occur at different levels (e.g., *prepare*, *pre-commit*, *commit*), each representing a higher level of "grade" or confidence. Once a *commit* QC is received, a replica finalizes the corresponding block b . This QC is built from a quorum Q_{com} , whose correct members must already have obtained a *locked* QC for the same block b and view v . To *unlock* from b to a conflicting block b' , a member of Q_{com} must observe a higher-view lock QC for b' . Meanwhile, members of Q_{com} are prohibited from voting for any block conflicting with the one they are locked on. With static committees, $f \leq t < n/3$, and quorum size

$n-t$, classical quorum intersection guarantees *agreement*. For forensic support, suppose process p_i finalizes b in view v via Q_{com} , while a conflicting block b' is finalized later in view $v' > v+2$. Then there exists a minimal view $v^* \in [v+1, v']$ with a *lock* QC, built from a quorum Q_{lock}^* , for some block b^* conflicting with b . If each vote includes the justifying view v_{jus} (the view of the highest lock QC the voter knows), the minimality of v^* implies $v_{\text{jus}} < v$. Hence, $G \triangleq Q_{\text{lock}}^* \cap Q_{\text{com}}$ designates replicas that *locked* on b at view v yet later *voted* for b^* conflicting with b , thereby violating the locking rule. Every member of G is therefore guilty. Moreover, when $f < n-t$, the QC at view v^* is observed by some correct replica q^* among the first quorum that extends b^* . Together, p_i and q^* hold irrefutable signed evidence of misbehavior by G .

In this deterministic setting, *fairness* (no false accusation) holds even if f later exceeds the completeness threshold t_{acc} , whereas *completeness* (the ability to identify a target number of culprits) requires $f \leq t_{\text{acc}} \leq n-t-1$. The $\mathcal{ABC}$ compiler achieves the optimal case $t_{\text{acc}} = n$.

Player-Replaceable BFT Forensics. HotStuff is *player-replaceable*: per-view leaders and per-round committees are sampled using a VRF. In the terminology of [5], each view decomposes into three consecutive *rounds*. Each round consists of a (sampled) leader-to-all *proposal step*, followed by a (sampled) committee-to-all *vote step*. As in HotStuff, a sampled leader proposes a block b . Collecting at least $t_H = \lceil \frac{2\lambda}{3} \rceil$ votes from the round's committee yields a quorum certificate QC_r for b . When three consecutive QCs are formed, the head block is finalized directly (and its ancestors indirectly). In the deterministic setting, the classical forensic argument identifies culprits via the *intersection* of two large, fixed quorums, for example $Q_{\text{com}} \cap Q_{\text{lock}}^*$. Under player replaceability, however, the commit QC for b and the conflicting lock QC for b^* are produced by *independently sampled* committees. As a result, $\hat{Q}_{\text{com}} \cap \hat{Q}_{\text{lock}}^*$ is typically empty, so the simple deterministic overlap argument no longer applies.

To address this, Sheng *et al.* [5] introduce *transition certificates* (TCs). During the subquadratic committee-to-all vote step (rather than a committee-to-leader step), committee members broadcast their freshest *lockedQC*. Each replica must *not* enter round $r+1$ until it has collected t_H such lock reports from verifiably elected committee members, forming the *transition certificate* $TC[r]$. This certificate is generated and disseminated at every round by the sampled processes. For fault levels up to $f \leq n(\frac{2}{3} - \epsilon)$, this echoing mechanism is claimed to ensure that at least one honest lock extending the freshest QC is included in every correct replica's $TC[r+1]$ before it advances. Informally, this keeps all correct replicas' freshest *lockedQC* *aligned within* one round. Hence, if a block b is finalized at round r via a quorum $\hat{Q}_{\text{com}}$, while a later chain first diverges at some minimal round $r^* \geq r+2$ under the votes of quorum $\hat{Q}_{\text{lock}}^*$, the members of $\hat{Q}_{\text{lock}}^*$ can be deemed faulty. Indeed, the locked QC associated with b 's finalization should have appeared among the t_H lock reports in some transition certificates between rounds r and r^* . Thus, any member of $\hat{Q}_{\text{lock}}^*$ who voted for a conflicting block effectively *voted behind its lock*.

B.2. $\mathcal{ABC}$

The $\mathcal{ABC}$ compiler introduced two main contributions: (1) the *accountable confirmer* object, which can be sequentially composed with any easily accountable agreement task; and (2) a *quadratic* implementation resilient against a strongly adaptive adversary, for $t < n/3$ and $t_{\text{acc}} = n$.

The implementation follows a classical all-to-all communication pattern. In the first round, processes broadcast their partially signed (pre-)decisions. Retrospectively, this phase implicitly realizes what we now formalize as the *ratifier* functionality, albeit at quadratic communication cost, even though adapting it to a player-replaceable setting is conceptually straightforward. After collecting partial signatures from a standard quorum, processes confirm their decision and enter a second all-to-all phase, during which they exchange signatures corresponding to potentially conflicting decisions. In hindsight, this second phase implements what we now define as the *propagator* functionality, again at quadratic cost.

The $\mathcal{ABC}^{++}$ compiler does not introduce new sophisticated mathematical tools, but refines $\mathcal{ABC}$ by adapting subquadratic *all-to-all gossiping* to accountable certificate propagation. It replaces the quadratic all-to-all phases of [2], [11] with subquadratic counterparts that preserves the same accountable guarantees.

Appendix C.

$\mathcal{ABC}^{++}$ in the Universal Composability Framework

In this section, we provide a proof of the $\mathcal{ABC}^{++}$ transformation in the Universal Composability framework.

Universal Composability. Universal composability (UC) [106] is a widely used framework for designing, modeling, and analyzing security protocols because of its simulation-based security guarantees and its natural support for modularity. In this framework, the first step is to define an *ideal functionality* $\mathcal{F}$, which specifies the desired behavior and security properties of the protocol in an abstract manner, omitting implementation details. For example, $\mathcal{F}$ might represent a trusted and incorruptible third party that helps the parties execute the computation. Each party sends its input to the trusted party,

who computes some desired function on top of as many inputs as it can expect, and returns the prescribed output to each party. The adversary’s only power in this ideal world is to choose the inputs for the corrupted parties, and which inputs might be ignored due to asynchrony. In the *real world*, parties communicate by message-passing. The adversary can manipulate the order in which messages are delivered, and arbitrarily control the corrupted processes. To prove that a concrete protocol $\mathcal{P}$ realizes an ideal functionality $\mathcal{F}$, it must be demonstrated that for any adversary $\mathcal{A}$ attacking $\mathcal{P}$ in the *real world*, there exists a simulator $\mathcal{S}$, acting as an adversary in the *ideal world*, such that the interactions of $\mathcal{A}|\mathcal{P}$ and $\mathcal{S}|\mathcal{F}$ with an external environment $\mathcal{E}$ (serving as a distinguisher) are computationally indistinguishable. One of the core benefits of UC is its composability: once a protocol $\mathcal{P}$ is proven to realize $\mathcal{F}$, it can replace $\mathcal{P}$ as a subroutine in larger protocols without compromising security. This modular design and analysis are further guaranteed by the composition theorem inherent in the UC framework, ensuring that protocols built atop $\mathcal{P}$ remain secure even after such substitutions.

Guaranteed Output Delivery in Asynchronous UC. In the UC framework, “ensuring liveness” is nuanced due to polynomially bounded executions. Traditionally, a property is said to *eventually* hold if, for every infinite execution where each machine/task is activated infinitely often, the property is not indefinitely false [107, Chapter 8.5].

In the framework of [108], adapted from [109], a party must actively request an output from the functionality $\mathcal{F}$. The simulator $\mathcal{S}$ can delay the output by ignoring requests for a polynomial number of activations. If the environment activates the party sufficiently often, the party will eventually receive its output from $\mathcal{F}$. This guarantees termination, ensuring all honest parties eventually obtain their outputs if the environment allocates enough resources.

Instead, we adopt the framework proposed by Shoup et al. [71], [110], [111], [112], [113], which reinterprets the notion of “eventually holds” by requiring that the property holds if and when all messages sent from one honest party to another are eventually delivered. However, without a constraint on the total message complexity, liveness properties could be trivially satisfied by protocols that perpetually generate new messages without delivering earlier ones. This motivates the need for *efficient* protocols, where communication and computational complexities are uniformly bounded. These bounds are fixed polynomials in the security parameter and the number of parties, holding with all but negligible probability for any polynomially bounded adversary. This efficiency is preserved under protocol composition and simplifies the definition of *guaranteed output delivery*. Clearly, we only propose efficient protocols.

Following [22], [114], we use the term *delayed output* to describe an output whose delivery is triggered by the adversary after an arbitrary amount of time.

Accountable Universal Composability (AUC). The AUC framework [20] extends Universal Composability (UC) with explicit accountability guarantees. It builds on iUC [115], which itself instantiates the IITM framework [116]; UC embeds into IITM [117]. In IITM and iUC, composition with joint or global state follows directly from the base composition theorems and does not require the GUC or JUC extensions: a globally available resource is simply modeled by exposing enough I/O tapes for the environment and all relevant protocols to connect to the same instance.

In AUC, ideal functionalities additionally specify how violations of their guarantees (e.g., breaches of agreement) must expose the responsible parties. Each accountable functionality is associated with a *public judge* J_{rat} (i.e., a protocol that can be locally executed by any process, including external clients), which produces *verdicts* based on *publicly verifiable evidence*. We consider judges that ensure (1) *fairness*—honest parties are never falsely blamed, (2) *external verifiability*—third parties can check evidence independently, and (3) *individual accountability*—each verdict has the form $\text{dis}(D)$, indicating that all parties in the set D are deemed dishonest. For simplicity, we refer to one globally accessible instance J_{rat} , although any correct process may locally execute the same code.

AUC also introduces a meta-party called the *supervisor*, which tracks violations of the system’s *underlying assumptions* (for example, synchrony conditions) to ensure that ideal and real executions remain aligned when assumptions fail. In our setting, the only tracked assumption is the fault threshold condition $f \leq t$, which is already managed through standard corruption/status queries; hence, we do not explicitly use the supervisor.

Accountable functionalities extend the adversarial interface with dedicated *Break* requests. We consider two: **BreakLiveness** and **BreakAgreement**(*report*), both effective only when $f > t$. If **BreakLiveness** is accepted, the adversary may indefinitely delay output notifications. If **BreakAgreement**(*report*) is accepted, the adversary may violate agreement as specified by the functionality, but only if *report* is a valid *judicial report* that enables incrimination: specifically, it must allow each correct process p_i to derive a proof π_i such that, if submitted to the judge, it yields a verdict $\text{dis}(D_i)$ with $|D_i|$ exceeding the functionality’s accountability threshold. Formally, the functionality must also accept **BreakAgreement**(*report*) when $f \leq t$ *without* affecting safety or liveness; in that case it merely disseminates the per-party proofs π_i . This models *absurd self-incrimination* (voluntary self-reporting without any strategic gain) which can occur in the real world and must remain simulatable.

The generation and verification of judicial reports are handled by auxiliary functionalities $\mathcal{F}_{\text{incert}}$ and $\mathcal{F}_{\text{lottery}}$, presented in §C.1, which serve as subroutines for $\mathcal{F}_{\text{rat}}$, $\mathcal{F}_{\text{prop}}$, and $\mathcal{F}_{\text{conf}}^{\text{acc}}$. Accordingly, the accountable realizations operate in the $\mathcal{J}$ -hybrid model, where $\mathcal{J} = J_{\text{rat}} \mid \mathcal{F}_{\text{incert}} \mid \mathcal{F}_{\text{lottery}}$. This setting captures *external verifiability* while keeping the environment interface identical in both ideal and real worlds.

Proof structure. We begin, in §C.1, by introducing the ideal functionalities $\mathcal{F}_{\text{mcert}}$ and $\mathcal{F}_{\text{lottery}}$, which model multi-certification and (aggregatable) lottery primitives, respectively. Next, we prove in §C.2 that the real protocol $\mathcal{P}_{\text{rat}}$ realizes the ratifier functionality $\mathcal{F}_{\text{rat}}$. In §C.3, we rely on a result from [18], stated in the 2020 revision of Canetti’s UC framework [118], which shows that the propagator functionality $\mathcal{F}_{\text{prop}}$ can be directly realized. This result is compatible with the IITM framework [117], from which the AUC framework inherits its composition guarantees.

We then show, in §C.4, that the sequential composition $\mathcal{F}_{\text{rat}} \triangleright \mathcal{F}_{\text{prop}}$ realizes the accountable confirmer functionality $\mathcal{F}_{\text{conf}}^{\text{acc}}$. Finally, in §C.5, we describe the ABC^{++} compiler, which sequentially composes any *easily accountable* functionality $\mathcal{F}_{\text{easy}}$ (see Definition 3), such as consensus, reliable broadcast, consistent broadcast, or one-to-many zero-knowledge proofs, with $\mathcal{F}_{\text{conf}}^{\text{acc}}$ to obtain its accountable counterpart.

C.1. Cryptographic Building Blocks

This subsection presents the cryptographic functionalities used in our construction: the multi-certification functionality $\mathcal{F}_{\text{mcert}}$, the (potentially aggregatable) lottery functionality $\mathcal{F}_{\text{lottery}}$, and the globally available judge J_{rat} , which issues verdicts.

C.1.1. The Multi-Certification Functionality. The first functionality is the *multi-certification functionality* $\mathcal{F}_{\text{mcert}}$ (Functionality 1–2), the multi-signature analogue of the certification functionality $\mathcal{F}_{\text{cert}}$. It provides a direct binding between a multi-signature on a message and the set of signers’ identities.

Following Canetti [92], $\mathcal{F}_{\text{mcert}}$ can be realized from a (multi-)signature functionality $\mathcal{F}_{(\text{m})\text{sig}}$ together with a certification authority (PKI) $\mathcal{F}_{\text{CA}}$. Each process is bound to a unique public key by $\mathcal{F}_{\text{CA}}$. Moreover, $\mathcal{F}_{\text{CA}}$ must prevent *rogue-key attacks*, in which a set of corrupted signers maliciously choose their public keys as functions of honest users’ keys. The BLS-based scheme of [88] is resistant to such attacks, thereby avoiding traditional countermeasures that require users to prove knowledge or possession [91] of their secret key during registration (the so-called *KOSK* assumption).

The specification of $\mathcal{F}_{\text{mcert}}$ in Functionality 1–2 essentially follows the presentation of [92].

Functionality 1 $\mathcal{F}_{\text{mcert}}$ – Signing, Aggregating

//A certification authority functionality $\mathcal{F}_{\text{ca}}$ allows processes to agree on one unique public key per process

Signature Generation:

Upon receiving $(\text{sign}, \text{sid}, p_i, m)$ from p_i :
 Forward $(\text{sign}, \text{sid}, p_i, m)$ to the adversary.
 Upon receiving $(\text{signature}, \text{sid}, p_i, m, \sigma_i)$ from the adversary:
 If no entry $(\text{sid}, m, \sigma_i, 0)$ is recorded:
 Record the entry $(\text{sid}, m, \sigma_i, \{p_i\}, 1)$.
 Output $(\text{signature}, \text{sid}, p_i, m, \sigma_i)$ to p_i .

Signature Combination:

Upon receiving $(\text{cb}, \text{sid}, G, m, \{\sigma_j\}_{p_j \in G})$, from requester R , s.t. $G \subseteq \Psi \wedge |G| \geq 2$:
 Forward $(\text{cb}, \text{sid}, G, m, \{\sigma_j\}_{p_j \in G})$ to the adversary.
 Upon receiving $(\text{cb}, \text{sid}, G, m, \{(\phi_j)\}_{p_j \in G}, \sigma_G)$ from the adversary:
 Determine $f_G \leftarrow \text{BatchIndividualVerif}(\text{sid}, G, m, \{(\sigma_j, \phi_j)\}_{p_j \in G})$
 If $f_G = 1$:
 Record the entry $(\text{sid}, m, \sigma_G, G, 1)$.
 Output $(\text{multisignature}, \text{sid}, G, m, \sigma_G)$ to R .
 Else:
 Output $(\text{multisignature}, \text{sid}, G, m, \perp)$ to R .

C.1.2. The Lottery Functionality $\mathcal{F}_{\text{lottery}}$. The second functionality is the (potentially aggregatable) *lottery functionality* $\mathcal{F}_{\text{lottery}}$ (Functionality 3). Following Rambaud [119], $\mathcal{F}_{\text{lottery}}$ can be realized from a verifiable random function (VRF) functionality $\mathcal{F}_{\text{vrf}}$ and a “synchronous” certification authority $\mathcal{F}_{\text{CA}}^{\tau_0}$, under the assumption that each correct process can publish its public key before a global start time τ_0 .

A trivial way to aggregate election tickets is to concatenate them. However, under a power-of- τ (also known as q -SDH) setup [120], recent work [19] has shown that the aggregation-friendliness of BLS [87] can be leveraged to realize an *aggregate lottery functionality* where multiple eligibility proofs are combined into just two group elements, verifiable with a single pairing check. This yields a concretely efficient implementation of $\mathcal{F}_{\text{lottery}}$.

Theorem 1. Let $H(s, \lambda)$ denote the random variable that returns the set of (so-far honest) processes p_i such that the sampling event $\text{sample}_i(s, \lambda)$ is not preceded by the corruption of p_i in the execution, and let $C(s, \lambda) = \{p_i \in \Psi \mid \text{sample}_i(s, \lambda) = \langle \text{true}, * \rangle\}$, sampled uniformly at random with expected size λ .⁵

5. Here, $\Pr$ represents the worst-case probability over adversarial strategies with up to t adaptive corruptions (no liveness guarantees when $f > t$).

Functionality 2 $\mathcal{F}_{\text{mcert}}$ – Verifying

Signature Verification:

Upon receiving (verify, sid, G, m, σ_G), s.t. $G \subseteq \Psi \wedge |G| \geq 1$:
 Forward (verify, sid, G, m, σ_G) to the adversary.
 Upon receiving (verified, sid, G, m, ϕ_G) from the adversary:
 Determine $f_G \leftarrow \text{Verif}(\text{sid}, G, m, \sigma_G, \phi_G)$
 Output (verified, sid, G, m, σ_G, f_G) to the requester (process or judge).

Verification Procedure:

Verif(sid, G, m, σ, ϕ_G):
 (1) If the entry (sid, $m, \sigma, G, 1$) is recorded: return $f = 1$.
(Completeness: Verification succeeds for valid (multi-)signatures.)
 (2) Else, if $\exists p_j \in G \cap \text{Correct}$, and no entry (sid, $m, \sigma', G' \ni p_i, 1$) is recorded:
 record the entry (sid, $m, \sigma, G, 0$) and return $f = 0$.
(Unforgeability: Verification fails if one signer did not sign m .)
 (3) Else, if an entry (sid, m, σ, G, f') is recorded: return $f = f'$.
(Consistency: Identical parameters produce the same verification result.)
 (4) Else: record the entry (sid, m, σ, G, ϕ_G) and return $f = \phi_G$.

BatchIndividualVerif(sid, $G, m, \{(\sigma_j, \phi_j)\}_{p_j \in G}$):
 Determine $\{f_j \leftarrow \text{Verif}(\text{sid}, \{p_j\}, m, \sigma_j, \phi_j)\}_{p_j \in G}$
 Return $\bigwedge_{p_j \in G} f_j$

- **Liveness.** With $W = (1 - \delta)(\frac{2}{3} + \epsilon)\lambda$, we have:

$$\Pr[|C(s, \lambda) \cap H(s, \lambda)| \leq W] \leq \exp\left(-\frac{\delta^2}{6}(2 + 3\epsilon)\lambda\right)$$

- **Forensics.** Let $Q, Q' \subseteq C(s, \lambda)$ with $|Q| = |Q'| = W_{\delta}^{\epsilon, \lambda}$. Let $B = 2W_{\delta}^{\epsilon, \lambda} - (1 + \hat{\delta})\lambda$. Then:

$$\Pr[|Q \cap Q'| \leq B] \leq \exp\left(-\frac{\hat{\delta}^2}{2 + \hat{\delta}}\lambda\right)$$

PROOF. Let X_j be the Bernoulli random variable that indicates if $p_j \in C(s, \lambda)$, so $\Pr[X_j = 1] = \lambda/n$. Define $Z_L = \sum_{p_j \in H} X_j$ and $Z_F = \sum_{p_j \in \Pi} X_j$. By linearity of expectation, $\mathbb{E}(Z_L) \geq (\frac{2}{3} + \epsilon)\lambda$ and $\mathbb{E}(Z_F) = \lambda$.

- (Liveness) $|C(s, \lambda) \cap H(s, \lambda)| \sim Z_L$. By Chernoff bound, $\Pr[Z_L \leq (1 - \delta) \cdot (\frac{2}{3} + \epsilon)\lambda] \leq \exp\left(-\frac{\delta^2(2+3\epsilon)}{6}\lambda\right)$
- (Forensic) $|Q \cap Q'| \geq |Q| + |Q'| - |Q \cup Q'| \geq 2W_{\delta} - Z_F$.

By Chernoff bound:

$$\begin{aligned} \Pr[Z_F \geq (1 + \hat{\delta}) \cdot \lambda] &\leq \exp\left(-\frac{\hat{\delta}^2}{2 + \hat{\delta}}\lambda\right) \\ \Pr[2W_{\delta} - Z_F \leq 2W_{\delta} - (1 + \hat{\delta}) \cdot \lambda] &\leq \exp\left(-\frac{\hat{\delta}^2}{2 + \hat{\delta}}\lambda\right) \\ \Pr[|Q \cap Q'| \leq B_{\delta, \hat{\delta}}] &\leq \exp\left(-\frac{\hat{\delta}^2}{2 + \hat{\delta}}\lambda\right) \end{aligned}$$

□

C.1.3. The Public Judge. We now specify the globally available *public judge* J_{rat} (and its extended version $J_{\text{rat},+}$) in Functionality 4. The judge is connected via I/O tapes to $\mathcal{F}_{\text{mcert}}$ and $\mathcal{F}_{\text{lottery}}$ (for $J_{\text{rat},+}$ only).

On a request

$$(\text{verdict}, \text{sid}, (m^1, Q^1, \sigma^1), (m^2, Q^2, \sigma^2)),$$

the judge J_{rat} (or $J_{\text{rat},+}$) queries $\mathcal{F}_{\text{mcert}}$ to verify that, for each $i \in \{1, 2\}$, σ^i is a valid multi-signature by Q^i on message m^i . If both signatures are valid and the messages conflict (e.g., same header but distinct payloads), it returns the verdict $\text{dis}(Q^1 \cap Q^2)$. Making J_{rat} globally accessible enables *external verifiability*: any third party, even if it did not participate in the protocol, can validate the evidence.

On a request

$$(\text{fully_verify}, \text{sid}, (m, Q, \sigma, \text{ticket})),$$

the enhanced judge $J_{\text{rat},+}$ verifies that σ is a valid multi-signature by Q on m via $\mathcal{F}_{\text{mcert}}$, and that *ticket* is a valid lottery ticket for Q via $\mathcal{F}_{\text{lottery}}$.

When the session ID sid is fixed, we note $\text{valid}(c)$, if $J_{\text{rat},+}$ returns a positive answer upon the reception of request $(\text{fully_verify}, \text{sid}, c)$. Similarly, for every pair of certificates $\pi = (c, c')$, we note $\text{conflict}(c, c') = \text{true}$ if $J_{\text{rat},+}$ returns a positive

Functionality 3 $\mathcal{F}_{\text{lottery}}$: Aggregatable Lottery (simplified)

Parameters:

$\lambda \in \mathbb{N}$: Statistical security parameter
 $p_1, \dots, p_n$: Parties interacting with the functionality
 Sim : Ideal process adversary (simulator)

Local State:

T : A table of tuples $(\text{sid}, Q, \text{ticket})$, initially empty

Functionality:

Upon receiving $(\text{participate}, \text{sid})$ from party p_i :
 Sample $c_i \sim \text{Bernoulli}(\lambda/n)$
if $c_i = 1$:
 Generate a fresh ticket ticket
 Store $(\text{sid}, \{p_i\}, \text{ticket})$ in T
 Send $(\text{election}, \text{sid}, \text{ticket})$ to p_i
else:
 Send $(\text{election}, \text{sid}, \perp)$ to p_i

Upon receiving $(\text{aggregate}, \text{sid}, Q, \text{Tickets})$ from requester R :
 Let $Q = \{p_{j_1}, \dots, p_{j_{|Q|}}\}$ and $\text{Tickets} = (\text{ticket}_1, \dots, \text{ticket}_{|Q|})$
if $\forall k \in [1..|Q|], (\text{sid}, \{p_{j_k}\}, \text{ticket}_k) \in \mathsf{T}$
 Generate a fresh aggregated ticket ticket_Q
 Store $(\text{sid}, Q, \text{ticket}_Q)$ in T
 Send $(\text{aggregation}, \text{sid}, \text{ticket}_Q)$ to R
else:
 Send $(\text{aggregation}, \text{sid}, \perp)$ to R

Upon receiving $(\text{verify}, \text{sid}, Q, \text{ticket})$ from requester R :
 $b := ((\text{sid}, Q, \text{ticket}) \in \mathsf{T})$:
 Send $(\text{verified}, \text{sid}, Q, \text{ticket}, b)$ to R

answer upon the reception of request $(\text{verdict}, \text{sid}, \pi)$. We note $c \equiv c'$ iff $\text{valid}(c) = \text{valid}(c') = \text{true}$ and $\text{conflict}(c, c') = \text{false}$.

Ideal accountable functionalities also interact with $J_{\text{rat},+}$ to validate requests from the ideal adversary, who must select concrete *judicial reports* (certificates and proofs) as parameters for delayed outputs. These reports can be produced by the ideal adversary through $\mathcal{F}_{\text{mcert}}$ and $\mathcal{F}_{\text{lottery}}$, used as subroutines of the ideal accountable functionality.

Functionality 4 $J_{\text{rat}}/J_{\text{rat},+}$ – Globally available

Verdict request:

Upon receiving $(\text{verdict}, \text{sid}, (m^1, Q^1, \sigma^1), (m^2, Q^2, \sigma^2))$ from a requester R :
 $\forall i \in \{1, 2\}$: send $(\text{verify}, \text{sid}, Q^i, m^i, \sigma^i)$ to $\mathcal{F}_{\text{mcert}}$
 When $(\text{verified}, \text{sid}, Q^i, m^i, \sigma^i, f^i)$ is received from $\mathcal{F}_{\text{mcert}}$ for each $i \in \{1, 2\}$:
 if $m^1.\text{header} = m^2.\text{header} \wedge m^1.\text{payload} \neq m^2.\text{payload} \wedge f^1 = f^2 = \text{true}$:
 send $(\text{verdict}, \text{sid}, \text{dis}(Q^1 \cap Q^2))$ to R . (**equivocation detected**)
 else
 send $(\text{verdict}, \text{sid}, \perp)$ to R . (**insufficient evidence**)

//for $J_{\text{rat},+}$ only

Full validity request:

Upon receiving $(\text{fully_verify}, \text{sid}, (m, Q, \sigma, \text{ticket}))$ from a requester R :
 Send $(\text{verify}, \text{sid}, Q, m, \sigma)$ to $\mathcal{F}_{\text{mcert}}$
 Send $(\text{verify}, \text{sid}, Q, \text{ticket})$ to $\mathcal{F}_{\text{lottery}}$
 When $(\text{verified}, \text{sid}, Q, m, \sigma, \text{bool}_{\text{msig}})$ and $(\text{verified}, \text{sid}, Q, \text{ticket}, \text{bool}_{\text{elected}})$ are received from $\mathcal{F}_{\text{mcert}}$ and $\mathcal{F}_{\text{lottery}}$ respectively:
 send $(\text{full_validity}, \text{sid}, \text{bool}_{\text{msig}} \wedge \text{bool}_{\text{elected}})$ to R .

We denote

$$\mathcal{J} = J_{\text{rat}} \mid \mathcal{F}_{\text{mcert}} \quad \text{and} \quad \mathcal{J}_+ = J_{\text{rat},+} \mid \mathcal{F}_{\text{mcert}} \mid \mathcal{F}_{\text{lottery}}.$$

Later, we introduce the ideal ratifier functionality $\mathcal{F}_{\text{rat}}$ and the ideal propagator functionality $\mathcal{F}_{\text{prop}}$, both of which use $\mathcal{J}_+$ as a subroutine. This allows the functionalities to leak cryptographic material (serving as judicial evidence) to the ideal adversary, and to formalize both *full certificates* and *conflicting certificates* directly within the UC framework through their interaction with the enhanced global judge $J_{\text{rat},+}$.

The reader may already refer to Figure 1 (Section C.2) for an overview of this architecture. A similar structure appears in the ideal propagator functionality $\mathcal{F}_{\text{prop}}$ and in the accountable confirmer functionality $\mathcal{F}_{\text{conf}}^{\text{acc}}$, though the latter only relies on $\mathcal{J}$ as a subroutine.

We thus introduce the local subroutines defined in Functionality 5, which are used by $\mathcal{F}_{\text{rat}}$, $\mathcal{F}_{\text{prop}}$, and $\mathcal{F}_{\text{conf}}^{\text{acc}}$.

Functionality 5 Subroutines on top of $\mathcal{F}_{\text{mcert}}$ and $\mathcal{F}_{\text{lottery}}$

Participants:

$p_1, \dots, p_n$: Parties interacting with the functionality.
 Sim: Ideal adversary (simulator).

Local Subroutine on top of $\mathcal{F}_{\text{mcert}}$:

Upon subroutine_sign(sid, p_i , m):
 Send (sign, sid, m) to $\mathcal{F}_{\text{mcert}}$'s input tape dedicated to process p_i .
 Upon receiving (signature, sid, m , σ_i) from $\mathcal{F}_{\text{mcert}}$'s output tape dedicated to process p_i .
 Send (signature, p_i , m , σ_i) to Sim

▷ Leak to Sim some signature material

Upon subroutine_cb(sid, G , m , $\{\sigma_j\}_{p_j \in G}$):
 Send (cb, sid, G , m , $\{\sigma_j\}_{p_j \in G}$) to $\mathcal{F}_{\text{mcert}}$
 Upon receiving (multisignature, sid, G , m , output) from $\mathcal{F}_{\text{mcert}}$
 Send (multisignature, sid, G , m , output) to Sim

▷ Allow Sim to generate proofs

Local Subroutine on top of $\mathcal{F}_{\text{lottery}}$:

Upon subroutine_participate(sid, p_i):
 Send (participate, sid) to $\mathcal{F}_{\text{lottery}}$'s input tape dedicated to process p_i .
 Upon receiving (election, sid, output) from $\mathcal{F}_{\text{lottery}}$
 Send (election, sid, output) to Sim
 return output

▷ Leak to Sim some election-proof material

Upon subroutine_agg(sid, G , Tickets):
 Send (aggregate, sid, G , Tickets) to $\mathcal{F}_{\text{lottery}}$
 Upon receiving (aggregation, sid, output) from $\mathcal{F}_{\text{lottery}}$
 Send (aggregation, sid, output) to Sim

▷ Allow Sim to generate proofs

Local Subroutine triggered by Sim:

Upon receiving (sign, sid, p_j , m) from Sim with p_j corrupted:
trigger subroutine_sign(sid, p_j , m)

 Upon receiving participate(sid, p_j) from Sim with p_j corrupted:
trigger participate(sid, p_j)

 Upon receiving (cb, sid, G , m , $\{\sigma_j\}_{p_j \in G}$) from Sim
trigger subroutine_cb(sid, G , m , $\{\sigma_j\}_{p_j \in G}$)

 Upon receiving (aggregate, sid, G , Tickets) from Sim
trigger subroutine_agg(sid, G , Tickets)

C.2. The Ratifier Functionality $\mathcal{F}_{\text{rat}}$

C.2.1. $\mathcal{F}_{\text{rat}}$'s Specification. When an easily accountable functionality (e.g., $\mathcal{F}_{\text{vba}}$, $\mathcal{F}_{\text{rbc}}$, or $\mathcal{F}_{\text{zkPoK}}^{1:M}$) yields a (pre-)decision v , processes must determine whether they unanimously agree on v . Furthermore, this confirmation must be accompanied by an externally verifiable *certificate*, enabling an external observer (such as a judge instance) to detect inconsistencies and identify dishonest processes when conflicting certificates are presented. These requirements are formally captured by the ideal functionality $\mathcal{F}_{\text{rat}}$, introduced in Functionality 6.

Namely, $\mathcal{F}_{\text{rat}}$ ensures the following properties:

- **Integrity:** For any $f \in [n]$, a correct process cannot confirm a value that it did not submit.
- **Optimistic convergence:** If $f \leq t$ and all correct process submit the same value v , then every correct process eventually confirms that value.
- **Validity:** If a so-far correct process triggers (confirm, sid, m , c), then $\text{valid}(c) = \text{true}$.
- **Forensic support for agreement:** If two correct processes p_i and p_j output different values, they respectively store conflicting certificates c_i and c_j , such that, when presented to $\mathcal{J}_{\text{rat},+}$, this latter returns the verdict $\text{dis}(D)$, exposing $|D| \geq d$ dishonest parties.

C.2.2. $\mathcal{F}_{\text{rat}}$'s Realization. We present the $\mathcal{F}_{\text{rat}}$'s realization $\mathcal{P}_{\text{rat}}$ (see Algorithm 6). It refers to the (public and fair) judge described in Functionality 4.

To implement $\mathcal{F}_{\text{rat}}$, a VRF-selected committee of expected size λ broadcasts a SUBMIT message containing a hash of their signed pre-decided value to all other processes. Once a process receives at least $W_{\delta}^{\epsilon, \lambda} \simeq 2\lambda/3$ valid SUBMIT messages supporting the (hashed) pre-decision v , it confirms v and stores the signed messages from a “VRF-quorum” of size $W_{\delta}^{\epsilon, \lambda}$ for future verification. Optimistic convergence is ensured by the first Chernoff bound in Theorem 1.

In case of an agreement violation, two processes will store conflicting *certificates*, each consisting of $W_{\delta}^{\epsilon, \lambda}$ signed SUBMIT messages from a VRF-selected committee but for differing (hashed) values. These two certificates serve as undeniable

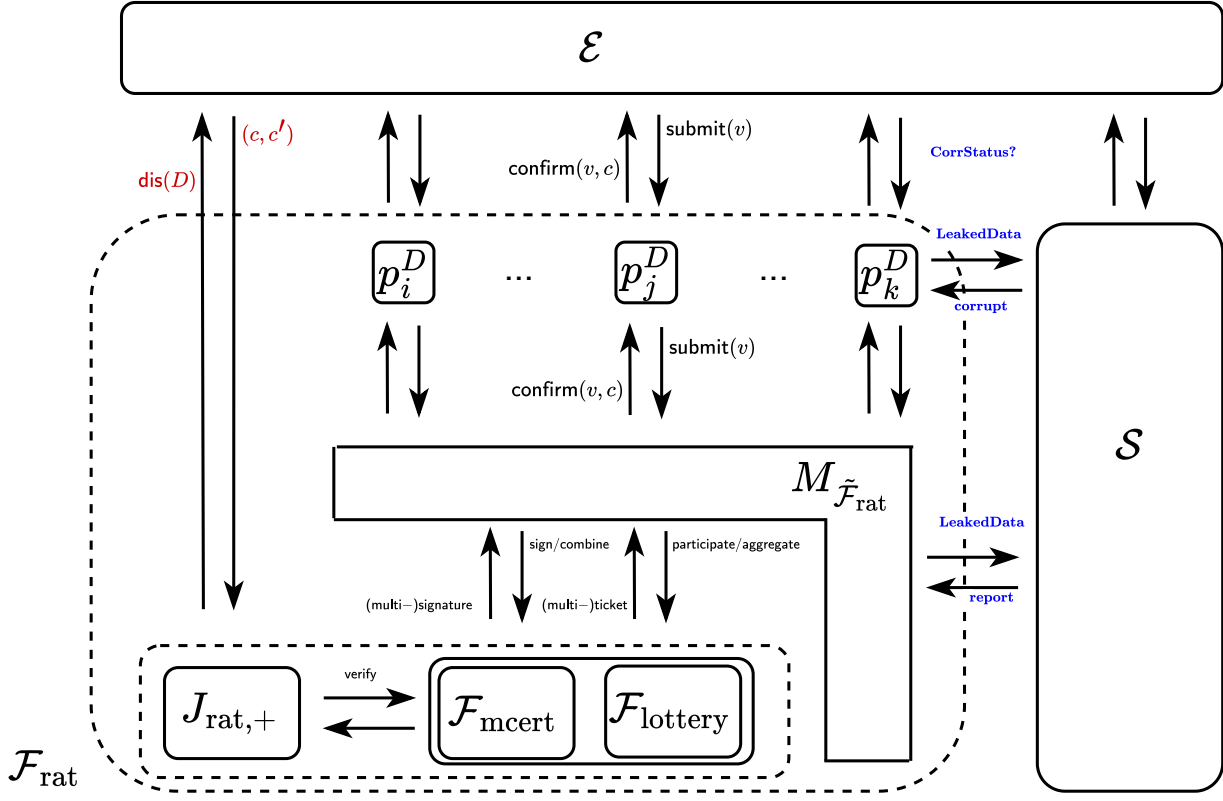


Figure 1: The ratifier functionality $\mathcal{F}_{\text{rat}}$

proof of misbehavior against processes that signed conflicting SUBMIT messages. With overwhelming probability, at least $B_{\delta, \delta}^{\epsilon, \lambda} \simeq \lambda/3$ processes will be provably dishonest, as supported by the second Chernoff bound in Theorem 1.

Lemma 1. $\mathcal{P}_{\text{rat}}$ (Algorithm 6), realizes the ideal functionality $\mathcal{F}_{\text{rat}}$ (Functionality 6) in the $\mathcal{I}_+$ -hybrid model, with t -resiliency, 1 round, and an expected message complexity of $O(\lambda n)$, each message being of size $O(\kappa)$.

PROOF. We consider the ideal system

$$\mathcal{F}_{\text{rat}} = p_1^D \mid \cdots \mid p_n^D \mid M_{\tilde{\mathcal{F}}_{\text{rat}}} \mid \mathcal{J}_+,$$

where $p_1^D, \dots, p_n^D$ are dummy processes, $\mathcal{J}_+ = \text{Judge}_+ \mid \mathcal{F}_{\text{mcert}} \mid \mathcal{F}_{\text{lottery}}$, and $M_{\tilde{\mathcal{F}}_{\text{rat}}}$ provides the interface tapes for dummy processes and the simulator.

The corresponding real system in the $\mathcal{I}_+$ -hybrid model is

$$\mathcal{P}_{\text{rat}} = p_1 \mid \cdots \mid p_n \mid \mathcal{J}_+ \mid \mathcal{F}_{\text{flood}}.$$

Let $\mathcal{A}$ be an adversary against $\mathcal{P}_{\text{rat}}$. We construct a simulator $\mathcal{S}$ for the ideal world.

Simulator. The simulator $\mathcal{S}$ internally executes a faithful simulation of the real execution

$$\mathcal{A}^{\text{sim}} \mid \mathcal{P}_{\text{rat}}^{\text{sim}} \quad \text{with} \quad \mathcal{P}_{\text{rat}}^{\text{sim}} = p_1^{\text{sim}} \mid \cdots \mid p_n^{\text{sim}} \mid \mathcal{J}_+^{\text{sim}} \mid \mathcal{F}_{\text{flood}}^{\text{sim}},$$

choosing its own randomness for $\mathcal{F}_{\text{lottery}}^{\text{sim}}$. No secret information of honest parties is required.

I/O forwarding. Messages from the environment $\mathcal{E}$ to $\mathcal{A}$ are forwarded to $\mathcal{A}^{\text{sim}}$. Likewise, outputs of $\mathcal{A}^{\text{sim}}$ addressed to $\mathcal{E}$ are forwarded back to $\mathcal{E}$ by $\mathcal{S}$.

Submit/confirm coupling. Whenever $\mathcal{S}$ receives from $\mathcal{F}_{\text{rat}}$ a notification (support, sid, p_i^D , m_i), it instructs p_i^{sim} to submit (submit, sid, m_i). Conversely, whenever p_i^{sim} outputs (confirm, sid, m_i , c_i), $\mathcal{S}$ sends (confirm, sid, p_i^D , m_i , c_i) to $\mathcal{F}_{\text{rat}}$. If the

Functionality 6 $\mathcal{F}_{\text{rat}}$

Parameters:

t : Optimistic Threshold
 t_{acc} : Pessimistic Threshold
 $d = B_{\delta, \delta}^{\epsilon, \lambda}$: Number of dishonest processes that should be exposed in case of disagreement

Variables:

Set(Message) *submitted* ▷ Set of submitted messages
Map(Message $\rightarrow$ Set(Process)) *supporters* ▷ Maps each message m to the set of processes submitting m .
Set(Process $\times$ Message $\times$ Certificate) *conf* ▷ Confirmation material

Subroutines:

$\mathcal{F}_{\text{mcert}}$: multi-certification functionality ▷ See Functionality 1-2 and subroutine in Functionality 5
 $\mathcal{F}_{\text{lottery}}$: lottery functionality ▷ See Functionality 3 and subroutine in Functionality 5

Participants:

$p_1, \dots, p_n$: Parties interacting with the functionality.
Sim: Ideal adversary (simulator).
Judge $_{+}$: (public and fair) judge

Events:

request (submit, sid, m): A party submits a message m .
notification (confirm, sid, m , c): A party confirms the message m , backed by a certificate c .

Functionality:

Upon receiving (submit, sid, m) from an uncorrupted process p_i :
 $submitted \leftarrow submitted \cup \{m\}$
 $supporters[m] \leftarrow supporters[m] \cup \{p_i\}$
Send (support, p_i , m) to Sim ▷ Notify Sim of the submission of m by p_i
//generating cryptographic material:
trigger $T \leftarrow$ subroutine_participate(sid, p_i): ▷ Leak election status and ticket if any
if $T \neq \perp$: ▷ p_i elected
 trigger subroutine_sign(sid, p_i , m): ▷ Leak signature
forensic($conf$):
 return true **if** $\forall (m, m') \in conf.\text{msgs}^2$ with $m \neq m'$, $\forall ((p_i, m_i, c_i), (p_j, m_j, c_j)) \in conf^2$ with $(m_i, m_j) = (m, m')$,
 for each request (verdict, sid, c_i, c_j), Judge $_{+}$ returns (verdict, sid, $dis(D_{ij})$) with $|D_{ij}| \geq d$, and *false* **otherwise**
Upon receiving a confirmation request (confirm, sid, $\hat{p}, \hat{m}, \hat{c}$) from Sim:
if Judge $_{+}$ returns *true* on request (fully_verify, sid, c_i), while $p_i \in supporters[m_i]$, and $c_i.\text{msg} = m_i$: ▷ Integrity and Validity
 Let $conf_{\text{new}} = conf \cup \{(\hat{p}, \hat{m}, \hat{c})\}$ and $conf_{\text{new}}.\text{msgs} = \{m \mid (p, m, c) \in conf_{\text{new}}\}$.
 if $|conf_{\text{new}}.\text{msgs}| \leq 1 \vee \text{forensic}(conf_{\text{new}})$: ▷ Disagreement implies Forensic support
 $conf \leftarrow conf_{\text{new}}$
 send (confirm, sid, $\hat{m}, \hat{c}$) to $\hat{p}$
If $\exists m$ s.t. $submitted = \{m\} \wedge |supporters[m] \cap \text{Correct}| \geq n - t$: ▷ Optimistic Convergence (Liveness)
 For each correct process p_i , sends, as a delayed output, (confirm, sid, m, c_i) with $(p_i, m, c_i) \in conf$.
Upon receiving (breakLiveness, sid) from Sim with $f > t$:
 Sim can indefinitely delay any (confirm, sid, *, *) notification.

Integrity, Validity, and Forensic-support checks of $\mathcal{F}_{\text{rat}}$ succeed, the functionality delivers the confirmation to p_i^D , which outputs it to $\mathcal{E}$.

Corruptions. $\mathcal{S}$ maintains synchronized corruption status: whenever $\mathcal{A}^{\text{sim}}$ corrupts p_k^{sim} , $\mathcal{S}$ corrupts the corresponding p_k^D . Messages from a corrupted p_k^D to $\mathcal{S}$ are forwarded to $\mathcal{A}^{\text{sim}}$, and outputs of corrupted p_k^{sim} destined for $\mathcal{E}$ are emitted by p_k^D .

Global resources. $\mathcal{S}$ keeps $\mathcal{J}_{+}$ and $\mathcal{J}_{+}^{\text{sim}}$ synchronized:

- Whenever $\mathcal{A}^{\text{sim}}$ (directly or via a corrupted party) sends a request to $\mathcal{F}_{\text{mcert}}^{\text{sim}}$ or $\mathcal{F}_{\text{lottery}}^{\text{sim}}$, $\mathcal{S}$ sends the same request to $M_{\tilde{\mathcal{F}}_{\text{rat}}}$, which forwards it to $\mathcal{F}_{\text{mcert}}$ or $\mathcal{F}_{\text{lottery}}$ (see mechanism described in Functionality 5).
- When a so-far correct p_k^{sim} issues (participate, sid) and, upon election, (sign, sid, m), this necessarily follows a (submit, sid, m) from $\mathcal{E}$. The corresponding ideal submission triggers the same subroutine calls in $\mathcal{J}_{+}$, maintaining identical state.
- When a so-far correct p_k^{sim} aggregates (agg, sid, $Q, \{ticket_j\}$) and (cb, sid, $Q, m, \{\sigma_j\}$), producing a fully valid c_k^{sim} , $\mathcal{S}$ requests the same aggregation in $\mathcal{F}_{\text{rat}}$ (forwarded to $\mathcal{F}_{\text{mcert}}$ and $\mathcal{F}_{\text{lottery}}$ as described in Functionality 5). Then, it sends (confirm, sid, $p_k^D, m, c_k^{\text{sim}}$) to $\mathcal{F}_{\text{rat}}$, as already mentioned in the confirm coupling paragraph.

Having $\mathcal{J}_{+}$ and $\mathcal{J}_{+}^{\text{sim}}$ synchronized means that any future query (fully_verify) or (verdict) from the environment to $\mathcal{J}_{+}$ returns the same result in both worlds.

Algorithm 6 $\mathcal{P}_{\text{rat}} \leq \mathcal{F}_{\text{rat}}$ in the $\mathcal{J}_+$ -hybrid model: Pseudocode (for process p_i)

```

1: Uses:
2:    $\mathcal{F}_{\text{mcert}}$  ▷ Multi-signature scheme (see Functionalities 1 and 2 in §C.1.1). Public key publication is handled by  $\mathcal{F}_{\text{CA}}$ .
3:    $\mathcal{F}_{\text{lottery}}$  parametrized with probability  $\lambda/n$  ▷ Verifiable committee sampling (see Functionality 3 in §C.1.2)
4:    $\mathcal{F}_{\text{flood}}$  ▷ Flooding functionality
5:    $\mathcal{J}_{\text{rat},+}$  ▷ Judge (see Functionality 4) allowing the definition of forensic support

6: Local Variables:
7:   Dictionary(Process  $\times$  Hash_Value  $\rightarrow$  Signature)  $\text{from\_unverified}_i \leftarrow \perp$ 
8:   Dictionary(Process  $\times$  Hash_Value  $\rightarrow$  Signature)  $\text{from}_i \leftarrow \perp$ 
9:   Dictionary(Process  $\times$  Hash_Value  $\rightarrow$  VRF_Proof)  $\text{vrf\_proofs}_i \leftarrow \perp$ 
10:  Message  $\text{predecided}_i \leftarrow \perp$ 
11:  Signature  $\sigma_i \leftarrow \perp$ 
12:  VRF_Proof  $\pi_i \leftarrow \perp$ 

13: upon (submit, sid, v):
14:    $\text{predecided}_i \leftarrow v$ 
15:   Request (participate, sid, ssid) from  $\mathcal{F}_{\text{lottery}}$ 

16: upon receiving (eligibility, sid,  $\text{coin}_i$ ,  $\pi'_i$ ) from  $\mathcal{F}_{\text{lottery}}$ :
17:   if  $\text{coin}_i = 1$ :
18:      $\pi_i \leftarrow \pi'_i$ 
19:     Request (sign, sid,  $p_i$ , (SUBMIT, hash( $v$ ))) from  $\mathcal{F}_{\text{mcert}}$ 

20: upon receiving (signature, sid,  $p_i$ , (SUBMIT, hash( $v$ )),  $\sigma_i$ ) from  $\mathcal{F}_{\text{mcert}}$ :
21:   send (broadcast, sid, ( $p_i$ , (SUBMIT, hash( $v$ )),  $\sigma_i$ ,  $\pi_i$ )) to  $\mathcal{F}_{\text{flood}}$ 

22: upon receiving (rcv, sid,  $p_j$ , (SUBMIT,  $h$ ),  $\sigma_j$ ,  $\pi_j$ ) from  $\mathcal{F}_{\text{flood}}$ :
23:    $\text{from\_unverified}_i[p_j, h] \leftarrow \sigma_j$ 
24:    $\text{vrf\_proofs}_i[p_j, h] \leftarrow \pi_j$ 
25:   Request (verify, sid,  $\{p_j\}$ , (SUBMIT,  $h$ ),  $\sigma_j$ ) from  $\mathcal{F}_{\text{mcert}}$ 
26:   Request (verify, sid,  $p_j$ ,  $\pi_j$ ) from  $\mathcal{F}_{\text{lottery}}$ 

27: upon receiving (verified, sid,  $\{p_j\}$ , (SUBMIT,  $h$ ), 1) from  $\mathcal{F}_{\text{mcert}}$  and (verified, sid,  $p_j$ , 1) from  $\mathcal{F}_{\text{lottery}}$ :
28:    $\text{from}_i[p_j, h] \leftarrow \text{from\_unverified}_i[p_j, h]$ 

29: upon  $|G| := \{p_j \mid \text{from}_i[p_j, h_i^*] \neq \perp\} \geq W_\delta^{\epsilon, \lambda}$  with  $h_i^* = \text{hash}(\text{predecided}_i) \neq \perp$ :
30:    $\sigma \leftarrow \{\text{from}_i[p_j, h_i^*]\}_{p_j \in G}$  and  $\pi \leftarrow \{\text{vrf\_proofs}_i[p_j, h_i^*]\}_{p_j \in G}$ 
31:   Request (cb, sid,  $G$ ,  $h_i^*$ ,  $\sigma$ ) from  $\mathcal{F}_{\text{mcert}}$ 
32:   Request (aggregate, sid,  $G$ ,  $\pi$ ) from  $\mathcal{F}_{\text{lottery}}$ 

33: upon receiving (multisignature, sid,  $G$ ,  $h_i^*$ ,  $\sigma_G$ ) from  $\mathcal{F}_{\text{mcert}}$  and (aggregation, sid,  $G$ ,  $\pi_G$ ) from  $\mathcal{F}_{\text{lottery}}$ :
34:    $\text{light\_cert}_i \leftarrow (h_i^*, G, \sigma_G)$ 
35:    $\text{full\_cert}_i \leftarrow (\text{light\_cert}_i, \pi_G)$ 
36:   trigger (confirm, sid,  $\text{predecided}_i$ ,  $\text{full\_cert}_i$ )

```

Correctness arguments.

Integrity and Validity. When process p_k^{sim} triggers (confirm, sid, m , c), it comes from correctly verified partial-signatures and proofs of eligibility. Thus, a (fully_verify, sid, c) request would succeed in the simulated real-world, and thus, since $\mathcal{J}_+^{\text{sim}}$ and $\mathcal{J}_+$ are synchronized, the corresponding fully_verify request triggered by $\mathcal{F}_{\text{rat}}$ will pass too.

Forensic support. The functionality requires that if two conflicting messages are confirmed by so-far correct processes with corresponding attached certificates c and c' , the judge returns (verdict, sid, $\text{dis}(D)$) with $|D| \geq d$, to the request (verdict, sid, (c , c')).

By the second Chernoff bound (Theorem 1), and the collision resistance of the hash function, this holds with overwhelming probability in the simulated-real world. Since $\mathcal{J}_+^{\text{sim}}$ and $\mathcal{J}_+$ are synchronized, this holds in the ideal world as well. Hence, the forensic support is preserved and confirmation are always correctly forwarded from the simulated real-world to the ideal world.

Liveness (optimistic case). If $f < t$, all correct parties submit the same value, and all their messages are delivered, then by the first Chernoff bound (Theorem 1), enough eligible supporters exist to confirm that value with overwhelming probability. In the ideal world, the ratifier delivers these confirmations as delayed outputs, mirroring the real execution.

Conclusion. The simulator $\mathcal{S}$ maintains an indistinguishable view for the environment through a probabilistic forward simulation. A fair real-world execution (where each delayed output is eventually triggered) corresponds to a fair ideal-world execution, ensuring that the *hyperliveness* property (optimistic convergence) is preserved. Therefore,

$\mathcal{P}_{\text{rat}}$ realizes $\mathcal{F}_{\text{rat}}$ in the $\mathcal{J}_+$ -hybrid model.

□

C.3. The Propagator Functionality $\mathcal{F}_{\text{prop}}$

As discussed earlier, forensic support does not specify how the judge acquires the necessary pieces of evidence to detect misbehavior. This task is handled by the propagator functionality $\mathcal{F}_{\text{prop}}$, defined in Functionality 7.

Functionality 7 $\mathcal{F}_{\text{prop}}$

Parameters:

t_{acc} : Pessimistic Threshold
 $d = B_{\delta, \hat{\delta}}^{\epsilon, \lambda}$: Number of dishonest processes that should be exposed in case of disagreement

Variables:

Set(Certificate) $Prop$

▷ Set of propagated certificates

Subroutines:

$\mathcal{F}_{\text{mcert}}$: multi-certification functionality
 $\mathcal{F}_{\text{lottery}}$: lottery functionality

Participants:

$p_1, \dots, p_n$: Parties interacting with the functionality.
 Sim : Ideal adversary (simulator).
 Judge_+ : (public and fair) judge

Events:

request (propagate, sid, $cert$): A party propagates a certificate
notification (detection, sid, D, π): A party outputs a proof π , exposing a set D of dishonest processes

Functionality:

Upon receiving (propagate, sid, $cert$) from p :
 if Judge_+ returns *true* on request (fully_verify, sid, $cert$):
 $Prop \leftarrow Prop \cup \{cert\}$
 send (propagated, sid, $p, cert$) to Sim

 If $f \leq t_{\text{acc}} \wedge \exists c, c' \in Prop$, such that Judge_+ returns (verdict, sid, $\text{dis}(D)$) on request (verdict, sid, (c, c')):
 Send (detection, sid, $D_i, \pi_i \in Prop^2$) to each p_i as a delayed output, s.t.
 Judge_+ returns (verdict, sid, $\text{dis}(D_i)$) on request (verdict, sid, π_i), with $|D_i| \geq d$

This functionality ensures:

- **Accountability for conflict-freeness:** If two correct processes collectively propagate a pair of certificates (c, c') verifying $\text{conflict}(c, c') = \text{true}$, then, every correct process p_i eventually gets a proof π_i exposing d dishonest processes.

A naive implementation would require every correct process to broadcast its certificate, resulting in quadratic communication overhead. Instead, we propose $\Pi_{\text{ERFlood}}^{\text{prop}}$ (Algorithm 7), a very minor modification of the flooding protocol Π_{ERFlood} described in [18]. The protocol's pseudocode is simple (though its analysis is far from trivial): upon receiving a verifiably-valid message m , each party flips a coin for each other process that decides, with some probability ρ^* , if m should be relayed to this process. When ρ^* is sufficiently large, Π_{ERFlood} guarantees the formation of a connected Erdős–Rényi graph with a low diameter.

In $\Pi_{\text{ERFlood}}^{\text{prop}}$, which is based on Π_{ERFlood} , a message m containing a valid certificate $(h, G, \sigma) \in \text{Certificate}$ is relayed by process p_i only if it has not already relayed m or another message m' with a certificate $(h', G', \sigma') \in \text{Certificate}$ where $h' = h$. The modification (excluding the accountability layer) is in **red**, line 18. It is easy to see that the analysis of $\Pi_{\text{ERFlood}}^{\text{prop}}$ mirrors the (non-trivial) analysis of Π_{ERFlood} .

Lemma 2. Let $x \in \{1, 2\}$. Let $\text{para} = (n, \kappa, t, t_{\text{acc}}, \gamma, \epsilon, \delta, \hat{\delta}, \rho_x)$ be a x -scalable parametrization. $\Pi_{\text{ERFlood}}^{\text{prop}}(\rho_x)$ (Algorithm 7) realizes $\mathcal{F}_{\text{prop}}$ (see Functionality 7) against a 1-delayed-adaptive adversary, with expected round complexity rc_x , and expected communication complexity mc_x , where each message is of size $\text{ms} = O(\lambda\kappa)$, such that $(\text{mc}_x, \text{rc}_x, d)$ is a x -scalable tuple w.r.t. para .

PROOF. Without loss of generality, we consider the set $\mathcal{A}_{\text{prop}}$ of adversaries for $\mathcal{P}_{\text{prop}}$ that never corrupt p_1 or p_2 , and the set $\mathcal{E}_{\text{prop}}$ of environments $\mathcal{E}_{\text{prop}}$ for machines of the form $\mathcal{P}_{\text{prop}} \mid \mathcal{A}$ with $\mathcal{A} \in \mathcal{A}_{\text{prop}}$. We further restrict $\mathcal{E}_{\text{prop}}$ to environments that only order processes to propagate valid certificates: p_1 propagates c_1 , p_2 propagates c_2 , where $\text{conflict}(c_1, c_2) = \text{true}$. We fix the session identifier sid. Finally, environments in $\mathcal{E}_{\text{prop}}$ must halt and return a decision in $\{0, 1\}$ if it receives a detection notification from a so-far correct process.

We define C_1, C_2 as the sets of valid certificates equivalent (i.e., non-conflicting) with c_1 and c_2 , respectively. Let C_3 be the set of valid certificates that are not in $C_1 \cup C_2$ —that is, certificates conflicting with both c_1 and c_2 . Let c_3 denote the first element of C_3 according to a fixed lexicographic order.

Let $\mathcal{A}_{\text{prop}} \in \mathcal{A}_{\text{prop}}$. We define an adversary $\mathcal{A}_{\text{flood}}$ for Π_{ERFlood} that behaves exactly as $\mathcal{A}_{\text{prop}}$, with one exception: when $\mathcal{A}_{\text{flood}}$ is notified via $\mathcal{F}_{\text{MT}}$ that process p_i has requested to send certificate c_i to process p_j , but $\mathcal{A}_{\text{flood}}$ has already

Algorithm 7 $\Pi_{\text{ERFlood}}^{\text{prop}}(\rho^*) \leq \mathcal{F}_{\text{prop}}$ in the $\mathcal{J}_+$ -hybrid model: Pseudocode (for process p_i)

```

1: Uses:
2:    $\mathcal{F}_{\text{MT}}$  ▷ Message Transfer Functionality
3:    $\mathcal{J}_+ = \mathcal{J}_{\text{rat},+} \mid \mathcal{F}_{\text{mcert}} \mid \mathcal{F}_{\text{lottery}}$  ▷ Judge (see Functionality 4)

4: Parameters:
5:   Integer  $\delta, \hat{\delta}, \epsilon, \lambda \in \Omega(1)$ 
6:   Integer  $d = B_{\delta, \hat{\delta}}^{\epsilon, \lambda}$ : Number of dishonest processes that should be exposed in case of disagreement

7: Local Variables:
8:   Set(Certificate)  $\text{certificates}_i \leftarrow \emptyset$ 

9: upon relay(sid, cert):
10:   $\mathcal{N}_{\text{new}} \xleftarrow{\$} \mathcal{N}(\rho^*)$ , where  $\mathcal{N}(\rho^*)$  is the random variable that returns a subset of processes in  $\Psi$ , where the inclusion of a process follows the
    Bernoulli distribution with expected value  $\rho^*$ .
11:   $\text{certificates}_i \leftarrow \text{certificates}_i \cup \{\text{cert}\}$ 
12:  Request (multicast, sid,  $\mathcal{N}_{\text{new}}, \langle \text{RELAY}, \text{cert} \rangle$ ) from  $\mathcal{F}_{\text{MT}}$  ▷  $p_i$  sends  $n\rho^*$  messages on expectation

13: upon (propagate, sid, cert):
14:   trigger relay(sid, cert)

15: upon receiving (deliver, sid,  $p'$ ,  $\langle \text{RELAY}, \text{cert}' \rangle$ ) from  $\mathcal{F}_{\text{MT}}$ :
16:   Request (fully_verify, sid,  $\text{cert}'$ ) from  $\mathcal{J}_{\text{rat},+}$  ▷ invalid certificates are ignored
17:   when receiving (full_validity, sid,  $\text{cert}', 1$ ) from  $\mathcal{J}_{\text{rat},+}$ :
18:     if  $\text{certificates}_i = \emptyset$ : ▷ There is no need to relay several non-conflicting certificates
19:       trigger relay(sid,  $\text{cert}'$ )
    //Accountability layer (code below has been added to  $\Pi_{\text{ERFlood}}$ )
20:     if  $\exists \text{cert} \in \text{certificates}_i$ , such that  $\mathcal{J}_{\text{rat},+}$  returns Dis( $D$ ) with  $|D| \geq d$ , upon reception of candidate proof ( $\text{cert}, \text{cert}'$ )
21:       Request (broadcast, sid,  $\langle \text{PROOF}, (\text{cert}, \text{cert}') \rangle$ ) from  $\mathcal{F}_{\text{MT}}$ 

22: upon receiving (deliver, sid,  $p_j$ ,  $\langle \text{PROOF}, \pi \rangle$ ) from  $\mathcal{F}_{\text{MT}}$ :
23:   send (verdict, sid,  $\pi$ ) to  $\mathcal{J}_{\text{rat},+}$ :
24:   upon receiving (verdict, sid, Dis( $D$ )) from  $\mathcal{J}_{\text{rat},+}$  with  $|D| \geq d$ :
25:     trigger (detection, sid,  $D, \pi$ )

```

ordered $\mathcal{F}_{\text{MT}}$ to deliver to p_j a certificate $c' \equiv c_i$, then $\mathcal{A}_{\text{flood}}$ stores (p_j, c_i) in a set Delayed. For any pair $(p, c) \in \text{Delayed}$, $\mathcal{A}_{\text{flood}}$ delays p 's reception of c as long as any messages remain in flight. We note $\mathfrak{E}_{\text{flood}}$ the set of environments for $\Pi_{\text{ERFlood}} \mid \mathcal{A}_{\text{flood}}$

Since [18, Theorem 3] states that Π_{ERFlood} UC-emulates $\mathcal{F}_{\text{flood}}$, there exists a simulator $\mathcal{S}_{\text{flood}}$ such that,

$$\forall \mathcal{E} \in \mathfrak{E}_{\text{flood}}, \mathcal{E} \mid \Pi_{\text{ERFlood}} \mid \mathcal{A}_{\text{flood}} \simeq \mathcal{E} \mid \mathcal{F}_{\text{flood}} \mid \mathcal{S}_{\text{flood}}.$$

We now define a simulator $\mathcal{S}_{\text{prop}}$ for $\mathcal{F}_{\text{prop}}$ that behaves exactly as $\mathcal{S}_{\text{flood}}$, with the following exception (besides the interpretation of propagation requests as flooding requests): when $\mathcal{S}_{\text{flood}}$ triggers the delivery of certificate c to process p , $\mathcal{S}_{\text{prop}}$ remains silent, unless a delivery of c' to p has already occurred with $\text{conflict}(c, c') = \text{true}$. In this case, $\mathcal{S}_{\text{prop}}$ triggers the notification

$$(\text{detection}, \text{sid}, D, (c, c'))$$

where $\mathcal{J}_{\text{rat},+}$ returns $(\text{verdict}, \text{sid}, \text{dis}(D))$ upon the reception of $(\text{verdict}, \text{sid}, (c, c'))$.

Let $\mathcal{E}_{\text{prop}} \in \mathfrak{E}_{\text{prop}}$. We define an environment $\mathcal{E}_{\text{flood}} \in \mathfrak{E}_{\text{flood}}$ that behaves exactly as $\mathcal{E}_{\text{prop}}$, except: propagation requests are replaced with corresponding flooding requests, and when $\mathcal{E}_{\text{flood}}$ receives a delivery notification for process p , it stores the certificate in $\text{Cert}[p]$. If $\text{Cert}[p]$ does not contain a pair of conflicting certificates, the environment behaves as if it received nothing.

Thus, we have:

$$\mathcal{E}_{\text{flood}} \mid \Pi_{\text{ERFlood}} \mid \mathcal{A}_{\text{flood}} \simeq \mathcal{E}_{\text{flood}} \mid \mathcal{F}_{\text{flood}} \mid \mathcal{S}_{\text{flood}}.$$

Moreover, by construction:

$$\mathcal{E}_{\text{flood}} \mid \mathcal{F}_{\text{flood}} \mid \mathcal{S}_{\text{flood}} \simeq \mathcal{E}_{\text{prop}} \mid \mathcal{F}_{\text{prop}} \mid \mathcal{S}_{\text{prop}}.$$

and, as long as (*) $\mathcal{A}_{\text{flood}}$ delays the reception of messages in Delayed until $\mathcal{E}_{\text{flood}}$'s decision

$$\mathcal{E}_{\text{flood}} \mid \Pi_{\text{ERFlood}} \mid \mathcal{A}_{\text{flood}} \simeq \mathcal{E}_{\text{prop}} \mid \mathcal{P}_{\text{prop}} \mid \mathcal{A}_{\text{prop}}$$

It follows that, if (*) holds with overwhelming probability:

$$\mathcal{E}_{\text{prop}} \mid \mathcal{P}_{\text{prop}} \mid \mathcal{A}_{\text{prop}} \simeq \mathcal{E}_{\text{prop}} \mid \mathcal{F}_{\text{prop}} \mid \mathcal{S}_{\text{prop}},$$

We now prove that (*) holds, except with negligible probability. We define $\mathcal{E}^*$ and $\mathcal{A}^*$ to behave identically to $\mathcal{E}_{flood}$ and $\mathcal{A}_{flood}$, except that every certificate c appearing on their external tape is replaced by c_i if $c \equiv c_i$ for some $i \in \{1, 2, 3\}$. Clearly, we have:

$$\mathcal{E}^* \mid \Pi_{\text{ERFlood}} \mid \mathcal{A}^* \simeq \mathcal{E}_{flood} \mid \Pi_{\text{ERFlood}} \mid \mathcal{A}_{flood}.$$

Now, by the specification of the flooding functionality, except with negligible probability, a correct process is expected to deliver a pair of conflicting certificates from $\{c_1, c_2, c_3\}$ *before* the delivery of all in-flight messages sent between so-far correct processes. Furthermore, the delayed messages in Delayed no longer affect the execution: a process never relays the same certificate twice. Thus, a correct process is expected to deliver a pair of conflicting certificates from $\{c_1, c_2, c_3\}$ before $\mathcal{A}_{flood}$ triggers delivery of the pending messages in Delayed. We can therefore conclude that, except with negligible probability, $\mathcal{A}_{flood}$ delays the reception of messages in Delayed until after $\mathcal{E}_{flood}$ has reached a decision. This concludes the proof. $\square$

C.4. The Accountable Confirmer Functionality $\mathcal{F}_{\text{conf}}^{\text{acc}} = \mathcal{F}_{\text{rat}} \triangleright \mathcal{F}_{\text{prop}}$

The accountable confirmer, introduced as the core of the $\mathcal{ABC}$ compiler in [2], [11], is defined in Functionality 8.

Functionality 8 $\mathcal{F}_{\text{conf}}^{\text{acc}}$

Parameters:

- t : Optimistic Threshold
- t_{acc} : Pessimistic Threshold
- $d = B_{\delta, \delta}^{\epsilon, \lambda}$: Number of dishonest processes that should be exposed in case of disagreement

Variables:

- variables are implicitly parametrized with session ID in the obvious manner
- Set(Message) *Submitted* $\leftarrow \emptyset$ $\triangleright$ Set of submitted messages
- Dict(Message $\rightarrow$ Set(Process)) *Submissions* $\leftarrow \text{Dict.init_empty}()$ $\triangleright$ Maps each message m to the set of processes submitting m .
- Set(Message) *Conf* $\leftarrow \emptyset$ $\triangleright$ Set of confirmed messages
- Boolean *exposed* $\leftarrow \text{false}$ $\triangleright$ Set of confirmed messages

Subroutine Functionality:

- $\mathcal{F}_{\text{mcert}}$ $\triangleright$ See Functionality 1-2 and subroutine in Functionality 5

Participants:

- $p_1, \dots, p_n$: Parties interacting with the functionality.
- Sim: Ideal adversary (simulator).
- Judge: (public and fair) judge

Events:

- request* (submit, sid, m): A party submits a message m .
- notification* (confirm, sid, m): A party confirms the message m .
- notification* (detection, sid, D, π): a party gets the proof π exposing D

Functionality:

- Upon receiving (submit, sid, m) from process p_i :
 - $\text{Submitted}[\text{sid}] \leftarrow \text{Submitted}[\text{sid}] \cup \{m\}$
 - $\text{Submissions}[\text{sid}][m] \leftarrow \text{Submissions}[\text{sid}][m] \cup \{p_i\}$
 - send (support, sid, p_i, m) to Sim
 - trigger** subroutine_sign(sid, p_i , hash(m))
 - Upon receiving (confirm, sid, p_i, m_i) from Sim with $p_i \in \text{Submissions}[\text{sid}][m_i]$: $\triangleright$ Confirmation is triggered by the adversary
 - if** $|\text{Conf}[\text{sid}] \cup \{m_i\}| \leq 1 \vee \text{exposed}[\text{sid}] = \text{true}$: $\triangleright$ Disagreement must imply detection (see BreakAgreement)
 - $\text{Conf}[\text{sid}] \leftarrow \text{Conf}[\text{sid}] \cup \{m_i\}$
 - send (confirm, sid, m_i) to process p_i .
 - Upon receiving (breakLiveness, sid) from Sim with $f > t$:
 - Sim can indefinitely delay any (confirm, sid, *, *) notification.
 - if** $f > t_{\text{acc}}$: Sim can indefinitely delay any (detection, sid, *, *) notification.
 - Upon receiving (breakAgreement, sid, π) from Sim with $f > t$:
 - if** $\forall i \in [n]$, for each request (verdict, sid, $\pi[i]$), Judge returns (verdict, sid, dis(D_i)) with $|D_i| \geq d$:
 - $\text{exposed}[\text{sid}] \leftarrow \text{true}$ $\triangleright$ Guard for disagreement
 - send (detection, sid, $D_i, \pi[i]$) to each p_i as a delayed output $\triangleright$ Accountability
 - If $\exists m$ s.t. $\text{Submitted}[\text{sid}] = \{m\} \wedge |\text{Submissions}[\text{sid}][m] \cap \text{Correct}| \geq n - t$: $\triangleright$ Optimistic convergence
 - Send (confirm, sid, m), with $\text{Conf} = \{m\}$, to all processes as a delayed output $\triangleright$ Agreement and Liveness
-

The functionality $\mathcal{F}_{\text{conf}}^{\text{acc}}$ ensures:

- **Integrity:** A process cannot confirm a value it did not submit.
- **Optimistic Convergence:** If $f \leq t$ and all correct processes submit the same value v , they all eventually confirms v .
- **Accountability for agreement:** If two correct processes output different values, every correct process eventually obtains a proof π_i exposing d dishonest processes.

Accountability can be obtained from the conjunction of three distinct properties:

- **Strong-forensic support for agreement:** If two correct processes p_i and p_j confirm different values, they store 2 valid conflicting certificates c_i and c_j , where the underlying $((\text{valid}(\cdot), \text{conflict}(\cdot, \cdot)))$ predicates are propagation-friendly.
- **Accountability for conflict-freeness:** If two correct processes collectively propagate a pair of conflicting certificates, then every correct process p_i eventually obtains a proof π_i exposing d dishonest processes.
- **Involvement:** If a correct process confirms a value, it propagates the corresponding valid certificate.

Essentially, $\mathcal{F}_{\text{rat}}$ ensures (1) integrity, (2) optimistic convergence, and (3) forensic support for disagreement, while $\mathcal{F}_{\text{prop}}$ guarantees (4) accountability for conflict-freeness. The composition simply ensures (5) involvement.

This leads to a direct realization of $\mathcal{F}_{\text{conf}}^{\text{acc}}$ in the $(\mathcal{F}_{\text{rat}}, \mathcal{F}_{\text{prop}})$ -hybrid model, presented in Algorithm 8.

Algorithm 8 $\mathcal{P}_{\text{conf}}^{\text{acc}} \leq \mathcal{F}_{\text{conf}}^{\text{acc}}$ in the $(\mathcal{F}_{\text{rat}}, \mathcal{F}_{\text{prop}})$ -hybrid model

```

1: Uses:
2:    $\mathcal{F}_{\text{rat}}$  ▷ Ratifier functionality for forensic support
3:    $\mathcal{F}_{\text{prop}}$  ▷ Propagator functionality for accountability
4:    $\mathcal{J}_+$  is shared by  $\mathcal{F}_{\text{rat}}$  and  $\mathcal{F}_{\text{prop}}$  ▷ In AUC, composition with joint state follows directly from the base composition theorem

5: upon (submit, sid,  $v$ ):
6:   Request (submit, sid,  $v$ ) from  $\mathcal{F}_{\text{rat}}$ 

7: upon receiving (confirm, sid,  $v$ ,  $\text{cert}$ ) from  $\mathcal{F}_{\text{rat}}$ : ▷ Forensic support is guaranteed
8:   trigger (confirm, sid,  $v$ )
9:   Request (propagate, sid,  $\text{cert}$ ) from  $\mathcal{F}_{\text{prop}}$ 

10: upon receiving (detection, sid,  $D$ ,  $\pi$ ) from  $\mathcal{F}_{\text{prop}}$ :
11:   trigger (detection, sid,  $D$ ,  $\pi$ ) ▷ Accountability is achieved

```

Lemma 3. $\mathcal{P}_{\text{conf}}^{\text{acc}}$ (Algorithm 8) realizes $\mathcal{F}_{\text{conf}}^{\text{acc}}$ in the $(\mathcal{F}_{\text{rat}}, \mathcal{F}_{\text{prop}})$ -hybrid model with no additional complexity overhead beyond that of the realizations of $\mathcal{F}_{\text{rat}}$ and $\mathcal{F}_{\text{prop}}$.

PROOF. We consider the ideal system

$$\mathcal{F}_{\text{conf}}^{\text{acc}} = p_1^D \mid \cdots \mid p_n^D \mid M_{\mathcal{F}_{\text{conf}}^{\text{acc}}} \mid \mathcal{J},$$

where $p_1^D, \dots, p_n^D$ are dummy processes, $\mathcal{J} = \mathcal{J}_{\text{rat}} \mid \mathcal{J}_{\text{incert}}$, and $M_{\mathcal{F}_{\text{conf}}^{\text{acc}}}$ provides the interface tapes for dummy processes and the simulator.

The corresponding real system in the $(\mathcal{F}_{\text{rat}} \mid \mathcal{F}_{\text{prop}})$ -hybrid model (where $\mathcal{J}_+$ is shared by $\mathcal{F}_{\text{rat}}$ and $\mathcal{F}_{\text{prop}}$) is

$$\mathcal{P}_{\text{conf}}^{\text{acc}} = p_1 \mid \cdots \mid p_n \mid \mathcal{F}_{\text{rat}} \mid \mathcal{F}_{\text{prop}}.$$

Let $\mathcal{A}$ be an adversary against $\mathcal{P}_{\text{conf}}^{\text{acc}}$. We construct a simulator $\mathcal{S}$ for the ideal world.

Simulator. The simulator $\mathcal{S}$ internally executes a faithful simulation of the real execution

$$\mathcal{A}^{\text{sim}} \mid (\mathcal{P}_{\text{conf}}^{\text{acc}})^{\text{sim}} \quad \text{with} \quad (\mathcal{P}_{\text{conf}}^{\text{acc}})^{\text{sim}} = p_1^{\text{sim}} \mid \cdots \mid p_n^{\text{sim}} \mid \mathcal{F}_{\text{rat}}^{\text{sim}} \mid \mathcal{F}_{\text{prop}}^{\text{sim}},$$

choosing its own randomness for $\mathcal{F}_{\text{lottery}}^{\text{sim}}$ shared by $\mathcal{F}_{\text{rat}}^{\text{sim}}$ and $\mathcal{F}_{\text{prop}}^{\text{sim}}$. No secret information of honest parties is required.

I/O forwarding. Messages from the environment $\mathcal{E}$ to $\mathcal{S}$ are forwarded to $\mathcal{A}^{\text{sim}}$. Likewise, outputs of $\mathcal{A}^{\text{sim}}$ addressed to $\mathcal{E}$ are forwarded back by $\mathcal{S}$.

Submit/confirm coupling. Whenever $\mathcal{S}$ receives from $\mathcal{F}_{\text{conf}}^{\text{acc}}$ a notification (support, sid, p_i^D , m_i), it instructs p_i^{sim} to submit (submit, sid, m_i) to $\mathcal{F}_{\text{rat}}^{\text{sim}}$. Conversely, whenever p_i^{sim} outputs (confirm, sid, m_i), $\mathcal{S}$ sends (confirm, sid, p_i^D , m_i) to $\mathcal{F}_{\text{conf}}^{\text{acc}}$. If the *Integrity* and *Accountability* checks of $\mathcal{F}_{\text{conf}}^{\text{acc}}$ succeed, the functionality delivers the confirmation to p_i^D , which outputs it to $\mathcal{E}$.

Corruptions. $\mathcal{S}$ maintains synchronized corruption status: whenever $\mathcal{A}^{\text{sim}}$ corrupts p_k^{sim} , $\mathcal{S}$ corrupts the corresponding p_k^D . Messages from a corrupted p_k^D to $\mathcal{S}$ are forwarded to $\mathcal{A}^{\text{sim}}$, and outputs of corrupted p_k^{sim} destined for $\mathcal{E}$ are emitted by p_k^D .

Global resources. $\mathcal{S}$ keeps $\mathcal{J}$ and $\mathcal{J}^{\text{sim}}$ synchronized:

Algorithm 9 $ABC(\mathcal{F}_{\text{ea}}) \leq \mathcal{F}_{\text{ea}}^{\text{acc}}$ on top of $\mathcal{F}_{\text{conf}}^{\text{acc}}$

```

1: Uses:
2:    $\mathcal{F}_{\text{ea}}$ 
3:    $\mathcal{F}_{\text{conf}}^{\text{acc}}$ 
4: Local Variables:
5:   Value  $\text{predecided}_i \leftarrow \perp$ 
6: upon (propose, sid, ssid,  $v$ ):
7:   Request (propose, sid, ssid,  $v$ ) from  $\mathcal{F}_{\text{ea}}$ 
8: upon receiving (decide, sid, ssid,  $w$ ) from  $\mathcal{F}_{\text{ea}}$ :
9:    $\text{predecided}_i \leftarrow w$ 
10:  Request (submit, sid, ssid,  $w$ ) from  $\mathcal{F}_{\text{conf}}^{\text{acc}}$ 
11: upon receiving (confirm, sid, ssid,  $\text{predecided}_i$ ) from  $\mathcal{F}_{\text{conf}}^{\text{acc}}$ :
12:  trigger (decide, sid, ssid,  $\text{predecided}_i$ ) from  $\mathcal{F}_{\text{conf}}^{\text{acc}}$ 
13: upon receiving (detection, sid, ssid,  $D$ ,  $\pi$ ) from  $\mathcal{F}_{\text{conf}}^{\text{acc}}$ :
14:  trigger (detection, sid, ssid,  $D$ ,  $\pi$ )

```

▷ An easily accountable functionality in the sense of Definition 3
 ▷ Accountable confirmer functionality (see Functionality 8)

▷ Forensic support is guaranteed

▷ Agreement and Liveness are preserved

▷ Accountability for agreement is achieved

- Whenever $\mathcal{A}^{\text{sim}}$ (directly or via a corrupted process) sends a request to $\mathcal{F}_{\text{mcert}}^{\text{sim}}$ through the forwarding mechanism described in Functionality 5, $\mathcal{S}$ sends the same request to $M_{\mathcal{F}_{\text{conf}}^{\text{acc}}}$, which forwards it to $\mathcal{F}_{\text{mcert}}$ through the same mechanism.
- The only case where $\mathcal{F}_{\text{mcert}}$ records a new signature without this mechanism is upon receiving (submit, sid, m) from a so-far correct process p_k^D . This triggers the subroutine `subroutine_sign`. In this case, $\mathcal{F}_{\text{conf}}^{\text{acc}}$ notifies $\mathcal{S}$ with (support, sid, m). Since $\mathcal{S}$ instructs p_k^{sim} to send (submit, sid, m) to $\mathcal{F}_{\text{rat}}^{\text{sim}}$, the same subroutine_sign is triggered in $\mathcal{J}^{\text{sim}}$.

Having $\mathcal{J}$ and $\mathcal{J}^{\text{sim}}$ synchronized ensures that any future query (verdict) from the environment to $\mathcal{J}$ returns the same result in both worlds.

Detection. $\mathcal{S}$ also synchronizes the status of detections:

- Whenever, for the first time, two so-far correct simulated processes p_i^{sim} and p_j^{sim} receive (confirm, sid, m_i, c_i) and (confirm, sid, $m_j \neq m_i, c_j$), respectively, from $\mathcal{F}_{\text{rat}}^{\text{sim}}$, $\mathcal{S}$ sends (breakAgreement, sid, π^*) to $\mathcal{F}_{\text{conf}}^{\text{acc}}$, where $\pi^*[k] = (c_i, c_j)$ for all $k \in [n]$. The check passes due to the forensic-support property of $\mathcal{F}_{\text{rat}}^{\text{sim}}$. Consequently, *exposed* is set to *true*, allowing the transmission of any confirmation verifying integrity.
- Whenever p_i^{sim} triggers (confirm, sid, m_i), $\mathcal{S}$ sends (confirm, sid, p_i^D, m_i) to $\mathcal{F}_{\text{conf}}^{\text{acc}}$. By the integrity of $\mathcal{F}_{\text{rat}}^{\text{sim}}$, and since *exposed* = *true* once two distinct messages are confirmed, $\mathcal{F}_{\text{conf}}^{\text{acc}}$ forwards the confirmation to p_i^D .
- Whenever p_i^{sim} triggers (detection, sid, D_i, π_i), $\mathcal{S}$ sends (breakAgreement, sid, π) to $\mathcal{F}_{\text{conf}}^{\text{acc}}$, with $\pi[i] = \pi_i$ and $\pi[k] = \pi^*[k]$ for each $k \in [n] \setminus \{i\}$, then triggers the delayed output (detection, sid, D_i, π_i). Both the breakAgreement and detection requests pass the checks of $\mathcal{F}_{\text{conf}}^{\text{acc}}$, by the accountability of $\mathcal{F}_{\text{prop}}^{\text{sim}}$.

Liveness (optimistic case). If $f < t$, all correct parties submit the same value, and all delayed outputs of the simulated real world are delivered, then the optimistic convergence of $\mathcal{F}_{\text{rat}}^{\text{sim}}$ ensures that all simulated correct processes confirm that value. This behavior is mirrored in the ideal world.

Liveness (degraded case). If $f < t_{\text{acc}}$, two correct parties confirm conflicting values, and all delayed outputs of the simulated real world are delivered, then the forensic-support of $\mathcal{F}_{\text{rat}}^{\text{sim}}$ ensures that these parties obtained conflicting valid certificates, which are passed to $\mathcal{F}_{\text{prop}}^{\text{sim}}$. The accountability of $\mathcal{F}_{\text{prop}}^{\text{sim}}$ guarantees that every simulated correct process p_i^{sim} outputs (detection, sid, D_i, π_i) with $|D_i| \geq d$. This behavior is mirrored in the ideal world.

Conclusion. The simulator $\mathcal{S}$ maintains an indistinguishable view for the environment through a probabilistic forward simulation. A fair real-world execution (where each delayed output is eventually triggered) corresponds to a fair ideal-world execution, ensuring that the *hyperliveness* properties—optimistic convergence and accountability—are preserved.

Therefore,

$$\mathcal{P}_{\text{conf}}^{\text{acc}} \text{ realizes } \mathcal{F}_{\text{conf}}^{\text{acc}} \text{ in the } \mathcal{J}_+-\text{hybrid model.}$$

□

C.5. The ABC Transformation: $\mathcal{F}_{\text{ea}}^{\text{acc}} = ABC(\mathcal{F}_{\text{ea}}) = \mathcal{F}_{\text{ea}} \triangleright \mathcal{F}_{\text{conf}}^{\text{acc}}$

We are now ready to prove the correctness of the ABC compiler (see Algorithm 9) in the $\mathcal{F}_{\text{conf}}^{\text{acc}}$ -hybrid model. The transformation sequentially composes any easily accountable functionality $\mathcal{F}_{\text{ea}}$ with the accountable confirmer $\mathcal{F}_{\text{conf}}^{\text{acc}}$ to obtain its accountable counterpart $\mathcal{F}_{\text{ea}}^{\text{acc}}$.

Lemma 4. Let $\mathcal{F}_{\text{ea}}$ be an easily accountable functionality with t' -resiliency, and let $\mathcal{F}_{\text{conf}}^{\text{acc}}$ be the accountable confirmer functionality parametrized with (t, t_{acc}, d) . Define $\mathcal{P}_{\text{ea}}^{\text{acc}} = \mathcal{ABC}(\mathcal{F}_{\text{ea}}, \mathcal{F}_{\text{conf}}^{\text{acc}})$ (see Algorithm 9). Then, $\mathcal{P}_{\text{ea}}^{\text{acc}}$ realizes $\mathcal{F}_{\text{ea}}$ with resiliency $\min(t, t')$. Moreover, if $\min(t, t') < f \leq t_{\text{acc}}$ and a disagreement occurs, every correct process eventually obtains an externally verifiable proof that exposes at least d dishonest processes.

Furthermore, the message, communication, and round complexity of $\mathcal{P}_{\text{ea}}^{\text{acc}}$ incur no additional overhead beyond the sum of the corresponding complexity metrics of the individual realizations of $\mathcal{F}_{\text{ea}}$ and $\mathcal{F}_{\text{conf}}^{\text{acc}}$.

PROOF.

Foreword. We consider the ideal protocol

$$\mathcal{F}_{\text{ea}}^{\text{acc}} = p_1^D \mid \cdots \mid p_n^D \mid M_{\tilde{\mathcal{F}}_{\text{ea}}^{\text{acc}}}$$

where $p_1^D, \dots, p_n^D$ represent the dummy processes, and its realization in the $(\mathcal{F}_{\text{ea}}, \mathcal{F}_{\text{conf}}^{\text{acc}})$ -hybrid model

$$\mathcal{P}_{\text{ea}}^{\text{acc}} = p_1 \mid \cdots \mid p_n \mid \mathcal{ABC}(\mathcal{F}_{\text{ea}}, \mathcal{F}_{\text{conf}}^{\text{acc}}) = (\check{p}_1 \mid \check{p}_1) \mid \cdots \mid (\check{p}_n \mid \check{p}_n) \mid \mathcal{F}_{\text{ea}} \mid \mathcal{F}_{\text{conf}}^{\text{acc}},$$

where $\check{p}_i$ corresponds to the same machine as p_i^D and $\check{p}_i$ corresponds to machine that (1) upon receiving a notification (decide, sid, y) from $\mathcal{F}_{\text{ea}}$, $\check{p}_i$ sends a request (submit, sid, y) to $\mathcal{F}_{\text{conf}}^{\text{acc}}$, and triggers the decision of y , upon receiving a notification (confirm, sid, y) from $\mathcal{F}_{\text{conf}}^{\text{acc}}$.

Let $\mathcal{A}$ be the adversary of $\mathcal{P}_{\text{ea}}^{\text{acc}}$. The simulator $\mathcal{S}$ internally simulates the adversary, denoted by $\mathcal{A}^{\text{sim}}$, and the accountable confirmer $\mathcal{F}_{\text{conf}}^{\text{acc}}$, denoted by $(\mathcal{F}_{\text{conf}}^{\text{acc}})^{\text{sim}}$. Additionally, for each $k \in [1 : n]$, $\mathcal{S}$ simulates $\check{p}_k$, denoted $\check{p}_k^{\text{sim}}$ and, upon the corruption of process p_k , it simulates a copy of the corresponding process of the compiled protocol, denoted p_k^{sim} . Importantly, $\mathcal{S}$ does not simulate $\mathcal{F}_{\text{ea}}$, which might manage private information, such as the prover's witness in the one-to-many zkPoK functionality [21].

Messages received by $\mathcal{S}$ from the environment $\mathcal{E}$ are forwarded to $\mathcal{A}^{\text{sim}}$. Similarly, any messages sent by $\mathcal{A}^{\text{sim}}$ that are destined for the environment are forwarded to $\mathcal{E}$. The simulator $\mathcal{S}$ keeps the corruption status synchronized; that is, whenever $\mathcal{A}$ corrupts a process p_k^{sim} , the simulator also corrupts the corresponding process p_k , learns its state—including its cryptographic material.

Incoming messages from a corrupted process p_k to $\mathcal{S}$ are forwarded to $\mathcal{A}^{\text{sim}}$ on behalf of the corresponding corrupted process p_k^{sim} . Conversely, whenever a corrupted process p_k^{sim} attempts to output a message to the environment, $\mathcal{S}$ instructs the corresponding corrupted process p_k to output the same message to $\mathcal{E}$.

The $\mathcal{ABC}$ Emulation. Let us describe the simulator strategy, beyond the foreword. We describe 6 situations A, B, C, D, E, and F, depicted in Figure 2.

- **A:** Whenever a malicious process p_k^{sim} intends to send a request to $\mathcal{F}_{\text{ea}}$, the simulator $\mathcal{S}$ instructs p_k to send the same request to $\mathcal{F}_{\text{ea}}^{\text{acc}}$.
- **B:** When a request is sent by a correct process, the simulator $\mathcal{S}$ is immediately notified via the corresponding leak tape. Upon notification, $\mathcal{S}$ writes the notification on the leak tape of $\mathcal{A}^{\text{sim}}$, related to $\mathcal{F}_{\text{ea}}$. Let us note that the leak does not necessarily include the request's payload. For example, if $\mathcal{F}_{\text{ea}}$ represents the one-to-many zkPoK functionality [21], the prover's witness is not leaked.
- **C:** Whenever $\mathcal{A}^{\text{sim}}$ orders to forward a (delayed) notification from $\mathcal{F}_{\text{ea}}$, the simulator orders the same to $\mathcal{F}_{\text{ea}}^{\text{acc}}$, except for the special decide notification (see D).
- **D:** Whenever $\mathcal{A}^{\text{sim}}$ orders $\mathcal{F}_{\text{ea}}$ to forward the delayed output (decide, sid, y) to process p_j , the simulator writes the notification on the corresponding tape of $\check{p}_j^{\text{sim}}$. Then, situation E occurs immediately after.
- **E:** Upon receiving (decide, sid, y), $\check{p}_j^{\text{sim}}$ reacts by sending (submit, sid, y) to $\mathcal{F}_{\text{conf}}^{\text{acc}}$.
- **F:** Whenever $\mathcal{A}^{\text{sim}}$ orders $(\mathcal{F}_{\text{conf}}^{\text{acc}})^{\text{sim}}$ to forward the delayed output (confirm, sid, y) or the delayed output (detection, sid, D, π) to process p_j^{sim} , $(\mathcal{F}_{\text{conf}}^{\text{acc}})^{\text{sim}}$ does so, while the simulator orders $\mathcal{F}_{\text{ea}}^{\text{acc}}$ to do the same, which it does as well.

Nominal Case. In this case, the adversary does not corrupt more than t processes. Moreover, the functionality $\mathcal{F}_{\text{ea}}$ is supposed to ensure Agreement & Termination. This means that the succession of situations D and E will occur for a subset of processes $\check{p}_*^{\text{sim}}$. Integrity guarantees the preservation of Agreement property. Assume all processes $\check{p}_*^{\text{sim}}$ face the succession of situations D and E. By agreement, they all submit the same value, say y . By optimistic convergence, situation F must occur, meaning they all confirm y . At the same time, all correct processes p_* do the same. This means that if Termination corresponds to Totality, Termination is preserved. Now, assume the opposite. This means that Totality is not ensured. Thus, Termination corresponds Partial Decidability. Hence, it is allowed for any subset of processes not to decide. The remaining structure of the simulation ensures that the ideal and the real world are indistinguishable.

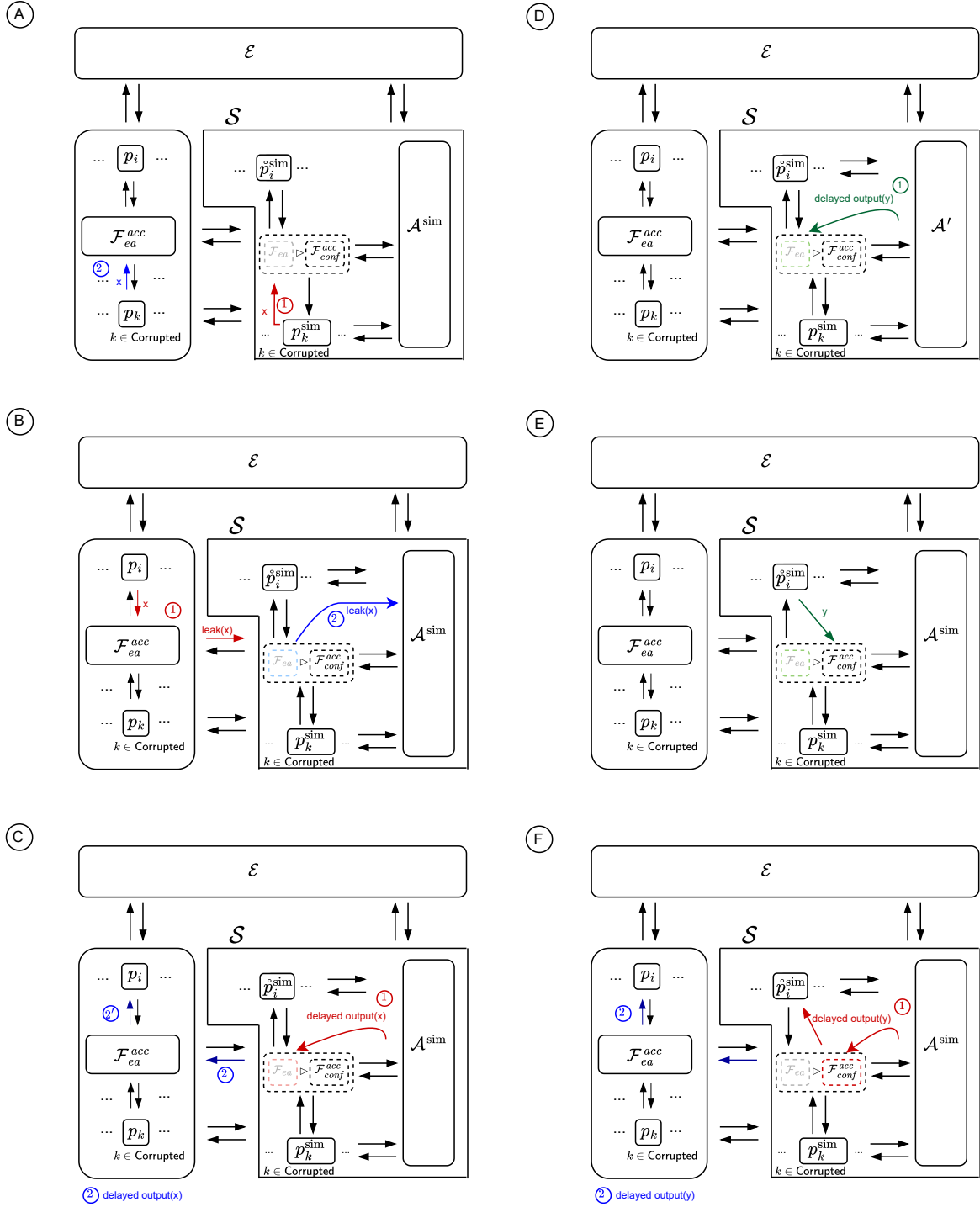


Figure 2: UC emulation of the $\mathcal{ABC}$ transformation

Degraded Case. In this case, the adversary can corrupt up to t_{acc} processes. The adversary can tackle both the optimistic convergence and the agreement property of $\mathcal{F}_{conf}^{acc}$. Violating liveness is not an issue in the degraded mode. If agreement is violated, the accountability property implies that the adversary $\mathcal{A}^{sim}$ will have to eventually trigger the delayed output for detection. This will be performed by the simulator as well, ensuring accountability. $\square$

Consequently, Theorem 5 holds.

Theorem 5. Let J_{rat} be the judge defined in Algorithm 3, and $\mathcal{F}_{ea}^{acc}$ be the $(t_{acc}, B_{\delta, \hat{\delta}}^{\epsilon, \lambda}, J_{rat})$ -accountable counterpart of an easily accountable agreement functionality $\mathcal{F}_{ea}$ (as defined in Definition 3). If $\mathcal{P}$ UC-realizes $\mathcal{F}_{ea}$, then $ABC^{++}(\mathcal{P})$ UC-realizes $\mathcal{F}_{ea}^{acc}$ with complexity overhead described in Theorem 4.

PROOF. By conjunction of Lemmas 1 to 4 and composability of UC-realization. $\square$

Appendix D.

The Certification Authority $\mathcal{F}_{CA}$

We assume the existence of a certification authority functionality, denoted $\mathcal{F}_{CA}$ (see Functionality 9), responsible for registering party identities along with their corresponding public keys [92]. A distributed realization of $\mathcal{F}_{CA}$ requires at least $2/3$ correct processes [54], and adopting an accountable variant would introduce a circular dependency. However, in the unauthenticated synchronous setting, $\mathcal{F}_{CA}$ can be emulated even in the presence of an unbounded number of Byzantine processes, provided that digital signatures are available and the adversary's hash rate is computationally bounded [121], [122]. This allows for a practical two-phase approach:

- Setup Phase: Establish an ideal public key infrastructure (PKI) under a conservative bound Δ on message delay.
- Main Phase: Execute asynchronous protocols atop the established PKI, avoiding the inherent trade-off between safety and efficiency imposed by Δ .

Functionality 9 $\mathcal{F}_{CA}$: Certification Authority

Participants:

$p_1, \dots, p_n$: Parties interacting with the functionality.
 Sim: Ideal process adversary (simulator).

Local State:

Reg: A table of registered pairs (p_j, v_j) , initially empty.

Events:

request (Register, sid, v): A party registers a value v (e.g., public keys)
request (Retrieve, sid, p_j): A party requests to retrieve the value registered by process p_j

Functionality:

Upon receiving (Register, sid, v_i) from party p_i for the first time:
 Send (Registered, sid, p_i, v_i) to Sim.
 Upon receiving ok from Sim:
 store (sid, p_i, v_i) in Reg.

Upon receiving (Retrieve, sid, p_j) from party p_i :
 Send (Retrieve, sid, p_i, p_j) to Sim.
 Upon receiving ok from Sim:
 If there exists a recorded pair (sid, p_j, v_j) in Reg:
 Send (Retrieve, sid, v_j) to p_i .
 Else:
 Send (Retrieve, sid, $\perp$) to p_i .
